

Komputasi Matematis dan Eksperimen yang Dapat Direproduksi

B80 - Bahasa Indonesia

Floris

2026-08-22

Daftar Isi

1	Pengantar	9
1.1	Cara menggunakan buku ini	9
1.2	Penyiapan lokal	9
1.3	Kontrak setiap unit	11
1.4	Lisensi, atribusi, dan batas sumber	11
2	Primer P01 - Menjalankan Eksperimen Python	12
2.1	Tujuan belajar	12
2.2	Runtime lokal dan terminal	12
2.3	Ekspresi, nama, dan assignment	13
2.4	Lima jenis nilai skalar pertama	14
2.5	Panggilan fungsi, impor, dan bantuan	15
2.6	Sel Jupyter dan blok Quarto	16
2.7	Komputasi bukan tampilan	16
2.8	Membaca traceback	17
2.9	Skrip pertama dan rekaman lokal	17
2.10	Keadaan tersembunyi dan restart-run-all	18
2.11	Latihan	19
2.11.1	Latihan 1 - nilai, jenis, dan operator	19
2.11.2	Latihan 2 - impor, panggilan, dan bantuan	20
2.11.3	Latihan 3 - membaca dan memperbaiki traceback	21
2.11.4	Latihan 4 - komputasi, tampilan, dan skrip	22
2.11.5	Latihan 5 - kernel bersih dan keadaan tersembunyi	23
2.12	Ringkasan	25
3	Kontrol, Koleksi, Fungsi, Modul, dan Berkas	26
3.1	Tujuan belajar	26
3.2	Empat koleksi dasar	26
3.3	Percabangan dengan <code>if</code>	28
3.4	Perulangan eksplisit: <code>for</code> lalu <code>while</code>	29
3.4.1	Dari comprehension ke ekspresi generator	30
3.5	Fungsi, parameter, <code>return</code> , dan ruang lingkup	31
3.6	Pengecualian yang informatif	31
3.7	Modul dan batas <code>__main__</code>	33

3.8	Jalur dan tiga format berkas	33
3.8.1	Teks	34
3.8.2	CSV	34
3.8.3	JSON	34
3.9	Tes merah-hijau dengan <code>unittest</code>	35
3.10	Artefak pendamping deterministik	36
3.11	Latihan	37
3.11.1	Latihan 1 - koleksi dan irisan	37
3.11.2	Latihan 2 - aliran kontrol sebelum <code>comprehension</code>	38
3.11.3	Latihan 3 - fungsi, ruang lingkup, dan galat	40
3.11.4	Latihan 4 - modul peserta dan tes merah-hijau	41
3.11.5	Latihan 5 - teks, CSV, JSON, dan manifest	43
3.12	Ringkasan	44
4	Komputasi, Representasi, dan Bukti	46
4.1	Tujuan belajar	46
4.2	Tiga lapisan yang tidak boleh dicampur	46
4.3	Eksak dan titik-mengambang	47
4.4	Kontrak eksperimen yang dapat direproduksi	48
4.5	Bukti, bukti kerja, dan contoh penyangkal	48
4.6	Pengujian sebagai klaim terbatas	49
4.7	Latihan	49
4.7.1	Latihan 1 - representasi	49
4.7.2	Latihan 2 - spesifikasi toleransi	50
4.7.3	Latihan 3 - batas eksperimen	50
4.7.4	Latihan 4 - contoh penyangkal	50
4.7.5	Latihan 5 - rekaman eksperimen	51
4.8	Ringkasan	51
5	Objek, Nilai, dan Fungsi	52
5.1	Tujuan belajar	52
5.2	Jenis matematika dan tipe program	52
5.3	Nilai, identitas, dan alias	53
5.4	Fungsi sebagai kontrak	54
5.5	Fungsi murni dan perubahan keadaan	54
5.6	Invarian dan pengujian	55
5.7	Keluaran kanonis	55
5.8	Latihan	56
5.8.1	Latihan 1 - memilih representasi	56
5.8.2	Latihan 2 - nilai dan identitas	56
5.8.3	Latihan 3 - kontrak fungsi	56
5.8.4	Latihan 4 - alias yang berbahaya	57
5.8.5	Latihan 5 - tes dan bukti	57

5.9	Ringkasan	58
6	Array, Bentuk, dan Vektorisasi	59
6.1	Tujuan belajar	59
6.2	Array sebagai data beserta kontraknya	59
6.3	<code>dtype</code> adalah representasi, bukan himpunan matematika	60
6.4	Vektorisasi dan referensi skalar	60
6.5	Broadcasting dengan pemeriksaan bentuk	61
6.6	Salinan dan <i>view</i>	62
6.7	Eksperimen Unit 3	63
6.8	Komputasi dan bukti	63
6.9	Latihan	64
6.9.1	Latihan 1 - bentuk dan makna sumbu	64
6.9.2	Latihan 2 - vektorisasi dan invarian	64
6.9.3	Latihan 3 - kontrak broadcasting	65
6.9.4	Latihan 4 - melacak <i>view</i>	65
6.9.5	Latihan 5 - eksperimen bukan bukti umum	66
6.10	Ringkasan	66
7	Visualisasi dengan Integritas	67
7.1	Tujuan belajar	67
7.2	Gambar dimulai dari pertanyaan	67
7.3	Pengodean: apa berarti apa?	68
7.4	Lab bertahap: dari <code>fig</code> dan <code>ax</code> ke grafik lengkap	68
7.4.1	Tahap 1 - buat <code>fig</code> dan <code>ax</code>	69
7.4.2	Tahap 2 - <code>plot</code> untuk model	69
7.4.3	Tahap 3 - <code>scatter</code> untuk pengukuran	69
7.4.4	Tahap 4 - <code>errorbar</code> untuk ketidakpastian	70
7.4.5	Tahap 5 - label bersatuan dan legenda	70
7.4.6	Tahap 6 - deskripsi aksesibel dan penyimpanan deterministik	71
7.5	Sumbu, skala, dan satuan	73
7.6	Rasio aspek dan kemiringan	73
7.7	Ketidakpastian bukan dekorasi	74
7.8	Deskripsi tekstual dan aksesibilitas	74
7.9	Artefak deterministik	75
7.10	Bukti visual dan bukti matematis	75
7.11	Latihan	76
7.11.1	Latihan 1 - pertanyaan dan pengodean	76
7.11.2	Latihan 2 - sumbu terpotong	76
7.11.3	Latihan 3 - makna batang galat	76
7.11.4	Latihan 4 - deskripsi aksesibel	77
7.11.5	Latihan 5 - gambar dan bukti	77
7.11.6	Latihan penguasaan - data ke plot ke manifes	78

7.12	Ringkasan	80
8	Komputasi Eksak dan Simbolik	82
8.1	Tujuan belajar	82
8.2	Ekspresi bukan nilai	82
8.3	Asumsi dan domain adalah bagian dari soal	83
8.4	Kesamaan struktur dan ekuivalensi	84
8.5	Faktorisasi dan penyelesaian eksak	85
8.6	Hasil simbolik dan bukti	85
8.7	Keluaran kanonis	86
8.8	Lab SageMath lokal wajib	86
8.8.1	Parent menentukan tempat objek hidup	87
8.8.2	Coercion harus dapat dijelaskan	87
8.8.3	Gelang polinom dan faktorisasi eksak	88
8.8.4	Penyelesaian dalam SR dan residual eksak	88
8.8.5	Konversi pendekatan harus eksplisit	89
8.8.6	Rekaman Sage yang dapat diaudit	89
8.9	Latihan	90
8.9.1	Latihan 1 - ekspresi dan nilai	90
8.9.2	Latihan 2 - asumsi yang hilang	90
8.9.3	Latihan 3 - ekuivalensi dan domain	90
8.9.4	Latihan 4 - faktor, akar, dan pemeriksaan	91
8.9.5	Latihan 5 - artefak simbolik yang stabil	91
8.9.6	Latihan Sage 1 - parent, coercion, dan faktor	92
8.9.7	Latihan Sage 2 - solusi eksak dan pendekatan	93
8.10	Ringkasan	95
9	Titik-Mengambang, Galat, dan Stabilitas	97
9.1	Tujuan belajar	97
9.2	Model biner yang berhingga	97
9.3	ulp, epsilon, dan jarak yang berubah	98
9.4	Luapan, subnormal, dan aliran-bawah	99
9.5	Pembatalan dan reformulasi stabil	99
9.6	SciPy: <code>exprel</code> stabil dekat nol	100
9.7	Galat maju dan galat mundur	102
9.8	Pengondisian bukan stabilitas	103
9.9	Penjumlahan dan urutan operasi	103
9.10	Toleransi sebagai anggaran galat	104
9.11	Rekaman deterministik dan batas bukti	105
9.12	Latihan	105
9.12.1	Latihan 1 - ulp dan kenaikan yang hilang	105
9.12.2	Latihan 2 - membuang pembatalan	106
9.12.3	Latihan 3 - galat maju dan mundur	106

9.12.4	Latihan 4 - kondisi atau algoritme?	106
9.12.5	Latihan 5 - jumlah, toleransi, dan batas klaim	107
9.12.6	Latihan SciPy - stabilitas <code>exprel</code>	107
9.13	Ringkasan	111
10	Rancangan Eksperimen yang Dapat Diperiksa	112
10.1	Tujuan belajar	112
10.2	Pertanyaan dahulu, hasil kemudian	112
10.3	Variabel, kontrol, dan perancu	113
10.4	Kisi parameter dan aturan pemilihan	113
10.5	Benih dan nondeterminisme	114
10.6	Replikasi dan aturan berhenti	114
10.7	Pengembangan dan data holdout	115
10.8	Ukuran efek, bukan kesan gambar	115
10.9	Hasil negatif tetap merupakan hasil	116
10.10	Rekaman kanonis Unit 7	116
10.11	Eksperimen dan bukti	116
10.12	Latihan	117
10.12.1	Latihan 1 - variabel dan perancu	117
10.12.2	Latihan 2 - benih bukan mesin waktu	117
10.12.3	Latihan 3 - pemilihan dan holdout	118
10.12.4	Latihan 4 - ukuran efek dan aturan berhenti	118
10.12.5	Latihan 5 - hasil negatif dan batas bukti	119
10.13	Ringkasan	119
11	Pengujian dan Validasi Komputasi	120
11.1	Tujuan belajar	120
11.2	<i>Assertion</i> dan kontrak	120
11.3	Contoh dan sifat	120
11.4	Tes unit, integrasi, dan regresi	122
11.5	Oracle dan nilai referensi	122
11.6	Eksak dan bertoleransi	123
11.7	Nilai batas dan domain	124
11.8	Invarian dan pengujian metamorfik	124
11.9	Validasi independen	125
11.10	Laporan deterministik	125
11.11	Cakupan, bukti kerja, dan bukti	125
11.12	Latihan	126
11.12.1	Latihan 1 - memilih <i>assertion</i>	126
11.12.2	Latihan 2 - membangun oracle	126
11.12.3	Latihan 3 - hubungan metamorfik	127
11.12.4	Latihan 4 - tes regresi batas	127
11.12.5	Latihan 5 - batas bukti pengujian	127

11.13	Ringkasan	128
12	Data, Konfigurasi, dan Provenans	129
12.1	Tujuan belajar	129
12.2	Tiga lapisan data	129
12.3	Schema dan satuan	130
12.4	Konfigurasi terpisah dari kode	131
12.5	Identitas dan kunci lingkungan	131
12.6	Jalur relatif yang aman	132
12.7	Provenans dan hak per komponen	132
12.8	SHA-256 dan ukuran byte	133
12.9	Privasi dan pengecualian	133
12.10	Manifest run yang kanonis	133
12.11	Semantik dan byte	134
12.12	Latihan	135
12.12.1	Latihan 1 - mentah, turunan, dan garis keturunan	135
12.12.2	Latihan 2 - schema dan satuan	135
12.12.3	Latihan 3 - konfigurasi dan jalur aman	135
12.12.4	Latihan 4 - hak dan privasi	136
12.12.5	Latihan 5 - semantik, byte, dan hash	136
12.13	Ringkasan	137
13	Otomasi dan Alur Kerja yang Dapat Direproduksi	138
13.1	Tujuan belajar	138
13.2	Dari daftar perintah ke graf	138
13.3	Sumber dan artefak	139
13.4	Kapan tugas dianggap mutakhir	139
13.5	Gagal secara atomik	140
13.6	Target bernama dan pemeriksaan	140
13.7	Contoh alur kerja	140
13.8	Otomasi tidak memutuskan kebenaran	141
13.9	Latihan	141
13.9.1	Latihan 1 - urutan tugas	141
13.9.2	Latihan 2 - siklus	141
13.9.3	Latihan 3 - cap waktu	142
13.9.4	Latihan 4 - kegagalan penulisan	142
13.9.5	Latihan 5 - alur kerja hijau	142
13.10	Ringkasan	143
14	Eksperimen Numerik Berprasyarat	144
14.1	Tujuan belajar	144
14.2	Metode datang bersama teorema	144
14.3	Jalur A30: bagi-dua	145

14.4	Jalur A30: pembandingan SciPy yang dapat dieksekusi	145
14.5	Jalur B30: kuadratur dan pemurnian	147
14.6	Jalur B40: sistem linear dan residual	147
14.7	Jalur B70: langkah Euler	147
14.8	Empat sumber ketidakcocokan	148
14.9	Artefak eksperimen	148
14.10	Latihan	148
14.10.1	Latihan 1 - invarian bagi-dua	148
14.10.2	Latihan 2 - batas iterasi	149
14.10.3	Latihan SciPy - dua implementasi pada akar kubik	149
14.10.4	Latihan 3 - pemeriksaan kuadratur	151
14.10.5	Latihan 4 - residual bukan galat solusi	152
14.10.6	Latihan 5 - eksperimen Euler	152
14.11	Ringkasan	153
15	Proyek Akhir yang Dapat Diperiksa	154
15.1	Tujuan belajar	154
15.2	Pertanyaan sebelum perangkat	154
15.3	Tiga jenis klaim	155
15.4	Paket reproduksi minimum	155
15.5	Reproduksi buta terbatas	156
15.6	Tinjauan dan revisi	156
15.7	Rubrik kelulusan	157
15.8	Latihan	157
15.8.1	Latihan 1 - klasifikasi klaim	157
15.8.2	Latihan 2 - manifest yang belum lengkap	158
15.8.3	Latihan 3 - jalur berbahaya	158
15.8.4	Latihan 4 - reproduksi tidak identik	158
15.8.5	Latihan 5 - tanggapan tinjauan	159
15.9	Ringkasan	159

1 Pengantar

1.1 Cara menggunakan buku ini

Buku ini mempelajari komputasi sebagai bagian dari kegiatan matematika: merumuskan pertanyaan, merepresentasikan objek, menjalankan eksperimen, memeriksa program, menyimpan bukti kerja, dan menjelaskan apa yang benar-benar ditunjukkan oleh hasilnya.

Prasyarat formalnya hanya kompetensi A30: aljabar, fungsi, dan trigonometri pada tingkat prakilas. Dua primer wajib memperkenalkan Python dari awal sebelum kedua belas unit utama. Topik kalkulus, aljabar linear, peluang, dan persamaan diferensial akan dipakai setelah pembaca mencapai mata kuliah terkait; topik itu tidak diselundupkan sebagai prasyarat Unit 1.

Setiap eksperimen dalam buku ini harus dapat dijalankan secara lokal dengan perangkat lunak terbuka. Sebuah hasil komputasi belum menjadi bukti hanya karena keluar dari komputer. Sebaliknya, komputasi dapat menemukan contoh penyangkal, menguji implementasi, menuntun dugaan, dan menghasilkan bukti kerja yang dapat diperiksa orang lain.

Edisi ini adalah versi 2026.08.22.1, dengan DOI <https://doi.org/10.5281/zenodo.22053905>. Rilis 12-unit 2026.08.22 tetap tersedia sebagai [versi historis](#) dan tidak ditimpa oleh perluasan ini.

1.2 Penyiapan lokal

Jalankan perintah dari akar paket sumber. Untuk profil Python standar pada Windows PowerShell, buat lingkungan terpisah, aktifkan, pasang seluruh versi yang dibekukan, lalu perlihatkan kernelspec milik proyek kepada Jupyter:

```
py -3.13 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r environment/python-lock.txt
$env:JUPYTER_PATH = (Resolve-Path build-support/jupyter).Path
python -m jupyter kernelspec list
```

Pada Linux atau macOS, padanannya ialah:

```
python3.13 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-build.txt
export
JUPYTER_PATH="$(pwd)/build-support/jupyter${JUPYTER_PATH:+:$JUPYTER_PATH}"
python -m jupyter kernelspec list
```

Daftar harus memuat `o002-frozen`. Berkas `requirements-build.txt` mendaftarkan akar dependensi lintas platform; `environment/python-lock.txt` memuat penutupan 95 paket Windows yang dipakai untuk rilis. Profil POSIX yang dihasilkan dari akar dependensi harus dicatat sebagai profil kompatibel, bukan byte-identik dengan lock Windows. Pada profil Windows, periksa kecocokan runtime dengan:

```
python scripts/python_environment.py verify --lock
environment/python-lock.txt --receipt
environment/PYTHON_ENVIRONMENT_RECEIPT.json
```

Quarto diperlukan untuk membangun pembaca, dan distribusi TeX diperlukan untuk PDF. Periksa keduanya dengan `quarto --version` dan `lualatex --version`. Eksperimen Python biasa tidak memerlukan TeX.

Lab SageMath memakai profil terpisah agar paket Sage tidak bercampur dengan lingkungan Python standar. Pada Windows, siapkan WSL Ubuntu 22.04 dan paket SageMath 9.5 dari repositori Ubuntu tersebut:

```
wsl.exe --install -d Ubuntu-22.04
wsl.exe -d Ubuntu-22.04 -- bash -lc "sudo apt-get update && sudo apt-get
install -y sagemath=9.5-4 python3-sage=9.5-4"
wsl.exe -d Ubuntu-22.04 -- /usr/bin/sage --version
python scripts/sage_environment.py verify --lock
environment/sage-ubuntu22.04-dpkg-lock.txt --receipt
environment/SAGE_ENVIRONMENT_RECEIPT.json
```

Jika verifikasi lock menolak profil lokal, profil itu boleh dipakai untuk belajar tetapi tidak boleh disebut byte-identik dengan lingkungan rilis. Receipt mencatat Ubuntu 22.04, SageMath 9.5, dan inventaris 1.064 paket yang benar-benar diuji. Tidak ada layanan berbayar atau runtime jarak jauh yang diperlukan.

1.3 Kontrak setiap unit

Setiap unit memuat:

1. tujuan belajar dan prasyarat lokal;
2. teks pembaca dengan identitas stabil;
3. kode yang dapat dijalankan tanpa layanan berbayar;
4. pengujian otomatis untuk klaim tentang program;
5. latihan, petunjuk, jawaban, dan solusi, dengan pemeriksaan yang dapat dijalankan pada lima belas latihan penguasaan baru;
6. catatan tentang batas antara eksperimen, bukti, dan kesimpulan; serta
7. rekaman versi, lingkungan, dan hash artefak pada batas produksi.

Lingkungan standar direkam dalam [lock Python lengkap](#) dan [receipt verifikasinya](#). Praktikum SageMath memakai profil lokal terpisah yang direkam dalam [lock Sage](#) dan [receipt Sage](#). Nama paket dan versi di sana merupakan bagian dari eksperimen, bukan sekadar saran pemasangan.

1.4 Lisensi, atribusi, dan batas sumber

Keempat belas unit dalam edisi ini ditulis secara mandiri. Patrick Walls, *Mathematical Python*; Irving dan rekan-rekan, *Research Software Engineering with Python*; *Scientific Python Lectures*; Hans Fangohr, *Introduction to Python for Computational Science and Engineering*; serta *Official Sage Tutorial 10.9* dipakai untuk membandingkan cakupan dan praktik produksi, bukan sebagai sumber prosa, latihan, gambar, data, atau kode yang disalin maupun diadaptasi ke edisi ini. Penyebutan mereka tidak menyiratkan dukungan atau pengesahan.

Teks asli edisi ini berlisensi **CC BY-SA 4.0**; kode asli berlisensi **MIT**. Pemakaian ulang harus mempertahankan atribusi, pemberitahuan lisensi, dan catatan perubahan yang diwajibkan.

Berkas lengkap tersedia sebagai [lisensi teks](#), [lisensi kode](#), [batas materi pihak ketiga](#), dan [lisensi runtime web](#).

Materi donor yang mungkin dipakai pada revisi mendatang tetap mengikuti lisensi komponennya sendiri dan tidak otomatis tercakup oleh lisensi edisi ini.

2 Primer P01 - Menjalankan Eksperimen Python

2.1 Tujuan belajar

Setelah menyelesaikan primer wajib ini, pembaca dapat:

- memastikan bahwa Python yang dipakai berasal dari lingkungan lokal yang dimaksud dan menjalankan perintah dari terminal;
- menulis ekspresi, mengikat nilai pada nama, dan mengenali `int`, `float`, `str`, `bool`, serta `None`;
- memakai operator, memanggil fungsi, mengimpor modul, dan mencari bantuan;
- membedakan sel Jupyter, blok kode Quarto, dan skrip Python;
- membaca bagian penting traceback tanpa menebak-nebak;
- membedakan nilai yang dihitung dari teks atau tampilan yang diperlihatkan; dan
- menemukan keadaan notebook yang tersembunyi dengan memulai ulang kernel dan menjalankan semua sel dari awal.

Primer ini tidak menganggap pengalaman pemrograman sebelumnya. Anda hanya memerlukan operasi aritmetika dasar. Semua kode pendamping primer ini asli, berjalan lokal dengan pustaka standar Python, dan berada di `source/code/primer01_execution.py`.

2.2 Runtime lokal dan terminal

Runtime adalah program yang benar-benar mengeksekusi kode. Untuk unit ini, runtime itu adalah Python lokal. Terminal adalah jendela teks tempat kita meminta sistem menjalankan program. Jupyter dan Quarto memakai runtime yang sama jika keduanya diarahkan ke lingkungan yang sama.

Dari akar proyek, periksa runtime dengan perintah berikut. Ketik perintahnya, bukan tanda prompt yang mungkin muncul di sebelah kirinya.

```
python --version
python -c "print(2 + 3)"
```

Perintah pertama seharusnya menyebut Python 3. Perintah kedua meminta Python menghitung $2 + 3$ lalu menulis 5 ke terminal. Jika `python` tidak ditemukan, jangan memasang paket secara acak. Ikuti dahulu [panduan penyiapan lokal](#), aktifkan lingkungan kursus, lalu periksa lagi versi yang aktif.

Tiga pertanyaan awal mencegah banyak hasil yang tidak dapat direproduksi:

1. Perintah apa yang dijalankan?
2. Dari direktori kerja mana perintah itu dijalankan?
3. Runtime dan lingkungan mana yang menerima perintah tersebut?

Jalur relatif seperti `source/code/primer01_execution.py` ditafsirkan dari direktori kerja. Karena itu, contoh dalam buku ini dijalankan dari akar proyek.

2.3 Ekspresi, nama, dan assignment

Ekspresi meminta Python menghasilkan nilai. Angka 4, ekspresi $2 + 3$, dan panggilan `len("data")` masing-masing menghasilkan nilai. Operator `=` adalah **assignment**: Python menghitung ruas kanan, lalu mengikat hasilnya pada nama di ruas kiri.

```
panjang = 8
lebar = 5
luas = panjang * lebar
luas
```

Daftar 2.1

40

Nama `luas` bukan kotak permanen. Assignment berikutnya dapat mengikat nama yang sama pada nilai baru. Untuk memeriksa kesamaan nilai, gunakan `==`, bukan `=`.

```
luas == 40
```

Daftar 2.2

True

Pilih nama yang menyatakan makna, misalnya `jumlah_sampel`, bukan nama kabur seperti `x1`, kecuali simbol pendek itu memang bagian dari model matematika.

2.4 Lima jenis nilai skalar pertama

Nilai skalar awal yang sering dipakai adalah:

Jenis	Contoh	Maksud awal
int	8	bilangan bulat
float	0.01	pendekatan bilangan real
str	"uji A"	teks
bool	True	nilai logika benar atau salah
NoneType	None	belum ada nilai atau tidak ada hasil

```
jumlah = 8
toleransi = 0.01
label = "uji A"
lulus = jumlah > 5
catatan = None

type(jumlah).__name__
```

Daftar 2.3

'int'

Panggil `type(...)` dengan nama lain untuk memeriksa jenisnya satu per satu. Daftar, tuple, pengulangan, dan comprehension sengaja baru diperkenalkan di P02; primer ini belum memerlukannya.

`None` berbeda dari 0, `False`, dan string kosong `""`. Ia menyatakan bahwa tidak ada nilai yang diberikan. Jangan memakai `None` sebagai angka.

Operator aritmetika dasar ialah `+`, `-`, `*`, `/`, `//`, `%`, dan `**`. Pembagian `/` menghasilkan `float`; `//` adalah pembagian lantai; `%` memberi sisa yang konsisten dengan pembagian lantai; dan `**` adalah perpangkatan.

```
hasil_tambah = 7 + 2
hasil_kurang = 7 - 2
hasil_kali = 7 * 2
hasil_bagi = 7 / 2
hasil_bagi_lantai = 7 // 2
hasil_sisa = 7 % 2
hasil_pangkat = 7**2
```

```
hasil_pangkat
```

Daftar 2.4

49

Ketujuh nama itu tetap menyimpan nilai skalar. Tidak ada koleksi yang dibuat; daftar dan tuple baru diperkenalkan di P02.

Perbandingan `==`, `!=`, `<`, `<=`, `>`, dan `>=` menghasilkan `bool`. Operator `and`, `or`, dan `not` menggabungkan nilai logika. Pakai tanda kurung ketika urutan operasi perlu tampak jelas bagi pembaca.

2.5 Panggilan fungsi, impor, dan bantuan

Fungsi menerima argumen dan dapat mengembalikan nilai. Dalam `round(3.14159, ndigits=3)`, `3.14159` adalah argumen posisional dan `ndigits=3` adalah argumen kata kunci.

```
round(3.14159, ndigits=3)
```

Daftar 2.5

3.142

Modul mengelompokkan nama yang dapat dipakai ulang. Impor modul `math`, lalu sebut nama modul agar asal fungsi tetap terlihat.

```
import math

jari_jari = 3
luas_lingkaran = math.pi * jari_jari**2
luas_lingkaran
```

Gunakan `help(math.sqrt)` pada Python atau `math.sqrt?` di sel Jupyter untuk melihat dokumentasi lokal. Bantuan menjelaskan kontrak fungsi, bukan menjamin bahwa fungsi tersebut cocok untuk pertanyaan Anda. Periksa arti argumen, satuan, domain, nilai kembali, dan kemungkinan galat.

Daftar 2.6

28.274333882308138

2.6 Sel Jupyter dan blok Quarto

Notebook Jupyter mempunyai sel kode dan sel Markdown. Sel kode dikirim ke kernel Python; sel Markdown menyimpan uraian, persamaan, dan hubungan antara pertanyaan dengan hasil. Quarto memakai pagar kode dalam berkas `.qmd` dan dapat menjalankannya saat membangun HTML atau PDF.

Sumber blok kode Quarto tampak seperti ini:

```
~~~{python}
nilai = 6 * 7
nilai
~~~
```

Menjalankan satu sel tidak otomatis menjalankan semua sel di atasnya. Kernel menyimpan nama selama proses masih hidup. Nomor eksekusi sel menunjukkan urutan yang benar-benar dijalankan; posisi sel pada halaman hanya menunjukkan urutan sumber. Perbedaan ini penting bagi reproduksibilitas.

2.7 Komputasi bukan tampilan

Python dapat menghitung nilai tanpa menampilkannya. Di notebook, nilai ekspresi terakhir biasanya dikirim ke mekanisme tampilan. Assignment biasanya tidak menampilkan representasi nilai. `print(...)` menulis teks ke keluaran standar. Di skrip biasa, ekspresi telanjang seperti `2 + 3` tetap dihitung tetapi tidak otomatis dicetak.

Daftar 2.7

```
nilai_hitung = 2 + 3
teks_tampilan = f"2 + 3 = {nilai_hitung}"
print(teks_tampilan)
```

2 + 3 = 5

`nilai_hitung` adalah `int` bernilai 5. `teks_tampilan` adalah `str` yang dipilih untuk komunikasi. Tampilan yang bagus tidak membuktikan bahwa komputasi benar, dan nilai di memori belum menjadi artefak tersimpan. Jika hasil perlu diaudit, simpan data dan metadata dalam berkas yang ditentukan.

Quarto dapat menangkap keluaran sel untuk dimasukkan ke pembaca. Keluaran itu masih harus ditafsirkan: apakah ia nilai, teks diagnostik, gambar, atau pesan galat?

2.8 Membaca traceback

Ketika Python gagal, ia menampilkan **traceback**. Baca baris terakhir dahulu: baris itu memberi jenis galat dan pesan ringkas. Lalu cari frame terdekat yang menunjuk berkas serta baris kode Anda. Frame di atasnya menjelaskan jalur panggilan yang membawa eksekusi ke sana.

```
Traceback (most recent call last):
  File "eksperimen.py", line 2, in <module>
    hasil = data * 2
NameError: name 'data' is not defined
```

`NameError` berarti nama belum terikat pada ruang nama yang sedang dipakai. `TypeError` biasanya berarti operasi menerima jenis nilai yang tidak sesuai. `ZeroDivisionError` menyatakan pembagian dengan nol. `SyntaxError` berarti sumber tidak dapat diurai sebagai program Python. Pesan itu adalah bukti awal, bukan alasan untuk menghapus baris secara acak.

Pada P01, jalankan contoh yang gagal di konsol atau sel latihan, baca traceback, lalu perbaiki sebabnya. Sintaks `try/except` baru diajarkan di P02; blok pemeriksa yang disediakan buku boleh memakainya sebagai scaffolding, tetapi Anda belum diminta menulis struktur itu sendiri. Jangan menyembunyikan seluruh program hanya agar keluaran terlihat hijau.

2.9 Skrip pertama dan rekaman lokal

Berkas `.py` adalah sumber skrip yang dapat dijalankan ulang. Dari akar proyek, jalankan pendamping P01:

```
python source/code/primer01_execution.py --output output/p01-results.json
```

Skrip menghitung contoh yang sama setiap kali dan menulis rekaman JSON. Ia juga mencetak satu pesan lokasi agar manusia tahu berkas mana yang dibuat. Buka `output/p01-results.json` sebagai teks. Medan `computed_value` menyimpan nilai komputasi, sedangkan `display_text`

menyimpan teks yang dipilih untuk pembaca. Medan runtime menyebut implementasi dan versi Python yang benar-benar menjalankan skrip.

Untuk sekarang, cukup pahami bahwa terminal menjalankan berkas dari awal sampai akhir. Struktur modul dan bagian utama skrip akan dibahas dalam P02.

2.10 Keadaan tersembunyi dan restart-run-all

Perhatikan dua sel berikut.

```
# Sel A
data = 7

# Sel B
hasil = data * 2
```

Jika Sel A pernah dijalankan, Sel B dapat berhasil walaupun Sel A kemudian dipindahkan, diubah, atau dihapus dari notebook. Nilai lama `data` masih hidup di kernel. Notebook tampak benar, tetapi sumber yang terlihat tidak cukup untuk menghasilkan hasil tersebut. Inilah **keadaan tersembunyi**.

Pemeriksaan wajib sebelum menerima notebook:

1. simpan sumber;
2. pilih **Restart Kernel and Run All Cells**;
3. jalankan sel dalam urutan sumber dari ruang nama bersih;
4. pastikan tidak ada input manual yang tidak dicatat; dan
5. bandingkan artefak yang dihasilkan dengan hasil yang diharapkan.

Notebook pendamping menyediakan tiga sel keadaan yang dapat dijalankan langsung dari antarmuka Jupyter. Uji wajibnya ialah: jalankan sel terakhir dengan keadaan lama, mulai ulang kernel, buktikan sel itu gagal sendiri, lalu pilih **Restart Kernel and Run All Cells** dan buktikan urutan sumber lengkap berhasil. Pendamping kode juga mempunyai `run_cells` untuk suite pengujian; tuple, dictionary, dan penanganan exception di dalam scaffolding itu dibahas di P02.

Keberhasilan restart-run-all membuktikan bahwa urutan sel yang direkam dapat dieksekusi pada lingkungan saat ini. Ia belum membuktikan bahwa model matematikanya tepat atau bahwa lingkungan lain akan memberi byte identik.

Latihan yang sama tersedia sebagai [notebook P01 untuk kernel bersih](#).

Notebook itu dapat dijalankan lokal dan menyimpan ketiga pemeriksaan dalam urutan sumber yang lengkap; metadata kernelspec-nya menunjuk lingkungan beku o002-frozen milik proyek.

2.11 Latihan

2.11.1 Latihan 1 - nilai, jenis, dan operator

Buat lima nama: `jumlah=8`, `toleransi=0.01`, `label="uji A"`, nilai logika `lulus` yang memeriksa apakah `jumlah > 5`, dan `catatan=None`. Hitung pula hasil bagi lantai dan sisa pembagian `jumlah` oleh 3. Jalankan kodenya.

Petunjuk

Gunakan `//` untuk hasil bagi lantai, `%` untuk sisa, dan `type(...)` untuk memeriksa jenis nilai.

Pemeriksaan mandiri

Tambahkan pemeriksaan berikut di bawah kode Anda. Sel harus selesai tanpa `AssertionError`.

```
assert type(jumlah) is int
assert type(toleransi) is float
assert type(label) is str
assert type(lulus) is bool and lulus is True
assert catatan is None
assert jumlah // 3 == 2
assert jumlah % 3 == 2
```

Jawaban dan solusi

Pemecahan 2.1.

```

jumlah = 8
toleransi = 0.01
label = "uji A"
lulus = jumlah > 5
catatan = None
hasil_bagi = jumlah // 3
sisas = jumlah % 3

assert hasil_bagi == 2
assert sisas == 2

```

Setiap nama mengikat nilai yang dihasilkan ruas kanannya. Perbandingan `jumlah > 5` menghasilkan `bool`; `None` bukan angka nol.

2.11.2 Latihan 2 - impor, panggilan, dan bantuan

Impor modul `math`, lalu hitung luas lingkaran berjari-jari 3 dengan fungsi `circle_area` dari pendamping P01. Gunakan bantuan lokal untuk menemukan arti `math.isclose`, kemudian bandingkan hasil fungsi dengan `math.pi * 3**2`.

Petunjuk

Impor pendamping dengan `from source.code.primero1_execution import circle_area`. Panggil `help(math.isclose)` di konsol terpisah jika keluarannya panjang.

Pemeriksaan mandiri

```

import math
from source.code.primero1_execution import circle_area

luas_latihan = circle_area(3)
assert math.isclose(luas_latihan, math.pi * 9, rel_tol=0.0,
abs_tol=1e-15)

```

💡 Jawaban dan solusi

Pemecahan 2.2.

```
import math
from source.code.primier01_execution import circle_area

jari_jari_latihan = 3
luas_latihan = circle_area(jari_jari_latihan)
assert math.isclose(
    luas_latihan,
    math.pi * jari_jari_latihan**2,
    rel_tol=0.0,
    abs_tol=1e-15,
)
```

Impor menyediakan nama modul atau fungsi. Panggilan memberi argumen 3 dan mengembalikan nilai. Dokumentasi `isclose` menjelaskan toleransi relatif dan absolut yang menjadi bagian dari pemeriksaan.

2.11.3 Latihan 3 - membaca dan memperbaiki traceback

Panggil `circle_area(-2)`, baca traceback, dan sebutkan jenis galat serta pesan terakhirnya. Lalu perbaiki masukan menjadi 2 tanpa mengubah fungsi hanya untuk menyembunyikan galat.

i Petunjuk

Baris terakhir menyebut `ValueError`. Jalankan kegagalan di konsol terpisah. Blok pemeriksaan berikut adalah scaffolding yang disediakan; sintaks `try/except`-nya akan dipelajari di P02.

! Pemeriksaan mandiri

```
from source.code.primero1_execution import circle_area

try:
    circle_area(-2)
except ValueError as galat_radius:
    assert str(galat_radius) == "radius tidak boleh negatif"
else:
    raise AssertionError("masukan negatif seharusnya ditolak")

assert circle_area(2) > 0
```

💡 Jawaban dan solusi

Jenisnya `ValueError` dan pesannya `radius tidak boleh negatif`. Fungsi menolak masukan di luar kontraknya. Solusi yang benar ialah memberi jari-jari taknegatif atau memperbaiki sumber data, bukan menghapus pemeriksaan.

Pemecahan 2.3.

```
from source.code.primero1_execution import circle_area

jari_jari_sah = 2
luas_sah = circle_area(jari_jari_sah)
assert luas_sah > 0
```

2.11.4 Latihan 4 - komputasi, tampilan, dan skrip

Jalankan skrip P01 dari terminal. Dalam Python, panggil `computation_and_display(8, 5)`. Tunjukkan medan yang menyimpan hasil hitung dan medan yang hanya menyimpan teks tampilan. Jelaskan mengapa mencetak teks tidak sama dengan menyimpan rekaman JSON.

i Petunjuk

Nilai hitung bertipe `int`; tampilan bertipe `str`. `print` mengirim teks ke keluaran standar, sedangkan skrip menulis byte ke jalur yang dinyatakan. Pengindeksan dictionary pada blok pemeriksaan disediakan sebagai scaffolding dan akan dijelaskan di P02.

! Pemeriksaan mandiri

```
from source.code.primero1_execution import computation_and_display

rekaman_tampilan = computation_and_display(8, 5)
assert rekaman_tampilan["computed_value"] == 13
assert rekaman_tampilan["display_text"] == "8 + 5 = 13"
assert type(rekaman_tampilan["computed_value"]) is int
assert type(rekaman_tampilan["display_text"]) is str
```

💡 Jawaban dan solusi

Pemecahan 2.4.

```
from source.code.primero1_execution import computation_and_display

rekaman_tampilan = computation_and_display(8, 5)
nilai_yang_dihitung = rekaman_tampilan["computed_value"]
teks_yang_ditampilkan = rekaman_tampilan["display_text"]
assert nilai_yang_dihitung == 13
assert teks_yang_ditampilkan.endswith("= 13")
```

Teks terminal dapat hilang ketika terminal ditutup dan tidak mengikat versi runtime atau parameter. Berkas JSON dari skrip merupakan artefak yang dapat dibaca ulang, diperiksa, dan di-hash. Keduanya tetap harus dinilai terhadap pertanyaan matematika yang sebenarnya.

2.11.5 Latihan 5 - kernel bersih dan keadaan tersembunyi

Gunakan notebook P01 untuk menunjukkan tiga fakta: Sel B berhasil pada kernel lama yang telah menjalankan Sel A; Sel B gagal dengan `NameError` pada kernel bersih; dan urutan A lalu B berhasil setelah **Restart Kernel and Run All Cells**. Ini adalah latihan penguasaan wajib.

i Petunjuk

Buka `source/notebooks/o002-p01-clean-kernel.ipynb`. Ikuti tiga keadaan yang ditandai di dalam notebook dan jangan menambahkan nilai manual yang tidak tercatat. Blok pemeriksaan di bawah adalah scaffolding tes proyek yang cukup dijalankan di P01;

list, Path, atribut, argumen kata kunci, subprocess, dan sys yang dipakainya dibahas di P02 atau unit proyek berikutnya.

! Pemeriksaan mandiri

```
from pathlib import Path
import subprocess
import sys

tes_kernel = subprocess.run(
    [
        sys.executable,
        "-m",
        "unittest",
        "tests.test_primer01.Primer01NotebookKernelTests",
    ],
    cwd=Path.cwd(),
    capture_output=True,
    text=True,
)
assert tes_kernel.returncode == 0, tes_kernel.stdout + tes_kernel.stderr
```

💡 Jawaban dan solusi

```
assert "OK" in tes_kernel.stderr
hasil_pemeriksaan_notebook = "kernel aktual: gagal di luar urutan;
run-all bersih lulus"
hasil_pemeriksaan_notebook
```

Pemecahan 2.5.

```
'kernel aktual: gagal di luar urutan; run-all bersih lulus'
```

Tes memulai kernel Jupyter sungguhan. Sel B saja gagal dengan `NameError`, sedangkan notebook lengkap menjalankan Sel A lalu Sel B dari kernel baru dan menghasilkan 14. Urutan sumber, bukan riwayat klik lama, adalah bukti yang dapat dijalankan ulang.

2.12 Ringkasan

- Terminal menjalankan perintah dengan runtime dan direktori kerja tertentu.
- Ekspresi menghasilkan nilai; assignment mengikat nilai itu pada nama.
- `int`, `float`, `str`, `bool`, dan `None` mempunyai peran berbeda.
- Fungsi dipanggil dengan argumen; modul diimpor; bantuan lokal menjelaskan kontrak yang harus diperiksa.
- Notebook dan Quarto menjalankan sel, sedangkan skrip menjalankan berkas dari awal. Urutan eksekusi merupakan bagian dari eksperimen.
- Nilai komputasi, teks tampilan, dan artefak tersimpan bukan hal yang sama.
- Traceback memberi lokasi, jenis, dan pesan galat yang harus dibaca sebelum memperbaiki kode.
- Restart Kernel and Run All Cells mendeteksi keadaan tersembunyi, tetapi tidak menggantikan pemeriksaan matematika atau pembekuan lingkungan.

3 Kontrol, Koleksi, Fungsi, Modul, dan Berkas

3.1 Tujuan belajar

Setelah menyelesaikan primer ini, pembaca dapat:

- memilih `list`, `tuple`, `dict`, atau `set` sesuai dengan maksud data;
- membaca dan mengubah koleksi melalui indeks dan irisan;
- menulis percabangan serta perulangan `for` dan `while` secara eksplisit sebelum meringkasnya menjadi *comprehension*;
- mendefinisikan fungsi dengan parameter, nilai kembalian, dan ruang lingkup lokal yang jelas;
- menolak masukan yang tidak sah dengan pengecualian yang informatif;
- memisahkan kode yang dapat diimpor dari tindakan baris perintah melalui modul dan `if __name__ == "__main__"`;
- membaca dan menulis teks, CSV, dan JSON memakai `Path`; dan
- menjalani satu siklus tes merah-hijau nyata dengan `unittest`.

Prasyarat lokal: Primer P01. Primer ini tidak menganggap pembaca pernah memprogram sebelumnya dan tidak memerlukan kalkulus, aljabar linear, atau persamaan diferensial.

3.2 Empat koleksi dasar

Sebuah koleksi menyimpan beberapa nilai, tetapi jenis koleksi menyatakan kontrak yang berbeda.

Jenis	Contoh	Kapan dipilih	Sifat penting
<code>list</code>	<code>[4, 1, 4]</code>	urutan yang dapat diubah	berurutan, boleh duplikat
<code>tuple</code>	<code>(4, 1, 4)</code>	nilai berurutan yang hendak diperlakukan tetap	berurutan, tidak dapat diubah
<code>dict</code>	<code>{"S01": 4}</code>	pemetaan kunci ke nilai	setiap kunci unik

Jenis	Contoh	Kapan dipilih	Sifat penting
set	{1, 4}	keanggotaan atau penghapusan duplikat	tidak mempunyai indeks

Gunakan kurung siku untuk mengambil unsur dari daftar atau tuple. Indeks Python dimulai dari nol; indeks negatif dihitung dari akhir.

```
values = [4, 1, 4, 2]
first = values[0]
last = values[-1]
middle = values[1:-1]
(first, last, middle)
```

Daftar 3.1

(4, 2, [1, 4])

Irisan `values[start:stop]` memasukkan posisi `start`, tetapi berhenti sebelum `stop`. Karena itu, `values[1:3]` mengambil posisi 1 dan 2. Bentuk `values[:]` membuat daftar baru dengan nilai yang sama; ia bukan nama kedua bagi daftar lama.

Daftar dapat diubah. Metode `append` menambah satu unsur ke daftar yang sama. Tuple tidak menyediakan perubahan itu.

```
measurements = [2, 5]
measurements.append(7)
frozen_measurements = tuple(measurements)
(measurements, frozen_measurements)
```

Daftar 3.2

([2, 5, 7], (2, 5, 7))

Dictionary memakai kunci, bukan posisi, untuk mengambil nilai. Metode `get` dapat menyediakan nilai bawaan ketika kunci tidak ada. Gunakan pengindeksan biasa jika ketiadaan kunci justru merupakan galat yang harus terlihat.

```
by_sample = {"S01": 2, "S02": 5, "S03": 2}
sample_ids = set(by_sample)
unique_values = set(by_sample.values())
(by_sample["S02"], by_sample.get("S99"), sample_ids, unique_values)
```

Daftar 3.3

```
(5, None, {'S01', 'S02', 'S03'}, {2, 5})
```

Jangan mengandalkan urutan tampilan sebuah set. Jika hasil harus stabil untuk dibaca atau di-*hash*, ubah ke daftar terurut, misalnya `sorted(unique_values)`. Dictionary Python mempertahankan urutan penyisipan, tetapi JSON kanonis pada proyek ini tetap mengurutkan kunci agar urutan kebetulan pembuatan dictionary tidak mengubah byte keluaran.

3.3 Percabangan dengan if

Percabangan memilih tindakan berdasarkan nilai boolean. Python hanya menjalankan blok pertama yang kondisinya benar.

```
residual = 0.0008
tolerance = 0.001

if residual < 0:
    status = "galat: residual tidak boleh negatif"
elif residual <= tolerance:
    status = "lulus pada toleransi yang ditetapkan"
else:
    status = "belum lulus"

status
```

Daftar 3.4

```
'lulus pada toleransi yang ditetapkan'
```

Indentasi menentukan blok. Empat spasi adalah kebiasaan proyek ini. Kondisi `residual <= tolerance` merupakan klaim yang dapat diperiksa untuk satu hasil; ia tidak membuktikan bahwa algoritme benar untuk semua masukan.

Operator perbandingan menghasilkan **True** atau **False**. Gabungkan kondisi dengan **and**, **or**, dan **not**, tetapi jangan membuat satu baris begitu padat hingga aturan domain sulit ditinjau.

3.4 Perulangan eksplisit: **for** lalu **while**

Gunakan **for** ketika hendak mengunjungi setiap unsur suatu koleksi. Contoh berikut memilih bilangan genap lalu menyimpan kuadratnya.

```
values = [5, 2, -4, 3, 0]
even_squares = []

for value in values:
    if value % 2 == 0:
        even_squares.append(value * value)

even_squares
```

Daftar 3.5

[4, 16, 0]

Keadaan loop dapat dibaca langkah demi langkah: **value** adalah unsur yang sedang diperiksa dan **even_squares** menyimpan hasil yang telah diterima. Bentuk eksplisit ini sebaiknya dikuasai sebelum bentuk ringkas berikut.

```
same_result = [value * value for value in values if value % 2 == 0]
same_result == even_squares
```

Daftar 3.6

True

List comprehension bukan algoritme baru; di sini ia meringkas loop dan percabangan yang sama. Jika transformasi memerlukan beberapa kondisi, penanganan galat, atau pencatatan antara, loop eksplisit biasanya lebih jelas.

3.4.1 Dari comprehension ke ekspresi generator

Kurung siku membangun seluruh list saat itu juga. Bentuk yang sama di dalam kurung biasa membuat *ekspresi generator*: nilai diproduksi satu per satu saat dikonsumsi. Pemanggilan `tuple(...)` berikut menghabiskan generator dan menyimpan hasilnya sebagai tuple.

```
generated = (value * value for value in values if value % 2 == 0)
frozen_result = tuple(generated)
(frozen_result, tuple(generated))
```

Daftar 3.7

`((4, 16, 0), ())`

Hasil kedua adalah tuple kosong karena generator hanya dapat dilalui satu kali. Jadi `[expression for ...]` menghasilkan list segera, sedangkan `tuple(expression for ...)` mengonsumsi aliran malas lalu membekukannya. Unit 2 memakai bentuk kedua ketika ingin menerima iterable apa pun tanpa menyimpan dua salinan perantara.

Gunakan `while` ketika pengulangan diteruskan selama suatu kondisi masih benar. Harus ada kemajuan yang membuat kondisi akhirnya salah.

```
current = 3
countdown = []

while current > 0:
    countdown.append(current)
    current -= 1

countdown.append(0)
countdown
```

Daftar 3.8

`[3, 2, 1, 0]`

Pada setiap putaran, `current` berkurang satu dan dibatasi bawah oleh nol. Fakta itu menjelaskan mengapa loop berhenti. Jika baris `current -= 1` hilang, kondisi selalu benar dan loop tidak selesai.

3.5 Fungsi, parameter, return, dan ruang lingkup

Fungsi memberi nama pada satu aturan yang dapat dipanggil berulang kali. Parameter adalah nama lokal yang menerima nilai dari pemanggil. `return` mengirim satu hasil kepada pemanggil.

```
def clipped_mean(values, lower=0, upper=10):
    selected = []
    for value in values:
        if lower <= value <= upper:
            selected.append(value)
    if not selected:
        raise ValueError("tidak ada nilai di dalam rentang")
    return sum(selected) / len(selected)

clipped_mean([-4, 2, 6, 20])
```

Daftar 3.9

4.0

`values`, `lower`, `upper`, dan `selected` bersifat lokal bagi pemanggilan. Nama `selected` tidak tersedia di luar fungsi. Nilai bawaan `lower=0` dan `upper=10` dipakai hanya jika pemanggil tidak menggantinya.

Hindari membuat fungsi matematika bergantung pada variabel global yang dapat berubah tanpa terlihat. Lebih jujur menerima keputusan sebagai parameter dan mengembalikan hasil. Ini membuat masukan, keluaran, dan tes lebih mudah dibaca.

Sebuah fungsi tanpa `return` eksplisit mengembalikan `None`. Mencetak bukan sama dengan mengembalikan: `print` mengirim teks ke layar, sedangkan `return` memberikan nilai yang dapat dipakai perhitungan berikutnya.

3.6 Pengecualian yang informatif

Masukan di luar kontrak harus gagal dengan jelas. Gunakan `TypeError` ketika jenis nilai tidak sesuai dan `ValueError` ketika jenisnya benar tetapi nilainya di luar domain.

```
def reciprocal(value):
    if isinstance(value, bool) or not isinstance(value, (int, float)):
        raise TypeError("value harus berupa angka")
    if value == 0:
        raise ValueError("value tidak boleh nol")
    return 1 / value

(reciprocal(4), reciprocal(-2))
```

Daftar 3.10

(0.25, -0.5)

Tangkap pengecualian hanya jika lapisan tersebut tahu cara menanganinya. Contoh pengurai berikut menambahkan nomor baris lalu meneruskan kegagalan.

```
def parse_integer_lines(text):
    values = []
    for line_number, raw_line in enumerate(text.splitlines(), start=1):
        token = raw_line.strip()
        if not token:
            continue
        try:
            values.append(int(token))
        except ValueError as error:
            raise ValueError(
                f"baris {line_number} bukan bilangan bulat: {token!r}"
            ) from error
    return values

parse_integer_lines("2\n\n-3\n")
```

Daftar 3.11

[2, -3]

Jangan memakai `except Exception: pass`. Bentuk itu menghapus bukti kegagalan dan dapat membiarkan eksperimen menghasilkan keluaran yang tampak lengkap padahal sebagian pemrosesan tidak terjadi.

3.7 Modul dan batas `__main__`

Satu berkas Python merupakan modul. Misalkan `positive_math.py` berisi fungsi berikut.

Daftar 3.12 `positive_math.py`

```
def positive_total(values):
    total = 0
    for value in values:
        if value > 0:
            total += value
    return total

def main():
    print(positive_total([3, -8, 2]))
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

Ketika dijalankan sebagai `python positive_math.py`, Python memberi nama khusus `"__main__"` kepada modul dan memanggil `main`. Ketika tes menjalankan `from positive_math import positive_total`, blok tersebut tidak berjalan. Karena itu, impor memperoleh fungsi tanpa diam-diam mencetak, membaca berkas, atau menimpa keluaran.

Impor modul standar atau pihak ketiga ditulis di bagian atas berkas. Impor lokal menunjuk modul yang menjadi bagian proyek. Nama modul bukan teks untuk dieksekusi; ia adalah batas yang membuat fungsi dapat digunakan dan diuji dari berkas lain.

3.8 Jalur dan tiga format berkas

Path membentuk jalur tanpa merangkai pemisah Windows atau POSIX secara manual.

```
from pathlib import Path

project_root = Path.cwd()
input_path = project_root / "data" / "measurements.csv"
input_path.as_posix().endswith("data/measurements.csv")
```

Daftar 3.13

True

Simpan jalur relatif terhadap akar proyek di konfigurasi dan manifest. Jangan menyimpan nama folder akun pribadi atau menganggap direktori kerja selalu sama.

3.8.1 Teks

Berkas teks cocok untuk baris sederhana. Nyatakan pengodean UTF-8. Ketika byte harus deterministik lintas sistem, tetapkan akhir baris LF.

Daftar 3.14 `write_text_example.py`

```
from pathlib import Path

path = Path("output") / "values.txt"
path.parent.mkdir(parents=True, exist_ok=True)
path.write_text("2\n-1\n5\n", encoding="utf-8", newline="\n")
```

3.8.2 CSV

CSV menyimpan tabel. Gunakan modul `csv` agar koma, tanda kutip, dan akhir baris ditangani konsisten. Buka berkas dengan `newline=""`.

Nilai dari CSV mula-mula berupa string. Konversi `int` atau `float` adalah bagian validasi, bukan sekadar kosmetik.

3.8.3 JSON

JSON menyimpan daftar, pemetaan, string, angka, boolean, dan `null`. JSON tidak menyimpan objek `Path`, set, atau fungsi secara langsung. Ubah data ke bentuk yang didukung dan tetapkan aturan serialisasi.

```
import json

summary = {"count": 3, "sample_ids": ["S01", "S02", "S03"]}
json_text = json.dumps(
```

Daftar 3.15 `read_csv_example.py`

```
import csv
from pathlib import Path

rows = []
with Path("data/measurements.csv").open(
    "r", encoding="utf-8", newline=""
) as stream:
    for row in csv.DictReader(stream):
        rows.append({
            "sample_id": row["sample_id"],
            "value": int(row["value"]),
        })
```

```
summary, ensure_ascii=False, indent=2, sort_keys=True
) + "\n"
json_text
```

Daftar 3.16

```
'{\n  "count": 3,\n  "sample_ids": [\n    "S01",\n    "S02",\n    "S03"\n  ]\n}'
```

CSV baik untuk tabel datar; JSON baik untuk struktur bertingkat; teks biasa baik untuk format yang sangat sederhana dan dinyatakan. Ekstensi berkas tidak membuktikan isinya sah. Selalu uraikan dan validasi struktur yang dibutuhkan.

3.9 Tes merah-hijau dengan unittest

Tes merah-hijau berarti melihat tes gagal karena perilaku yang belum benar, memperbaiki kode, lalu melihat tes yang sama lulus. Jangan menulis kode dan tes yang sudah hijau lalu mengklaim bahwa tahap merah pernah diperiksa.

Buat dua berkas di direktori latihan kosong. Versi pertama modul sengaja salah:

Tes menyatakan perilaku yang dimaksud:

Jalankan:

Daftar 3.17 `positive_math.py`

```
def positive_total(values):  
    return sum(values) # salah: nilai negatif ikut dijumlahkan
```

Daftar 3.18 `test_positive_math.py`

```
import unittest  
  
from positive_math import positive_total  
  
class PositiveTotalTests(unittest.TestCase):  
    def test_only_positive_values_are_added(self):  
        self.assertEqual(positive_total([3, -8, 2, 0]), 5)  
  
if __name__ == "__main__":  
    unittest.main()
```

```
python -m unittest -v test_positive_math.py
```

Tes harus **merah**: fungsi salah menghasilkan -3, bukan 5. Perbaiki modul dengan loop dan `if` pada bagian sebelumnya, lalu jalankan perintah yang sama. Tes harus **hijau**. Tambahkan kasus tanpa nilai positif dan kasus masukan yang ditolak. Rekam perintah serta versi Python; warna antarmuka bukan bukti, tetapi status proses dan daftar tes memberi hasil yang dapat diperiksa.

Tes yang hijau memberi bukti kerja pada kasus yang dijalankan. Ia tidak membuktikan teorema tentang semua program atau semua masukan. Kontrak fungsi, pilihan kasus batas, dan penalaran matematis tetap diperlukan.

3.10 Artefak pendamping deterministik

Kode pendamping asli berlisensi MIT berada di `source/code/primer02_control_files.py`. Dari akar proyek, jalankan:

```
python source/code/primer02_control_files.py --output-dir
output/primer02-demo
```

Perintah tersebut membuat `values.txt`, `measurements.csv`, `summary.json`, dan `manifest.json`. Semua keluaran memakai UTF-8, LF, urutan medan tetap, dan serialisasi JSON kanonis. Manifest mencatat jalur relatif, ukuran byte, dan SHA-256 dari tiga artefak data. Ia tidak memakai jaringan, waktu saat ini, folder pengguna, atau bilangan acak.

Jalankan tes pendamping dengan:

```
python tests/test_primer02.py -v
```

Tes tersebut memeriksa kontrak, jenis kegagalan, impor tanpa efek samping, hasil empat berkas, ikatan hash, dan dua bundel yang identik secara byte.

3.11 Latihan

3.11.1 Latihan 1 - koleksi dan irisan

Dalam modul peserta `latihan_p02.py`, tuliskan `collection_report(values)` untuk daftar bilangan bulat yang tidak kosong. Kembalikan dictionary dengan unsur pertama, unsur terakhir, irisan tanpa kedua ujung, pasangan dua unsur pertama sebagai tuple, nilai unik yang terurut, dan dictionary `positions` yang memetakan indeks berbentuk string ke nilai. Tolak daftar kosong dengan `ValueError`.

Petunjuk

Gunakan indeks 0 dan -1, irisan `1:-1`, `tuple(values[:2])`, `sorted(set(values))`, serta `enumerate` untuk membentuk `positions`.

Cek yang dapat dijalankan

Dari akar proyek, jalankan:

```
python tests/test_primer02.py --learner-module latihan_p02.py --exercise
01
```

Cek memuat `latihan_p02.py` yang Anda tulis, bukan modul jawaban proyek. Ia memeriksa semua medan, urutan hasil, dan penolakan daftar kosong.

💡 Jawaban dan solusi

```
def collection_report(values):
    if not values:
        raise ValueError("values tidak boleh kosong")
    return {
        "first": values[0],
        "last": values[-1],
        "middle": values[1:-1],
        "first_pair": tuple(values[:2]),
        "unique_sorted": sorted(set(values)),
        "positions": {
            str(index): value for index, value in enumerate(values)
        },
    }

collection_report([4, 1, 4, 2])
```

Daftar 3.19

```
{'first': 4,
 'last': 2,
 'middle': [1, 4],
 'first_pair': (4, 1),
 'unique_sorted': [1, 2, 4],
 'positions': {'0': 4, '1': 1, '2': 4, '3': 2}}
```

Set dipakai hanya untuk keunikan; `sorted` mengembalikan urutan deterministik. Validasi tipe per unsur dapat ditambahkan sebelum laporan dibentuk.

3.11.2 Latihan 2 - aliran kontrol sebelum comprehension

Tambahkan ke `latihan_p02.py` fungsi `even_squares_explicit(values)` yang, untuk `[5, 2, -4, 3, 0]`, membangun daftar kuadrat bilangan genap dengan `for`, `if`, dan `append`. Tulis `even_squares_comprehension(values)` yang setara. Tambahkan `countdown(start)` dengan `while` yang menghasilkan `[start, ..., 1, 0]` dan menolak `start < 0`.

Petunjuk

Suatu bilangan genap memenuhi `value % 2 == 0`. Pada loop `while`, kurangi variabel kendali tepat satu kali setiap putaran.

Cek yang dapat dijalankan

```
python tests/test_primer02.py --learner-module latihan_p02.py --exercise 02
```

Cek mengimpor ketiga fungsi dari modul peserta, membandingkan bentuk eksplisit dengan comprehension, dan memeriksa hitung mundur berhenti di nol serta menolak awal negatif.

Jawaban dan solusi

```
def even_squares_explicit(values):
    result = []
    for value in values:
        if value % 2 == 0:
            result.append(value * value)
    return result

def even_squares_comprehension(values):
    return [value * value for value in values if value % 2 == 0]

def countdown(start):
    if start < 0:
        raise ValueError("start harus taknegatif")
    result = []
    while start > 0:
        result.append(start)
        start -= 1
    result.append(0)
    return result

values = [5, 2, -4, 3, 0]
explicit = even_squares_explicit(values)
compact = even_squares_comprehension(values)
(explicit, compact, countdown(3))
```

Daftar 3.20

([4, 16, 0], [4, 16, 0], [3, 2, 1, 0])

Kedua daftar kuadrat harus sama dengan [4, 16, 0]. Bentuk eksplisit menunjukkan urutan operasi yang kemudian diringkas comprehension.

3.11.3 Latihan 3 - fungsi, ruang lingkup, dan galat

Tambahkan `safe_mean(values)` ke `latihan_p02.py`. Fungsi mengembalikan rata-rata sebagai `float`, memakai variabel `total` lokal, menolak koleksi kosong dengan `ValueError`, dan menolak unsur bukan angka dengan `TypeError`. Tunjukkan bahwa `total` tidak dapat diakses setelah fungsi selesai.

Petunjuk

Bekukan iterable menjadi tuple, periksa kekosongan, lalu jumlahkan melalui loop. `bool` sebaiknya ditolak walaupun secara teknis merupakan subkelas `int`.

Cek yang dapat dijalankan

```
python tests/test_primer02.py --learner-module latihan_p02.py --exercise 03
```

Cek memanggil `safe_mean` dari modul peserta dan mengharuskan rata-rata tepat untuk contoh serta jenis pengecualian yang berbeda bagi koleksi kosong dan unsur teks.

💡 Jawaban dan solusi

```
def safe_mean(values):
    frozen = tuple(values)
    if not frozen:
        raise ValueError("values tidak boleh kosong")
    total = 0.0
    for value in frozen:
        if isinstance(value, bool) or not isinstance(value, (int,
float)):
            raise TypeError("setiap unsur harus berupa angka")
        total += value
    return total / len(frozen)

safe_mean([2, 4, 9])
```

Daftar 3.21

5.0

Hasilnya 5.0. Nama `total` hanya hidup di dalam fungsi; mencoba membaca `total` dari ruang global menghasilkan `NameError`. Pengecualian itu bukti ruang lingkup, bukan galat yang perlu disembunyikan.

3.11.4 Latihan 4 - modul peserta dan tes merah-hijau

Buat modul Anda sendiri bernama `positive_math.py` dan berkas tes `test_positive_math.py`. Mulailah dengan implementasi salah yang memakai `sum(values)`, jalankan tes sehingga gagal untuk `[3, -8, 2, 0]`, lalu perbaiki fungsi memakai loop dan `if`. Tambahkan tes kasus `[-3, 0]`.

i Petunjuk

Tes pertama harus mengharapkan 5; tes kedua harus mengharapkan 0. Simpan pemanggilan demo di dalam batas `if __name__ == "__main__"` agar impor oleh tes tidak mencetak apa pun.

! Cek yang dapat dijalankan

Di direktori yang berisi kedua berkas peserta, jalankan `python -m unittest -v test_positive_math.py`. Cek acuan proyek juga tersedia:

```
python tests/test_primer02.py --learner-module positive_math.py
--learner-test test_positive_math.py --exercise 04
```

Simpan keluaran tahap merah dan tahap hijau sebagai bukti bahwa tes yang sama benar-benar mendeteksi perbaikan. Cek proyek di atas menjalankan berkas tes yang Anda tulis dua kali dalam direktori terisolasi: implementasi `sum(values)` harus merah dan modul akhir Anda harus hijau. Cek juga mengimpor `positive_math.py` milik Anda dan memeriksa kasus tambahan.

💡 Jawaban dan solusi

Isi akhir `positive_math.py`:

Daftar 3.22 `positive_math.py`

```
def positive_total(values):
    total = 0
    for value in values:
        if value > 0:
            total += value
    return total

def main():
    print(positive_total([3, -8, 2, 0]))
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

Isi akhir `test_positive_math.py`:

Daftar 3.23 test_positive_math.py

```
import unittest

from positive_math import positive_total

class PositiveTotalTests(unittest.TestCase):
    def test_mixed_values(self):
        self.assertEqual(positive_total([3, -8, 2, 0]), 5)

    def test_no_positive_values(self):
        self.assertEqual(positive_total([-3, 0]), 0)

if __name__ == "__main__":
    unittest.main()
```

Versi salah menjadi merah karena `sum([3, -8, 2, 0]) == -3`. Versi akhir menjadi hijau karena hanya nilai yang memenuhi `value > 0` dijumlahkan.

3.11.5 Latihan 5 - teks, CSV, JSON, dan manifest

Tambahkan `write_demo_bundle(output_dir)` ke `latihan_p02.py`. Fungsi harus membangun satu direktori keluaran yang berisi: teks satu bilangan per baris; CSV dengan kolom `sample_id,value`; JSON ringkasan yang konsisten dengan kedua berkas data; dan manifest berisi jalur relatif, ukuran byte, serta SHA-256 ketiga artefak. Tambahkan `core_sha256` atas bagian inti manifest. Jalankan dua kali ke direktori berbeda dan buktikan semua berkas bersesuaian identik secara byte.

i Petunjuk

Gunakan `Path`, `encoding="utf-8"`, `newline=""` untuk CSV, `lineterminator="\n"`, serta `json.dumps(..., sort_keys=True, indent=2)`. Jangan masukkan waktu saat ini atau jalur absolut ke keluaran.

! Cek yang dapat dijalankan

```
python tests/test_primer02.py --learner-module latihan_p02.py --exercise 05
```

Cek memanggil `write_demo_bundle` dari modul peserta pada dua direktori sementara. Ia mem-parse bilangan teks, kolom dan baris CSV, serta JSON ringkasan; memeriksa konsistensi nilai dan ID sampel; memastikan jalur manifest relatif; mengikat ukuran, SHA-256, dan `core_sha256` ke byte aktual; lalu membandingkan setiap berkas dari kedua run. Jadi cek latihan ini menilai keluaran yang Anda buat, bukan sekadar menjalankan tes terhadap modul jawaban proyek.

💡 Jawaban dan solusi

Fungsi `write_demo_bundle` dalam kode pendamping merupakan solusi acuan. Ia menulis ketiga format, membaca kembali teks dan CSV, membuat ringkasan JSON, lalu mengikat setiap artefak ke manifest. Jalankan:

```
python source/code/primer02_control_files.py --output-dir
output/primer02-demo
python tests/test_primer02.py -v
```

Keempat berkas harus ada. `manifest.json` harus memakai schema `o002.p02.bundle-manifest.v1`; tiga rekaman artefaknya harus cocok dengan byte aktual. Kesamaan hash membuktikan kesamaan byte pada run yang diperiksa, bukan kebenaran ilmiah data atau program.

3.12 Ringkasan

- List dan tuple menyimpan urutan; dictionary memetakan kunci; set menyatakan keanggotaan unik tanpa indeks.
- `if`, `for`, dan `while` membuat aliran kendali terlihat. Comprehension dipakai setelah loop eksplisit dipahami.
- Fungsi menerima masukan melalui parameter, memakai keadaan lokal, dan mengirim hasil melalui `return`.
- Pengecualian harus menyatakan pelanggaran kontrak dan tidak boleh ditelan tanpa penanganan.
- Batas `__main__` menjaga agar impor modul tidak menjalankan efek samping.
- `Path`, UTF-8, parser CSV/JSON, dan serialisasi kanonis membuat pekerjaan berkas lebih portabel dan dapat diperiksa.

- Siklus merah-hijau menunjukkan bahwa tes mampu melihat satu cacat konkret; tes tetap bukan bukti matematis universal.

4 Komputasi, Representasi, dan Bukti

4.1 Tujuan belajar

Setelah menyelesaikan unit ini, pembaca dapat:

- membedakan objek matematika, representasi di komputer, dan keluaran program;
- menjelaskan perbedaan aritmetika eksak dan aritmetika titik-mengambang;
- membandingkan bilangan titik-mengambang dengan toleransi yang dinyatakan;
- menyusun rekaman eksperimen yang dapat dijalankan ulang;
- menulis pengujian kecil untuk perilaku program; dan
- membedakan contoh, contoh penyangkal, bukti komputasional terbatas, dan bukti matematis umum.

Prasyarat lokal: operasi bilangan rasional, fungsi, persamaan sederhana, dan penalaran “untuk setiap” versus “ada”. Tidak diperlukan kalkulus atau aljabar linear.

4.2 Tiga lapisan yang tidak boleh dicampur

Pertimbangkan bilangan rasional

$$\frac{1}{10} + \frac{2}{10} = \frac{3}{10}.$$

Ada tiga lapisan berbeda:

1. **Objek matematika.** Bilangan rasional dan operasi penjumlahan ditentukan secara matematis. Kesamaan di atas eksak.
2. **Representasi.** Komputer harus menyimpan objek melalui pola bit atau struktur data tertentu. Tidak semua bilangan rasional mempunyai representasi biner titik-mengambang yang berhingga.
3. **Keluaran program.** Keluaran bergantung pada representasi, algoritme, lingkungan, dan kode yang benar-benar dijalankan.

Kesalahan umum adalah menganggap keluaran sebagai objek itu sendiri. Program yang mencetak 0.30000000000000004 tidak membantah kesamaan rasional tadi. Ia menunjukkan sifat representasi titik-mengambang yang dipakai program.

4.3 Eksak dan titik-mengambang

Python menyediakan bilangan titik-mengambang untuk perhitungan numerik dan `Fraction` untuk bilangan rasional eksak.

```
0.1 + 0.2
```

Daftar 4.1

```
0.30000000000000004
```

```
from fractions import Fraction  
  
Fraction(1, 10) + Fraction(2, 10)
```

Daftar 4.2

```
Fraction(3, 10)
```

Representasi yang berbeda menjawab kebutuhan berbeda. Aritmetika eksak mempertahankan kesamaan aljabar, tetapi ukuran pembilang dan penyebut dapat bertambah. Titik-mengambang cepat dan menjadi dasar banyak pustaka numerik, tetapi membawa galat pembulatan.

Karena itu, perbandingan numerik harus menyatakan toleransi dan skala:

```
from math import isclose  
  
x = 0.1 + 0.2  
isclose(x, 0.3, rel_tol=1e-12, abs_tol=1e-15)
```

Daftar 4.3

```
True
```

Toleransi bukan mantra. Toleransi adalah bagian dari spesifikasi eksperimen. Kita harus dapat menjelaskan mengapa besarnya memadai bagi pertanyaan yang diajukan.

4.4 Kontrak eksperimen yang dapat direproduksi

Eksperimen komputasional minimal memuat tujuh unsur berikut.

Unsur	Pertanyaan yang harus dijawab
Pertanyaan	Klaim atau gejala apa yang sedang diselidiki?
Lingkungan	Versi bahasa dan perangkat lunak apa yang dipakai?
Masukan	Data, parameter, dan satuannya apa?
Metode	Algoritme dan representasi apa yang dipakai?
Pemeriksaan	Tes apa yang dapat mendeteksi kesalahan implementasi?
Keluaran	Data mentah dan ringkasan apa yang disimpan?
Kesimpulan	Apa yang didukung hasil, dan apa yang belum dibuktikan?

Jalankan eksperimen Unit 1 dari akar proyek:

```
python source/code/unit01_experiment.py
```

Berkas JSON yang dihasilkan mencatat versi Python, parameter, hasil eksak, hasil titik-mengambang, dan pemeriksaan dua dugaan. Menjalankan perintah yang sama pada lingkungan yang sama harus menghasilkan byte yang sama.

4.5 Bukti, bukti kerja, dan contoh penyangkal

Eksperimen dapat memeriksa bahwa

$$n^2 + n$$

genap untuk seribu bilangan bulat pertama. Pemeriksaan itu merupakan bukti kerja yang berguna: ia dapat menemukan kekeliruan kode atau contoh penyangkal. Namun, seribu kasus tidak membuktikan klaim untuk semua bilangan bulat.

Bukti umum singkatnya adalah

$$n^2 + n = n(n + 1).$$

Dua bilangan bulat berurutan selalu memuat satu bilangan genap, sehingga hasil kalinya genap. Argumen ini berlaku untuk setiap bilangan bulat, bukan hanya masukan yang sempat dijalankan.

Sebaliknya, satu contoh penyangkal memang cukup untuk meruntuhkan klaim universal. Polinom

$$p(n) = n^2 + n + 41$$

menghasilkan bilangan prima bagi banyak nilai kecil, tetapi $p(40)=41^2$. Komputer dapat menemukan contoh ini; faktorisasi tersebut menjelaskan secara matematis mengapa klaim “prima untuk setiap bilangan bulat taknegatif” salah.

4.6 Pengujian sebagai klaim terbatas

Tes program adalah klaim terbatas dan dapat diperiksa, misalnya:

- penjumlahan rasional eksak menghasilkan $3/10$;
- hasil titik-mengambang dekat dengan 0.3 pada toleransi yang ditetapkan;
- pemeriksa paritas tidak menemukan pelanggaran pada rentang uji;
- eksperimen menampilkan contoh penyangkal $n=40$; dan
- dua penulisan JSON pada lingkungan dan parameter yang sama identik.

Tes meningkatkan kepercayaan terhadap implementasi. Tes tidak dengan sendirinya membuktikan teorema yang sedang diimplementasikan.

4.7 Latihan

4.7.1 Latihan 1 - representasi

Jelaskan dengan kata-kata sendiri mengapa keluaran 0.30000000000000004 tidak membuat persamaan $1/10 + 2/10 = 3/10$ menjadi salah.

Petunjuk

Pisahkan objek matematika dari cara komputer menyimpannya.

Jawaban dan solusi

Kesamaan itu adalah kesamaan eksak dalam bilangan rasional. Bilangan 0.1 dan 0.2 tidak tersimpan secara eksak dalam representasi biner titik-mengambang yang dipakai Python. Galat kecil pada keluaran berasal dari pendekatan representasi, bukan dari perubahan aturan penjumlahan rasional.

4.7.2 Latihan 2 - spesifikasi toleransi

Tuliskan satu situasi yang wajar memakai `abs_tol` dan satu situasi yang lebih wajar memakai `rel_tol`. Nyatakan alasan, bukan hanya angkanya.

Petunjuk

Bandingkan besaran yang dekat nol dengan besaran yang mempunyai skala alami besar.

Jawaban dan solusi

Untuk residual persamaan yang secara teori nol, toleransi absolut sesuai karena galat dibandingkan langsung dengan nol. Untuk membandingkan dua jarak sekitar satu juta meter, toleransi relatif lebih informatif karena galat yang dapat diterima mengikuti skala jarak. Dalam pekerjaan nyata, keduanya sering dipakai bersama dan harus diturunkan dari kebutuhan masalah.

4.7.3 Latihan 3 - batas eksperimen

Ubah batas pemeriksaan paritas dari 1.000 menjadi 100.000. Kesimpulan baru apa yang sah, dan kesimpulan apa yang tetap tidak sah?

Petunjuk

Lebih banyak kasus memperbesar wilayah yang telah diperiksa, bukan mengubahnya menjadi himpunan semua bilangan bulat.

Jawaban dan solusi

Sah: tidak ada contoh penyangkal yang ditemukan dalam rentang 0 sampai 99.999, dengan asumsi pemeriksa diimplementasikan benar. Tidak sah: klaim telah dibuktikan untuk semua bilangan bulat. Bukti umum masih memerlukan argumen seperti faktorisasi $n(n+1)$.

4.7.4 Latihan 4 - contoh penyangkal

Gunakan program untuk mencari nilai bulat taknegatif terkecil yang membuat $n^2 + n + 41$ komposit. Lalu jelaskan hasilnya tanpa bergantung pada program.

Petunjuk

Periksa nilai 40 dan faktorkan hasilnya.

Jawaban dan solusi

Nilai terkecilnya adalah $n=40$. Secara eksak, $40^2 + 40 + 41 = 1681 = 41^2$, sehingga nilainya komposit. Perhitungan eksak ini menjadi penjelasan matematis yang dapat diperiksa terpisah dari program.

4.7.5 Latihan 5 - rekaman eksperimen

Baca `output/unit01-results.json`. Sebutkan dua medan yang diperlukan untuk menjalankan ulang eksperimen dan satu informasi yang masih perlu ditambahkan jika eksperimen memakai pustaka eksternal.

Petunjuk

Perhatikan parameter dan lingkungan.

Jawaban dan solusi

Contoh jawaban: `limit` dan versi Python diperlukan. Jika eksperimen memakai pustaka eksternal, rekaman juga perlu menyimpan nama dan versi pustaka tersebut, serta idealnya berkas lingkungan atau kunci dependensi yang dapat dipasang ulang.

4.8 Ringkasan

- Objek matematika tidak identik dengan representasinya di komputer.
- Aritmetika eksak dan titik-mengambang melayani tujuan berbeda.
- Toleransi merupakan bagian dari spesifikasi, bukan perbaikan setelah hasil terlihat buruk.
- Eksperimen yang dapat direproduksi menyimpan pertanyaan, lingkungan, masukan, metode, pemeriksaan, keluaran, dan batas kesimpulan.
- Banyak contoh dapat mendukung dugaan; satu contoh penyangkal dapat meruntuhkan klaim universal; bukti umum memerlukan argumen yang berlaku bagi seluruh domain.

5 Objek, Nilai, dan Fungsi

5.1 Tujuan belajar

Setelah menyelesaikan unit ini, pembaca dapat:

- memilih representasi Python yang sesuai untuk bilangan bulat, rasional, dan barisan berhingga;
- membedakan kesamaan nilai dari kesamaan identitas objek;
- menulis fungsi dengan prasyarat, hasil, dan perilaku galat yang dinyatakan;
- menghindari perubahan tak disengaja melalui nilai masukan yang dapat diubah;
- merumuskan invarian yang dapat diperiksa oleh tes; dan
- menghasilkan keluaran terstruktur yang tidak bergantung pada urutan kebetulan.

Prasyarat lokal: Unit 1, himpunan dan fungsi pada tingkat A30, serta operasi dasar pada vektor berhingga. Unit ini belum memerlukan NumPy.

5.2 Jenis matematika dan tipe program

Pernyataan matematika menentukan himpunan objek dan operasi yang sah. Program membutuhkan tipe data konkret. Keduanya berkaitan, tetapi tidak identik.

Maksud matematika	Representasi awal	Batas penting
bilangan bulat	<code>int</code>	ukuran praktis dibatasi memori
bilangan rasional	<code>Fraction</code>	pembilang dan penyebut dapat membesar
pendekatan bilangan real	<code>float</code>	pembulatan dan rentang berhingga
vektor berhingga yang tetap	<code>tuple</code>	panjang dan jenis komponen harus diperiksa
koleksi yang sengaja diubah	<code>list</code>	alias dapat melihat perubahan yang sama

```
from fractions import Fraction

values = (7, Fraction(7, 3), 7 / 3)
[(type(value).__name__, value) for value in values]
```

Daftar 5.1

```
[('int', 7), ('Fraction', Fraction(7, 3)), ('float', 2.3333333333333335)]
```

Nama tipe tidak membuktikan bahwa nilai memenuhi maksud matematika. Sebuah **tuple** berpanjang tiga dapat mewakili vektor di ruang tiga dimensi, koordinat warna, atau tiga data yang tidak berhubungan. Kontrak programlah yang memberi makna tambahan itu.

5.3 Nilai, identitas, dan alias

Operator `==` membandingkan nilai menurut aturan tipe. Operator `is` memeriksa apakah dua nama menunjuk objek Python yang sama. Untuk alasan reproduksibilitas, jangan mengganti pertanyaan tentang nilai dengan pertanyaan tentang identitas objek.

```
a = [1, 2, 3]
b = a
c = list(a)

(a == c, a is c, a is b)
```

Daftar 5.2

```
(True, False, True)
```

Karena **b** adalah alias bagi **a**, perubahan melalui salah satu nama tampak melalui nama lainnya. Sebaliknya, **c** mula-mula mempunyai nilai sama, tetapi merupakan daftar lain. Efek semacam ini dapat membuat eksperimen bergantung pada urutan eksekusi sel.

Jika suatu vektor dimaksudkan sebagai nilai yang tidak berubah, **tuple** memberi kontrak yang lebih jujur daripada **list**. Ini bukan larangan memakai daftar; pilih perubahan hanya ketika perubahan tersebut bagian dari metode.

5.4 Fungsi sebagai kontrak

Fungsi komputasional yang baik menyatakan tiga hal:

1. **Prasyarat:** masukan apa yang diterima?
2. **Hasil:** nilai apa yang dikembalikan jika prasyarat dipenuhi?
3. **Kegagalan:** apa yang terjadi jika prasyarat dilanggar?

Contoh berikut menghitung rata-rata rasional secara eksak.

```
from fractions import Fraction

def mean_fraction(values):
    """Kembalikan rata-rata eksak dari barisan rasional yang tidak kosong."""
    values = tuple(Fraction(value) for value in values)
    if not values:
        raise ValueError("values tidak boleh kosong")
    return sum(values, start=Fraction(0)) / len(values)

mean_fraction([Fraction(1, 3), Fraction(1, 2)])
```

Daftar 5.3

Fraction(5, 12)

Konversi ke `tuple` membekukan masukan yang benar-benar dipakai fungsi. Jika pemanggil mengubah daftar asal setelah pemanggilan dimulai, perhitungan lokal tidak ikut berubah.

5.5 Fungsi murni dan perubahan keadaan

Fungsi berikut mengalikan vektor dengan skalar tanpa mengubah argumennya.

```
def scale_vector(scalar, vector):
    return tuple(scalar * component for component in vector)

v = (2, -1, 4)
(scale_vector(3, v), v)
```

Daftar 5.4

$((6, -3, 12), (2, -1, 4))$

Untuk masukan yang sama, fungsi murni menghasilkan keluaran yang sama dan tidak mengubah keadaan di luar fungsi. Sifat ini memudahkan pengujian dan pelacakan asal hasil. Program nyata tetap perlu membaca berkas atau menulis keluaran; tempatkan efek tersebut di batas program, bukan tersebar di setiap fungsi matematika.

5.6 Invarian dan pengujian

Invarian adalah sifat yang harus tetap benar selama operasi tertentu. Untuk perkalian skalar pada vektor, beberapa invarian yang dapat diuji ialah:

- panjang vektor tidak berubah;
- mengalikan dengan 1 mempertahankan nilai;
- mengalikan dengan 0 menghasilkan vektor nol; dan
- `scale_vector(a*b, v)` sama dengan `scale_vector(a, scale_vector(b, v))`.

Tes terhadap banyak contoh dapat mendeteksi pelanggaran implementasi. Bukti aljabar tetap diperlukan untuk menyatakan sifat bagi semua skalar dan semua vektor pada domain yang dimaksud.

5.7 Keluaran kanonis

Dua eksperimen dapat menghasilkan informasi sama tetapi byte berbeda karena urutan medan, spasi, atau akhir baris. Bila keluaran dipakai sebagai bukti kerja, tentukan bentuk kanonis. Skrip Unit 2:

```
python source/code/unit02_objects.py --output output/unit02-results.json
```

menulis JSON UTF-8 dengan nama medan terurut, inden tetap, dan akhir baris LF. Hash berkas lalu mengikat satu urutan byte yang dapat dibandingkan.

5.8 Latihan

5.8.1 Latihan 1 - memilih representasi

Pilih representasi awal untuk setiap objek berikut dan jelaskan batasnya: $2/7$, pendekatan numerik bagi $\sqrt{2}$, dan vektor tetap $(1, 0, -1)$.

Petunjuk

Bedakan nilai eksak dari pendekatan dan nilai tetap dari koleksi yang diubah.

Jawaban dan solusi

Gunakan `Fraction(2, 7)` bagi bilangan rasional eksak, `float` bagi pendekatan numerik terhadap akar dua dengan toleransi yang dinyatakan, dan `tuple` bagi vektor tetap. `float` tidak menyimpan akar dua secara eksak, sedangkan `tuple` tidak dengan sendirinya menjamin bahwa semua komponennya numerik.

5.8.2 Latihan 2 - nilai dan identitas

Buat dua daftar berbeda dengan nilai `[1, 2]`. Tunjukkan satu perbandingan yang benar dan satu perbandingan yang salah. Jelaskan hasilnya.

Petunjuk

Gunakan `==` dan `is`.

Jawaban dan solusi

Jika `a=[1,2]` dan `b=[1,2]`, maka `a == b` benar karena nilainya sama, sedangkan `a is b` salah karena keduanya objek berbeda. Pertanyaan matematika tentang kesamaan daftar unsur biasanya memakai `==`.

5.8.3 Latihan 3 - kontrak fungsi

Tuliskan prasyarat, hasil, dan kegagalan untuk fungsi yang mengembalikan sisa terkecil taknegatif dari pembagian a dengan modulus m .

i Petunjuk

Modulus harus positif.

💡 Jawaban dan solusi

Prasyarat: a dan m bilangan bulat serta $m > 0$. Hasil: bilangan bulat r dengan $0 \leq r < m$ dan $a-r$ habis dibagi m . Jika $m \leq 0$, fungsi harus menolak masukan, misalnya dengan `ValueError`.

5.8.4 Latihan 4 - alias yang berbahaya

Jelaskan mengapa fungsi yang menambahkan elemen ke daftar masukan dapat membuat hasil eksperimen bergantung pada urutan sel notebook. Berikan satu perbaikan.

i Petunjuk

Nama lain mungkin menunjuk daftar yang sama.

💡 Jawaban dan solusi

Pemanggilan pertama mengubah daftar yang kemudian dipakai pemanggilan kedua. Menjalankan sel dalam urutan lain memberi keadaan awal berbeda. Perbaikannya dapat berupa mengembalikan daftar baru, memakai `tuple` bagi nilai tetap, atau membuat perubahan eksplisit serta mengatur ulang keadaan sebelum eksperimen.

5.8.5 Latihan 5 - tes dan bukti

Apakah menguji empat invarian `scale_vector` untuk sejuta vektor membuktikan bahwa implementasinya benar bagi semua masukan? Berikan jawaban yang tepat.

i Petunjuk

Pisahkan domain yang diuji dari domain universal.

💡 Jawaban dan solusi

Tidak. Tes tersebut memberi bukti kerja kuat pada sampel dan dapat menemukan contoh penyangkal, tetapi tidak mencakup semua masukan. Kebenaran umum fungsi diperoleh

dengan memeriksa definisi implementasi terhadap hukum distributif dan asosiatif pada domain skalar serta komponen yang dinyatakan.

5.9 Ringkasan

- Tipe program mewakili objek matematika, tetapi tidak menentukan seluruh maknanya.
- `==` membandingkan nilai; `is` membandingkan identitas objek.
- Fungsi harus menyatakan prasyarat, hasil, dan kegagalan.
- Fungsi murni dan nilai tak berubah memudahkan reproduksi serta pengujian.
- Invarian menghubungkan kontrak matematika dengan tes program.
- Keluaran kanonis membuat hash artefak bermakna.

6 Array, Bentuk, dan Vektorisasi

6.1 Tujuan belajar

Setelah menyelesaikan unit ini, pembaca dapat:

- membuat array NumPy dengan bentuk dan `dtype` yang dinyatakan;
- membaca hubungan antara indeks, sumbu, dan bentuk array;
- mengganti perulangan komponen yang sederhana dengan operasi vektorisasi;
- memakai broadcasting hanya setelah memeriksa kecocokan bentuk;
- membedakan *view* yang berbagi memori dari salinan yang terpisah;
- merumuskan invarian bentuk, nilai, dan keadaan masukan; dan
- membatasi kesimpulan eksperimen agar tidak disalahartikan sebagai bukti umum.

Prasyarat lokal: Unit 1-2, fungsi linear sederhana, daftar dan tuple, serta koordinat pada bidang. Unit ini tidak mengandaikan kalkulus atau aljabar linear formal.

6.2 Array sebagai data beserta kontraknya

Array bukan sekadar kumpulan angka. Kontrak minimalnya memuat:

1. **bentuk** (`shape`), yaitu banyaknya posisi pada setiap sumbu;
2. **jumlah dimensi** (`ndim`);
3. **jenis representasi** (`dtype`); dan
4. **makna setiap sumbu**, yang harus kita dokumentasikan sendiri.

```
import numpy as np

temperatur = np.array(
    [[28.0, 29.5, 30.0], [27.5, 29.0, 31.0]],
    dtype=np.float64,
)
(temperatur.shape, temperatur.ndim, temperatur.dtype)
```

Daftar 6.1

```
((2, 3), 2, dtype('float64'))
```

Bentuk (2, 3) hanya mengatakan bahwa ada dua baris dan tiga kolom. Bentuk itu belum mengatakan apakah baris menyatakan kota, hari, atau pengulangan percobaan. Makna tersebut merupakan bagian dari spesifikasi data.

Array satu dimensi dengan bentuk (3,) berbeda dari tabel satu baris berbentuk (1, 3) dan tabel satu kolom berbentuk (3, 1). Ketiganya memuat tiga angka, tetapi operasi terhadap sumbunya dapat berbeda.

6.3 dtype adalah representasi, bukan himpunan matematika

NumPy menyimpan elemen array biasa dalam satu `dtype`. Pilihan eksplisit membuat kontrak lebih mudah diperiksa:

```
bilangan_bulat = np.array([1, 2, 3], dtype=np.int64)
pendekatan = np.array([0.1, 0.2, 0.3], dtype=np.float64)
(bilangan_bulat.dtype, pendekatan.dtype, pendekatan.sum())
```

Daftar 6.2

```
(dtype('int64'), dtype('float64'), np.float64(0.6000000000000001))
```

`int64` menyimpan bilangan bulat dalam rentang berhingga. Operasi yang keluar dari rentang itu dapat meluap; ia tidak sama dengan himpunan seluruh bilangan bulat matematika. `float64` menyimpan pendekatan biner dengan ketelitian berhingga; ia tidak sama dengan bilangan real.

NumPy dapat memakai `dtype=object` untuk objek Python seperti bilangan bulat tak terbatas praktis atau `Fraction`, tetapi banyak keuntungan komputasi array numerik hilang. Jika pertanyaan menuntut aritmetika eksak, pilih alat eksak secara sadar (misalnya `Fraction`, `SymPy`, atau `Sage`), alih-alih menyebut hasil `float64` eksak.

6.4 Vektorisasi dan referensi skalar

Misalkan setiap data x_i diubah oleh fungsi

$$y_i = 3x_i - 2.$$

Perulangan skalar yang jelas dapat menjadi referensi:

```
x = np.array([1, 2, 3, 4], dtype=np.int64)
y_referensi = np.array([3 * int(nilai) - 2 for nilai in x])
y_referensi
```

Daftar 6.3

```
array([ 1,  4,  7, 10])
```

NumPy memungkinkan aturan yang sama ditulis untuk seluruh array:

```
y = 3 * x - 2
y
```

Daftar 6.4

```
array([ 1,  4,  7, 10])
```

Vektorisasi menyatakan operasi pada array tanpa perulangan Python yang kita tulis sendiri. Ia sering lebih ringkas dan dapat lebih cepat, tetapi keringkasan bukan bukti kebenaran. Bandingkan hasilnya dengan referensi sederhana dan periksa invarian yang berasal dari rumus:

- `y.shape == x.shape`;
- `x` tidak berubah; dan
- $\sum_i y_i = 3 \sum_i x_i - 2n$, dengan n banyaknya elemen.

Referensi skalar juga harus diperiksa. Dua implementasi yang menyalin kesalahan konseptual sama dapat sepakat dan tetap salah.

6.5 Broadcasting dengan pemeriksaan bentuk

Broadcasting memperluas operasi antararray tanpa membuat salinan ubin secara manual. Misalkan setiap kolom tabel menerima offset berbeda:

```
tabel = np.array([[10, 20, 30], [40, 50, 60]], dtype=np.int64)
offset = np.array([1, -2, 3], dtype=np.int64)
tabel + offset
```

Daftar 6.5

```
array([[11, 18, 33],
       [41, 48, 63]])
```

Bentuk (2, 3) dan (3,) cocok karena sumbu terakhir sama-sama berukuran tiga. Offset diterapkan pada setiap baris. Jangan mengandalkan keberhasilan operasi sebagai satu-satunya pemeriksaan: bentuk yang tidak dimaksudkan kadang tetap dapat di-*broadcast* dan menghasilkan jawaban yang tampak masuk akal.

Fungsi `add_column_offsets` dalam kode Unit 3 menyatakan kontrak lebih ketat: tabel harus dua dimensi, offset harus satu dimensi, dan jumlah offset harus sama dengan jumlah kolom.

6.6 Salinan dan *view*

Irisan array NumPy biasanya merupakan *view*: objek baru yang melihat memori array asal. Mengubah *view* dapat mengubah data asal.

```
asal = np.arange(6)
jendela = asal[1:4]
jendela[0] = -10
asal
```

Daftar 6.6

```
array([ 0, -10,  2,  3,  4,  5])
```

Gunakan `.copy()` bila eksperimen membutuhkan nilai terpisah:

```
asal = np.arange(6)
terpisah = asal[1:4].copy()
terpisah[0] = -10
asal
```

Daftar 6.7

```
array([0, 1, 2, 3, 4, 5])
```

Pilihan ini harus disengaja. *View* berguna untuk menghindari salinan besar; salinan berguna untuk mencegah perubahan tak terduga. `np.shares_memory(a, b)` dapat membantu memeriksa hubungan itu pada tes.

6.7 Eksperimen Unit 3

Jalankan dari akar proyek:

```
python source/code/unit03_arrays.py --output output/unit03-results.json
```

Skrip mencatat versi NumPy, bentuk, `dtype`, hasil vektorisasi, hasil broadcasting, jejak *view* dan salinan, batas titik-mengambang, serta batas epistemik kesimpulan. JSON ditulis sebagai UTF-8 dengan kunci terurut, inden tetap, dan akhir baris LF. Pada lingkungan yang sama, pemanggilan yang sama menghasilkan byte yang sama.

6.8 Komputasi dan bukti

Untuk array `[1, 2, 3, 4]`, program memeriksa bahwa operasi vektorisasi sama dengan referensi skalar. Hasil itu mendukung klaim terbatas tentang kasus dan implementasi yang dijalankan. Ia tidak membuktikan kesamaan bagi setiap array, setiap `dtype`, atau setiap ukuran.

Untuk aritmetika tanpa luapan, argumen umum berjalan per komponen. Pada indeks i , kedua implementasi menghitung ekspresi yang sama, yaitu $3x_i - 2$. Karena indeks i dipilih sembarang, kesamaan berlaku pada setiap komponen; bentuk juga tetap karena tepat satu keluaran dibuat untuk setiap masukan.

Argumen itu harus dibatasi oleh domain representasi. Pada `int64`, luapan dapat mengubah hubungan dengan bilangan bulat matematika. Pada `float64`, pembulatan dapat membuat dua susunan operasi yang ekuivalen secara aljabar menghasilkan bit berbeda. Tes membantu menemukan gejala tersebut; spesifikasi dan argumen menentukan apa yang sebenarnya diklaim.

6.9 Latihan

6.9.1 Latihan 1 - bentuk dan makna sumbu

Sebuah array menyimpan pengukuran tiga sensor pada empat waktu dan mempunyai bentuk (4, 3). Jelaskan arti kedua sumbu. Apa bentuk yang dihasilkan oleh `data[2, :]` dan `data[:, 1]`?

Petunjuk

Indeks tunggal menghapus sumbu yang dipilih, sedangkan `:` mempertahankan semua posisi pada sumbu itu.

Jawaban dan solusi

Sumbu 0 menyatakan empat waktu dan sumbu 1 menyatakan tiga sensor. `data[2, :]` memilih seluruh sensor pada waktu ketiga sehingga berbentuk (3,). `data[:, 1]` memilih sensor kedua pada seluruh waktu sehingga berbentuk (4,). Makna ini berasal dari kontrak data, bukan dari bentuk saja.

6.9.2 Latihan 2 - vektorisasi dan invarian

Vektorisasikan perulangan yang menghitung $z_i = 5x_i + 1$. Sebutkan dua invarian yang dapat diuji dan turunkan invarian jumlahnya.

Petunjuk

Terapkan operasi langsung pada `x`, lalu jumlahkan persamaan komponen.

Jawaban dan solusi

Tuliskan `z = 5*x + 1`. Dua invarian yang berguna ialah `z.shape == x.shape` dan `x` tidak berubah. Jika ada n elemen, maka

$$\sum_i z_i = \sum_i (5x_i + 1) = 5 \sum_i x_i + n.$$

Tes dapat membandingkan hasil vektorisasi dengan referensi skalar serta memeriksa ketiga sifat tersebut pada kasus yang dipilih.

6.9.3 Latihan 3 - kontrak broadcasting

Tentukan apakah pasangan bentuk berikut cocok untuk penambahan yang dimaksudkan sebagai “satu offset per kolom”: (5, 3) dengan (3,), (5, 3) dengan (5,), dan (5, 3) dengan (1, 3). Jelaskan setiap jawaban.

Petunjuk

Bandingkan dimensi dari kanan. Ukuran harus sama atau salah satunya satu.

Jawaban dan solusi

(5, 3) dengan (3,) cocok dan menerapkan tiga offset pada setiap baris. (5, 3) dengan (5,) tidak cocok karena ukuran terakhir 3 dan 5 berbeda. (5, 3) dengan (1, 3) cocok; sumbu pertama berukuran satu diperluas menjadi lima. Meskipun dua pasangan cocok menurut NumPy, kontrak fungsi tetap perlu menyatakan bahwa data kedua benar-benar berarti offset kolom.

6.9.4 Latihan 4 - melacak *view*

Untuk kode berikut, tentukan nilai akhir *a*, *b*, dan *c* tanpa menjalankannya.

```
a = np.array([0, 1, 2, 3])
b = a[1:3]
c = a[1:3].copy()
b[0] = 8
c[1] = 9
```

Petunjuk

b berbagi memori dengan *a*, sedangkan *c* tidak.

Jawaban dan solusi

Nilai akhirnya adalah *a* == [0, 8, 2, 3], *b* == [8, 2], dan *c* == [1, 9]. Perubahan melalui *b* mengenai elemen yang sama pada *a*. Salinan *c* dibuat sebelum perubahan dan perubahan pada *c* tidak kembali ke *a*.

6.9.5 Latihan 5 - eksperimen bukan bukti umum

Anda membangkitkan satu juta array bilangan kecil dan mendapati bahwa $3x-2$ selalu sama dengan referensi perulangan. Nyatakan kesimpulan terkuat yang sah, lalu berikan kerangka bukti umum dan satu batas representasinya.

Petunjuk

Pisahkan cakupan sampel, argumen per komponen, dan perilaku `dtype` berhingga.

Jawaban dan solusi

Kesimpulan empiris terkuat ialah tidak ditemukan perbedaan pada satu juta kasus dan lingkungan yang diuji. Untuk bukti umum pada domain aritmetika yang ditetapkan, pilih indeks sembarang i : kedua cara menghitung $3x_i - 2$, sehingga semua komponennya sama dan bentuknya tetap. Batasnya, `int64` dapat meluap dan `float64` membulatkan; karena itu hubungan dengan aritmetika bilangan bulat atau real harus menyebut rentang dan model representasi.

6.10 Ringkasan

- `shape`, `ndim`, `dtype`, dan makna sumbu bersama-sama membentuk kontrak array.
- Vektorisasi mengungkap operasi per komponen, tetapi tetap perlu referensi dan invarian.
- Broadcasting aman secara pedagogis bila bentuk dan maksud sumbunya diperiksa.
- Irisan biasanya merupakan *view*; `.copy()` memutus hubungan memori.
- Keluaran kanonis merekam eksperimen yang dapat dibandingkan byte demi byte.
- Pemeriksaan banyak kasus adalah bukti kerja, bukan pengganti argumen umum; batas `dtype` harus menjadi bagian dari klaim.

7 Visualisasi dengan Integritas

7.1 Tujuan belajar

Setelah menyelesaikan unit ini, pembaca dapat:

- memulai visualisasi dari pertanyaan yang dapat dinyatakan dengan jelas;
- membangun grafik secara bertahap melalui `fig` dan `ax`, kemudian `plot`, `scatter`, serta `errorbar`;
- memetakan variabel ke posisi, tanda, garis, dan interval tanpa menyembunyikan satuan;
- menjelaskan bagaimana pemotongan sumbu dan rasio aspek mengubah kesan visual;
- menyajikan ketidakpastian tanpa memberinya makna statistik yang tidak ditetapkan;
- menyediakan data, gambar, dan deskripsi tekstual yang dapat diperiksa ulang; dan
- membedakan pola pada gambar dari bukti matematis umum.

Prasyarat lokal: Unit 1, koordinat Kartesius, persamaan garis lurus, rasio, dan interval pada tingkat A30. Unit ini tidak memerlukan kalkulus.

7.2 Gambar dimulai dari pertanyaan

Grafik bukan hiasan yang ditambahkan setelah perhitungan selesai. Grafik adalah argumen terstruktur tentang data. Sebelum memilih warna atau jenis garis, nyatakan pertanyaannya.

Dalam unit ini pertanyaannya adalah:

Apakah tujuh pengukuran jarak konsisten dengan model $d = 2t + 1$, jika setiap pengukuran mempunyai ketidakpastian $\pm 0,4$ meter?

Data yang digunakan sengaja kecil agar setiap tanda pada grafik dapat dicocokkan dengan tabel.

Waktu t (s)	Jarak terukur d (m)	Ketidakpastian (m)	Nilai model (m)
0	1,0	0,4	1,0
1	3,1	0,4	3,0
2	4,8	0,4	5,0
3	7,2	0,4	7,0

Waktu t (s)	Jarak terukur d (m)	Ketidakpastian (m)	Nilai model (m)
4	9,0	0,4	9,0
5	10,9	0,4	11,0
6	13,1	0,4	13,0

Karena pertanyaannya membandingkan pengukuran dengan model, grafik harus menampilkan keduanya. Menampilkan hanya garis model akan menghapus bukti pengukuran; menampilkan hanya titik akan menyembunyikan objek pembanding.

7.3 Pengodean: apa berarti apa?

Setiap unsur visual harus mempunyai arti yang dapat dituliskan tanpa melihat gambar.

Unsur visual	Makna
posisi mendatar	waktu t , dalam detik
posisi tegak	jarak d , dalam meter
titik lingkaran	nilai pengukuran
batang vertikal	interval pengukuran $d \pm 0,4$ meter
garis putus-putus	nilai model $d = 2t + 1$

Warna membantu membedakan unsur, tetapi tidak boleh menjadi satu-satunya pengodean. Titik berbentuk lingkaran dan garis putus-putus tetap dapat dibedakan pada cetakan abu-abu atau oleh pembaca yang tidak membedakan warna tertentu.

Pilih pengodean yang sesuai dengan pertanyaan. Posisi pada sumbu berskala berguna untuk membandingkan besaran. Diagram lingkaran tidak sesuai di sini: jarak pada waktu berbeda bukan bagian-bagian dari satu keseluruhan.

7.4 Lab bertahap: dari fig dan ax ke grafik lengkap

Matplotlib memisahkan **gambar keseluruhan** (**fig**) dari **daerah koordinat** (**ax**). **fig** mengatur kanvas yang akan disimpan. **ax** menerima data, skala, label, dan legenda. Pemisahan ini membuat setiap keputusan terlihat dan dapat diuji. Lab berikut dimulai dari kanvas kosong, lalu menambahkan tepat satu lapisan pada setiap tahap.

Daftar 7.1

```
from matplotlib import pyplot as plt
from source.code.unit04_visualization import ALT_TEXT, OBSERVATIONS

fig_u04_lab, ax_u04_lab = plt.subplots(figsize=(8.0, 4.8))
```

7.4.1 Tahap 1 - buat fig dan ax

Nama variabel dapat berbeda, tetapi pola `fig, ax = plt.subplots(...)` layak dipertahankan: pembaca kode langsung tahu objek mana yang disimpan dan objek mana yang menerima pengodean data. Ukuran gambar ditetapkan agar tidak bergantung pada ukuran jendela interaktif.

Siapkan empat barisan data dari tabel. Daftar dibuat dalam urutan pengamatan yang sudah divalidasi.

Daftar 7.2

```
waktu_u04 = [baris.time_s for baris in OBSERVATIONS]
jarak_u04 = [baris.measured_distance_m for baris in OBSERVATIONS]
galat_u04 = [baris.uncertainty_m for baris in OBSERVATIONS]
model_u04 = [baris.model_distance_m for baris in OBSERVATIONS]
```

7.4.2 Tahap 2 - plot untuk model

`ax.plot` cocok untuk nilai model yangurut menurut waktu. Garis putus-putus menjadi pengodean kedua selain warna.

Koma pada `garis_model_u04, = ...` membongkar daftar satu garis yang dikembalikan `plot`. Pada tahap ini belum ada titik pengukuran dan belum ada batang ketidakpastian.

7.4.3 Tahap 3 - scatter untuk pengukuran

`ax.scatter` menambahkan tujuh pengukuran sebagai tanda terpisah. Bentuk lingkaran membuat seri tetap dikenali tanpa warna.

Daftar 7.3

```
garis_model_u04, = ax_u04_lab.plot(
    waktu_u04,
    model_u04,
    color="#D55E00",
    linestyle="--",
    linewidth=2,
    label="Model  $d = 2t + 1$ ",
)
assert garis_model_u04.get_linestyle() == "--"
```

Daftar 7.4

```
titik_u04 = ax_u04_lab.scatter(
    waktu_u04,
    jarak_u04,
    marker="o",
    s=36,
    color="#0072B2",
    label="Pengukuran",
    zorder=3,
)
assert len(titik_u04.get_offsets()) == 7
```

7.4.4 Tahap 4 - errorbar untuk ketidakpastian

Batang ketidakpastian merupakan lapisan tersendiri. `fmt="none"` mencegah `errorbar` menggambar titik kedua di atas hasil `scatter`.

Nilai `yerr` menyatakan simpangan vertikal dari setiap titik. Arti statistiknya tidak berasal dari nama metode; arti tetap harus dinyatakan oleh prosedur pengukuran.

7.4.5 Tahap 5 - label bersatuan dan legenda

Data kini terlihat, tetapi grafik belum dapat ditafsirkan tanpa satuan, rentang, dan kunci pengodean.

Daftar 7.5

```
batang_u04 = ax_u04_lab.errorbar(  
    waktu_u04,  
    jarak_u04,  
    yerr=galat_u04,  
    fmt="none",  
    capsize=4,  
    ecolor="#4D4D4D",  
    label="Ketidakpastian ( $\pm 0,4$  m)",  
    zorder=2,  
)  
assert batang_u04.has_yerr
```

7.4.6 Tahap 6 - deskripsi aksesibel dan penyimpanan deterministik

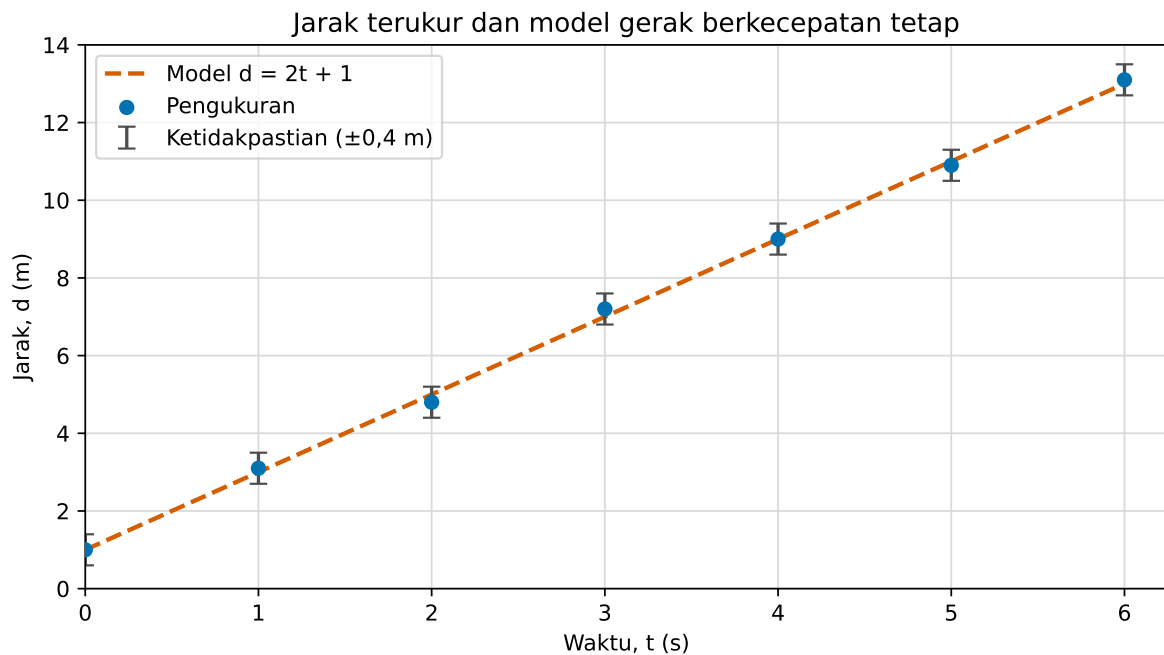
Gambar statis memerlukan deskripsi yang menyatakan pertanyaan, sumbu, tanda, pola, dan batas kesimpulan. Quarto memasang deskripsi berikut dengan gambar lab. Deskripsi yang sama tersedia dalam konstanta `ALT_TEXT` dan ditulis ke artefak teks oleh skrip Unit 4.

```
fig_u04_lab
```

Daftar 7.6

```
ax_u04_lab.set_title("Jarak terukur dan model gerak berkecepatan tetap")
ax_u04_lab.set_xlabel("Waktu, t (s)")
ax_u04_lab.set_ylabel("Jarak, d (m)")
ax_u04_lab.set_xlim(0, 6.25)
ax_u04_lab.set_ylim(0, 14)
ax_u04_lab.set_xticks(waktu_u04)
ax_u04_lab.set_yticks(range(0, 15, 2))
ax_u04_lab.grid(True, color="#D9D9D9", linewidth=0.8)
ax_u04_lab.set_axisbelow(True)
legenda_u04 = ax_u04_lab.legend(loc="upper left", frameon=True)
fig_u04_lab.subplots_adjust(left=0.11, right=0.97, bottom=0.15, top=0.87)

assert "(s)" in ax_u04_lab.get_xlabel()
assert "(m)" in ax_u04_lab.get_ylabel()
assert len(legenda_u04.get_texts()) == 3
```



Gambar 7.1: Konstruksi lengkap setelah garis model, titik pengukuran, batang ketidakpastian, label bersatuan, dan legenda ditambahkan.

Memanggil `savefig` tanpa menetapkan ukuran, metadata, jenis huruf, dan garam SVG dapat menghasilkan byte berbeda. Fungsi `write_artifacts` menetapkan semua itu, menyimpan data dan deskripsi bersama gambar, lalu mengikat ketiganya dalam manifes SHA-256. Tutup kanvas lab setelah selesai agar proses panjang tidak menumpuk gambar di memori.

Daftar 7.7

```
plt.close(fig_u04_lab)
```

7.5 Sumbu, skala, dan satuan

Label Waktu, t (s) menyatakan nama variabel, lambang, dan satuan. Label Jarak, d (m) melakukan hal yang sama. Angka tanpa satuan tidak cukup ketika besaran fisis sedang dibandingkan.

Sumbu tegak pada gambar Unit 4 dimulai dari nol. Keputusan ini membuat tinggi visual sebanding dengan jarak yang dinyatakan. Memulai dari nol bukan aturan mutlak untuk semua grafik garis, tetapi pemotongan harus terlihat jelas dan tidak boleh dipakai untuk membesar-besarkan selisih.

Misalkan dua batang menyatakan 98 dan 100. Dengan garis dasar nol, rasio tingginya adalah

$$\frac{100 - 0}{98 - 0} = \frac{100}{98} \approx 1,02.$$

Jika garis dasar disembunyikan pada 97, rasionya menjadi

$$\frac{100 - 97}{98 - 97} = 3.$$

Selisih datanya tetap 2, tetapi batang kedua tampak tiga kali setinggi batang pertama. Pemotongan tersebut dapat berguna untuk melihat perubahan kecil, namun garis dasar dan nilai sebenarnya harus dinyatakan dengan terang.

7.6 Rasio aspek dan kemiringan

Rasio aspek adalah perbandingan lebar dan tinggi daerah gambar. Grafik yang sangat sempit dapat membuat garis yang sama tampak curam; grafik yang sangat lebar dapat membuatnya tampak datar. Kemiringan matematis model tetap

$$m = \frac{\Delta d}{\Delta t} = 2 \text{ m/s},$$

apa pun ukuran gambar di layar. Karena itu, baca nilai sumbu dan satuannya, bukan sudut piksel semata. Skrip unit menetapkan ukuran gambar dan batas sumbu secara eksplisit agar hasil tidak berubah mengikuti ukuran jendela.

7.7 Ketidakpastian bukan dekorasi

Pada contoh ini, batang galat berarti interval toleransi pengukuran $[d - 0,4, d + 0,4]$ meter. Batang itu **bukan** otomatis simpangan baku, interval kepercayaan, atau rentang semua nilai yang mungkin. Makna tersebut baru sah jika prosedur pengukuran atau model statistik menetapkan.

Semua nilai model berada di dalam interval pengukuran. Pernyataan yang sah adalah “data ini konsisten dengan model pada tujuh waktu yang diperiksa.” Pernyataan yang terlalu kuat adalah “data membuktikan bahwa model selalu benar.” Model lain juga mungkin cocok dengan tujuh interval yang sama.

Ketika membandingkan dua seri, gunakan definisi ketidakpastian yang sama atau jelaskan perbedaannya. Batang dengan makna berbeda tidak boleh dibandingkan seolah-olah identik.

7.8 Deskripsi tekstual dan aksesibilitas

Teks alternatif yang berguna tidak sekadar berkata “grafik garis”. Ia memuat:

1. tujuan grafik;
2. variabel dan satuannya;
3. pengodean tanda dan garis;
4. pola utama serta penyimpangan penting; dan
5. batas kesimpulan jika gambar mudah disalahartikan.

Skrip Unit 4 menanamkan judul dan deskripsi ke metadata SVG serta menulis deskripsi yang sama sebagai `unit04-alt.txt`. Data tetap tersedia sebagai CSV, sehingga pembaca tidak dipaksa memperoleh nilai dari posisi visual saja.

Legenda, bentuk, dan jenis garis juga mengulang informasi warna. Pengulangan semacam ini bukan pemborosan: ia membuat informasi dapat diakses melalui lebih dari satu petunjuk.

7.9 Artefak deterministik

Jalankan dari akar proyek:

```
python source/code/unit04_visualization.py --output-dir output/unit04
```

Perintah itu menghasilkan empat berkas:

- `unit04-data.csv`: data bertabel dengan satuan pada nama kolom;
- `unit04-figure.svg`: gambar vektor dengan skala dan metadata tetap;
- `unit04-alt.txt`: deskripsi visual dalam teks biasa; dan
- `unit04-manifest.json`: pertanyaan, kebijakan sumbu, makna pengodean, batas klaim, serta SHA-256 ketiga artefak lainnya.

Skrrip tidak memakai jaringan, waktu saat ini, atau bilangan acak. Urutan data, format angka, jenis huruf, ukuran gambar, garam pengenalan SVG, dan metadata ditetapkan. Pada lingkungan perangkat lunak yang sama, dua eksekusi harus menghasilkan byte yang sama.

Deterministik tidak berarti benar. Kesalahan yang ditulis secara deterministik tetap merupakan kesalahan. Hash membantu memastikan bahwa dua orang memeriksa byte yang sama; tes dan penalaran memeriksa apakah byte itu mewakili maksud yang benar.

7.10 Bukti visual dan bukti matematis

Gambar dapat menunjukkan pola, anomali, dan calon contoh penyangkal. Ia sangat berguna untuk mengajukan pertanyaan berikutnya. Akan tetapi, tujuh titik tidak membuktikan persamaan bagi semua waktu.

Jika gerak **didefinisikan** mempunyai posisi awal 1 meter dan kecepatan tetap 2 meter per detik, maka persamaan

$$d(t) = 1 + 2t$$

mengikuti definisi gerak berkecepatan tetap. Itu adalah argumen model. Grafik kemudian memeriksa apakah pengukuran terbatas konsisten dengan konsekuensi model tersebut.

Sebaliknya, satu pengamatan yang intervalnya tidak memuat nilai model dapat menjadi bukti bahwa model, ketidakpastian, data, atau implementasi perlu diperiksa. Ia tidak secara otomatis mengatakan bagian mana yang salah.

7.11 Latihan

7.11.1 Latihan 1 - pertanyaan dan pengodean

Seorang teman membuat grafik tujuh pengukuran tetapi tidak menampilkan garis model. Sebutkan informasi yang hilang dan satu perbaikan yang tidak hanya mengandalkan warna.

Petunjuk

Kembali ke pertanyaan utama: dua objek apa yang sedang dibandingkan?

Jawaban dan solusi

Grafik kehilangan nilai pembanding $d = 2t + 1$, sehingga pembaca tidak dapat menilai konsistensi pengukuran terhadap model secara langsung. Tambahkan garis model putus-putus dengan label pada legenda atau label langsung. Jenis garis yang berbeda membuatnya tetap dapat dibedakan tanpa bergantung pada warna.

7.11.2 Latihan 2 - sumbu terpotong

Dua nilai adalah 98 dan 100. Hitung rasio tinggi tampaknya untuk garis dasar 0 dan 97. Jelaskan mengapa judul “nilai kedua tiga kali lebih besar” tidak sah.

Petunjuk

Tinggi tampak adalah nilai dikurangi garis dasar; besar data tetap nilai asli.

Jawaban dan solusi

Dengan garis dasar 0, rasionya $100/98 \approx 1,02$. Dengan garis dasar 97, rasio tinggi tampak adalah $(100 - 97)/(98 - 97) = 3$. Akan tetapi, nilai kedua hanya $100/98$ kali nilai pertama, bukan tiga kali. Angka 3 menggambarkan geometri gambar yang dipotong, bukan rasio data.

7.11.3 Latihan 3 - makna batang galat

Apakah batang $\pm 0,4$ meter pada unit ini merupakan interval kepercayaan 95%? Tuliskan pernyataan yang sah tentang batang tersebut.

Petunjuk

Gunakan hanya definisi ketidakpastian yang benar-benar diberikan.

Jawaban dan solusi

Tidak ada dasar untuk menyebutnya interval kepercayaan 95%. Unit hanya mendefinisikannya sebagai toleransi pengukuran dari $d - 0,4$ sampai $d + 0,4$ meter. Pernyataan yang sah: pada ketujuh waktu, nilai model berada di dalam interval toleransi yang dinyatakan.

7.11.4 Latihan 4 - deskripsi aksesibel

Tuliskan teks alternatif dua atau tiga kalimat untuk grafik Unit 4. Jangan sekadar mengulang judul.

Petunjuk

Sebutkan sumbu, pengodean, pola, ketidakpastian, dan batas klaim.

Jawaban dan solusi

Contoh: “Grafik membandingkan jarak terukur dalam meter terhadap waktu 0–6 detik dengan model $d = 2t + 1$. Titik lingkaran dan batang $\pm 0,4$ meter mewakili pengukuran, sedangkan garis putus-putus mewakili model; semua nilai model berada di dalam interval pengukuran. Kecocokan tujuh titik mendukung konsistensi pada data ini, bukan bukti bahwa model berlaku untuk semua waktu.”

7.11.5 Latihan 5 - gambar dan bukti

Grafik menunjukkan 1.000 titik tepat pada kurva $y = x^2$. Apakah hal itu membuktikan bahwa program selalu menghitung kuadrat dengan benar? Berikan satu langkah komputasional dan satu langkah matematis yang memperkuat pemeriksaan.

Petunjuk

Gambar hanya memuat masukan berhingga dan resolusi visual terbatas.

💡 Jawaban dan solusi

Tidak. Seribu titik hanya memeriksa masukan yang dipilih, dan perbedaan kecil dapat tersembunyi oleh resolusi gambar. Langkah komputasional: uji nilai batas, nilai negatif, dan masukan acak deterministik dengan membandingkan keluaran numerik terhadap hasil eksak. Langkah matematis: periksa definisi algoritme dan tunjukkan bahwa operasi yang diterapkan pada setiap masukan memang menghasilkan $x \cdot x$ pada domain yang dinyatakan.

7.11.6 Latihan penguasaan - data ke plot ke manifes

Jalankan `write_artifacts` pada direktori baru. Buktikan dengan kode bahwa alur tersebut menghasilkan data CSV, gambar SVG statis, deskripsi teks, dan manifes JSON; bahwa manifes memuat satuan serta deskripsi aksesibel; dan bahwa hash ketiga artefak yang dicatat manifes cocok dengan byte sebenarnya.

i Petunjuk

Gunakan `paths = write_artifacts(direktori)`, baca JSON dengan `json.loads`, lalu hitung `hashlib.sha256(paths[kunci].read_bytes()).hexdigest()` untuk `data`, `figure`, dan `alt_text`.

! Pemeriksaan mandiri

Kode berikut harus selesai tanpa `AssertionError`. Direktori sementara menjaga pemeriksaan agar tidak mengubah artefak rilis.

```

import hashlib
import json
from pathlib import Path
import tempfile

from source.code.unit04_visualization import ALT_TEXT, write_artifacts

with tempfile.TemporaryDirectory() as tmp_u04:
    paths_u04 = write_artifacts(Path(tmp_u04))
    manifest_u04 =
    json.loads(paths_u04["manifest"].read_text(encoding="utf-8"))

    assert set(paths_u04) == {"data", "figure", "alt_text", "manifest"}
    assert manifest_u04["units"] == {
        "distance": "m",
        "time": "s",
        "uncertainty": "m",
    }
    assert manifest_u04["accessibility"]["description"] == ALT_TEXT
    for kunci_u04 in ("data", "figure", "alt_text"):
        digest_u04 =
        hashlib.sha256(paths_u04[kunci_u04].read_bytes()).hexdigest()
        assert manifest_u04["artifacts"][paths_u04[kunci_u04].name] ==
        digest_u04

```

💡 Jawaban dan solusi

Pemecahan 7.1.

```

import hashlib
import json
from pathlib import Path
import tempfile

from source.code.unit04_visualization import ALT_TEXT, write_artifacts

with tempfile.TemporaryDirectory() as direktori_u04:
    artefak_u04 = write_artifacts(Path(direktori_u04))
    manifes_u04 = json.loads(
        artefak_u04["manifest"].read_text(encoding="utf-8")
    )

    assert artefak_u04["data"].suffix == ".csv"
    assert artefak_u04["figure"].suffix == ".svg"
    assert artefak_u04["alt_text"].read_text(encoding="utf-8").strip()
    == ALT_TEXT
    assert manifes_u04["accessibility"]["svg_metadata"] is True
    assert manifes_u04["construction"][-1] == "deterministic_save"

    for nama_u04 in ("data", "figure", "alt_text"):
        byte_u04 = artefak_u04[nama_u04].read_bytes()
        hash_u04 = hashlib.sha256(byte_u04).hexdigest()
        assert manifes_u04["artifacts"][artefak_u04[nama_u04].name] ==
        hash_u04

```

CSV mempertahankan nilai dan satuan, SVG memberi keluaran statis, teks alternatif menyediakan jalur nonvisual, dan manifes mengikat byte ketiganya. Kecocokan hash membuktikan identitas byte, bukan kebenaran model.

7.12 Ringkasan

- Visualisasi dimulai dari pertanyaan, bukan pilihan warna.
- Posisi, tanda, garis, sumbu, skala, dan satuan harus mempunyai arti eksplisit.
- Sumbu terpotong dan rasio aspek dapat mengubah kesan tanpa mengubah data.
- Makna ketidakpastian harus dinyatakan; batang galat tidak membawa makna statistik secara otomatis.
- Data bertabel, teks alternatif, gambar, dan manifes hash membuat artefak dapat diperiksa melalui lebih dari satu jalur.

- Konstruksi bertahap melalui `fig`, `ax`, `plot`, `scatter`, dan `errorbar` membuat pengodean, satuan, legenda, serta penyimpanan dapat diuji terpisah.
- Grafik memberi bukti visual terbatas dan dapat menemukan kejanggalan; bukti umum tetap memerlukan argumen yang mencakup seluruh domain.

8 Komputasi Eksak dan Simbolik

8.1 Tujuan belajar

Setelah menyelesaikan unit ini, pembaca dapat:

- membedakan ekspresi simbolik dari nilai hasil substitusi;
- memakai `Fraction` dan `SymPy` untuk mempertahankan hasil eksak;
- menyatakan asumsi dan domain sebelum menyederhanakan ekspresi;
- memeriksa ekuivalensi aljabar tanpa menghapus pembatasan domain;
- memfaktorkan dan menyelesaikan persamaan sederhana secara eksak;
- menjelaskan batas antara hasil sistem aljabar komputer dan bukti matematis;
- menghasilkan rekaman simbolik kanonis yang dapat dibandingkan byte demi byte; dan
- memakai `SageMath` lokal untuk `ZZ`, `QQ`, `RR`, `SR`, `parent` dan `coercion`, gelang polinom, faktorisasi, penyelesaian eksak, serta konversi pendekatan yang dinyatakan.

Prasyarat lokal: Unit 1 dan Unit 2, operasi polinomial, persamaan kuadrat, dan pengertian domain fungsi. Jalur pertama memakai pustaka standar Python dan `SymPy`; lab `SageMath` 9.5 lokal merupakan bagian wajib unit. Tidak diperlukan kalkulus atau aljabar linear.

8.2 Ekspresi bukan nilai

Ekspresi

$$p(x) = x^2 - 1$$

memuat simbol bebas x . Ia belum mempunyai satu nilai numerik. Substitusi $x = 3$ menghasilkan nilai eksak 8, tetapi tidak mengubah ekspresi asal menjadi bilangan.

```
import sympy as sp

x = sp.Symbol("x")
p = x**2 - 1
(p, p.free_symbols, p.subs(x, 3))
```

Daftar 8.1

```
(x**2 - 1, {x}, 8)
```

Perbedaan ini penting bagi notebook yang dijalankan tidak berurutan. Nama `x` dalam kode dapat menunjuk objek Python, sedangkan simbol SymPy bernama `x` merupakan bagian dari pohon ekspresi. Tampilkan dan simpan pohon yang sebenarnya dipakai, bukan hanya hasil terakhir di layar.

Untuk bilangan rasional kecil, pustaka standar sudah cukup:

```
from fractions import Fraction

Fraction(1, 10) + Fraction(2, 10)
```

Daftar 8.2

```
Fraction(3, 10)
```

`Fraction(3, 10)` dan `sp.Rational(3, 10)` sama-sama mempertahankan nilai rasional eksak. Pilih batas antarpustaka secara eksplisit; jangan mengubahnya menjadi `float` hanya untuk memperoleh keluaran yang tampak lebih pendek.

8.3 Asumsi dan domain adalah bagian dari soal

Tanpa keterangan tanda, rumus $\sqrt{x^2} = x$ tidak benar untuk setiap bilangan real. Jika x real, hasil yang benar ialah $|x|$. Jika $x > 0$, barulah hasilnya x .

```
x_real = sp.Symbol("x", real=True)
x_positive = sp.Symbol("x", positive=True)

(
    sp.simplify(sp.sqrt(x_real**2)),
    sp.simplify(sp.sqrt(x_positive**2)),
)
```

Nama cetak kedua simbol sama, tetapi asumsi internalnya berbeda. Oleh sebab itu, sebuah rekaman komputasi simbolik harus menyimpan definisi simbol atau pernyataan domain yang menyertainya.

Daftar 8.3

`(Abs(x), x)`

Domain juga harus dipertahankan ketika faktor dibatalkan. Untuk

$$r(x) = \frac{x^2 - 1}{x - 1},$$

SymPy memperoleh $x + 1$ setelah pembatalan. Kedua rumus memberi nilai sama pada domain bersama, yaitu $x \neq 1$. Namun, keduanya bukan fungsi yang sama jika domain asal ikut menjadi bagian definisi: rumus asal tidak terdefinisi pada $x = 1$, sedangkan $x + 1$ bernilai 2 di sana.

8.4 Kesamaan struktur dan ekuivalensi

Operator `==` pada dua ekspresi SymPy terutama membandingkan struktur kanonisnya. Ia bukan pertanyaan universal “apakah kedua rumus ini mempunyai nilai sama bagi semua masukan yang sah?”. Contohnya, ekspresi $(x + 1)^2$ dan $x^2 + 2x + 1$ mempunyai struktur berbeda tetapi merupakan polinom yang sama.

```
left = (x + 1) ** 2
right = x**2 + 2*x + 1

structurally_equal = left == right
polynomial_identity = sp.expand(left - right) == 0
(structurally_equal, polynomial_identity)
```

Daftar 8.4

`(False, True)`

Prosedur pemeriksaan harus sesuai dengan kelas ekspresi. Ekspansi selisih cocok untuk identitas polinomial. `simplify(left-right) == 0` berguna sebagai pemeriksaan lain, tetapi bukan pengganti pencatatan domain, cabang akar, asumsi, atau titik singular.

8.5 Faktorisasi dan penyelesaian eksak

Komputasi simbolik mempertahankan akar dan faktor tanpa pendekatan desimal.

```
x = sp.Symbol("x")
polynomial = x**4 - 1
factored = sp.factor(polynomial)
(factored, sp.expand(factored) == polynomial)
```

Daftar 8.5

```
((x - 1)*(x + 1)*(x**2 + 1), True)
```

Faktorisasi yang diharapkan ialah

$$(x - 1)(x + 1)(x^2 + 1).$$

Pemeriksaan dengan `expand` menguji bahwa hasil tersebut kembali menjadi polinom masukan. Untuk persamaan $x^2 = 2$, domain solusi harus dinyatakan:

```
x = sp.Symbol("x", real=True)
solutions = sp.solve(sp.Eq(x**2, 2), x, domain=sp.S.Reals)
solutions
```

Daftar 8.6

$$\{-\sqrt{2}, \sqrt{2}\}$$

Hasil real eksaknya adalah $\{-\sqrt{2}, \sqrt{2}\}$. Substitusi setiap kandidat ke $x^2 - 2$ harus memberikan residual eksak nol. Jika domain diubah, bentuk atau jumlah solusi dapat berubah; karena itu, jangan menyimpan daftar akar tanpa persamaan dan domainnya.

8.6 Hasil simbolik dan bukti

Keluaran sistem aljabar komputer mempunyai beberapa peran yang berbeda:

- **kandidat:** `solve` mengusulkan nilai yang kemudian dapat diperiksa;
- **sertifikat lokal:** ekspansi kembali atau substitusi residual memeriksa hubungan tertentu;
- **eksperimen:** banyak contoh dapat menguji sebuah dugaan; dan

- **bukti:** argumen menjelaskan mengapa klaim berlaku pada seluruh domain.

Faktorisasi $x^4 - 1$ dapat diperiksa langsung dengan distributivitas. Namun, keberhasilan satu perintah tidak membuktikan bahwa setiap keluaran algoritme untuk setiap masukan pasti benar. Klaim lebih luas juga bergantung pada spesifikasi algoritme dan kebenaran implementasinya. Dalam tulisan matematis, nyatakan tepatnya hubungan mana yang diperiksa komputer dan langkah umum mana yang dibuktikan secara matematis.

8.7 Keluaran kanonis

Tampilan dua dimensi yang nyaman dibaca dapat berubah karena lebar terminal, pengaturan printer, atau versi pustaka. Unit ini memakai `sympy.srepr` untuk merekam pohon ekspresi dan JSON dengan kunci terurut, inden tetap, UTF-8, serta akhir baris LF. Versi SymPy ikut disimpan karena bentuk kanonis pustaka dapat berubah antarrilis.

Jalankan skrip dari akar proyek:

```
python source/code/unit05_symbolic.py --output output/unit05-results.json
```

Medan `core_sha256` mengikat rekaman inti sebelum medan hash ditambahkan. Menjalankan perintah yang sama pada lingkungan yang sama harus menghasilkan byte identik. Hash bukan bukti kebenaran matematika; ia membuktikan bahwa dua urutan byte sama.

8.8 Lab SageMath lokal wajib

SageMath menyatukan banyak struktur matematika dalam satu sistem yang memakai sintaks Python. Dalam unit ini, Sage bukan layanan web dan bukan potongan opsional. Lab dijalankan lokal dengan SageMath 9.5 yang dibekukan di WSL Ubuntu 22.04. SageCell atau layanan jarak jauh tidak memenuhi persyaratan karena runtime, ketersediaan jaringan, dan byte hasilnya tidak berada di bawah kendali eksperimen lokal.

Jika profil itu belum tersedia, ikuti [panduan penyiapan lokal](#) dan jalankan verifikasi lock sebelum lab. Perbedaan versi harus dicatat; jangan menyebut profil yang berbeda sebagai lingkungan rilis yang identik.

Dari akar proyek pada PowerShell, jalankan:

```
wsl.exe -d Ubuntu-22.04 -- /usr/bin/sage -python  
source/code/unit05_sage_lab.py --output output/unit05-sage-results.json  
wsl.exe -d Ubuntu-22.04 -- /usr/bin/sage -python tests/test_unit05_sage.py
```

Perintah pertama menulis JSON deterministik ke jalur relatif yang diberikan. Perintah kedua menjalankan seluruh tes lab di runtime yang sama. Blok Sage di pembaca ini sengaja berupa pagar `python` biasa, bukan blok Quarto `{python}`. Karena itu, build Windows tidak mencoba mengimpor Sage melalui kernel Python biasa.

8.8.1 Parent menentukan tempat objek hidup

Sage menanyakan `parent()` suatu objek, bukan hanya tipe implementasi Python. Empat parent inti lab ini adalah:

Nama	Parent matematika	Peran
ZZ	gelang bilangan bulat	aritmetika bulat eksak
QQ	lapangan bilangan rasional	pembagian rasional eksak
RR	lapangan real 53 bit	pendekatan titik-mengambang
SR	gelang simbolik	ekspresi dan persamaan simbolik

```
from sage.all import QQ, RR, SR, ZZ

bilangan_bulat = ZZ(6)
rasional = QQ(1) / 3
pendekatan = RR(rasional)
simbolik = SR(rasional)

print(bilangan_bulat.parent())
print(rasional.parent())
print(pendekatan.parent())
print(simbolik.parent())
```

Nilai cetak saja tidak cukup. $1/3$ dalam QQ merupakan nilai eksak, sedangkan nilai dalam RR adalah pendekatan pada presisi parent-nya.

8.8.2 Coercion harus dapat dijelaskan

Coercion adalah konversi kanonis yang dipilih Sage ketika operasi mencampurkan parent. Misalnya, $ZZ(6) + QQ(1)/3$ hidup di QQ karena ada peta kanonis dari ZZ ke QQ. Lab memeriksa peta ZZ ke QQ, QQ ke RR, dan QQ ke SR secara eksplisit.

```

from sage.all import QQ, RR, SR, ZZ

assert QQ.has_coerce_map_from(ZZ)
assert RR.has_coerce_map_from(QQ)
assert SR.has_coerce_map_from(QQ)
campuran = ZZ(6) + QQ(1) / 3
assert campuran.parent() is QQ

```

Coercion yang tersedia tidak berarti semua perubahan representasi tanpa kerugian. Pemetaan QQ ke RR mengubah rasional eksak menjadi pendekatan. Karena itu, kode ilmiah harus menuliskan titik konversinya.

8.8.3 Gelang polinom dan faktorisasi eksak

Parent polinom menyimpan gelang koefisien. `PolynomialRing(QQ, "x")` menyatakan polinom satu variabel dengan koefisien rasional. Ini lebih spesifik daripada sekadar ekspresi simbolik yang kebetulan tampak seperti polinom.

```

from sage.all import PolynomialRing, QQ

R = PolynomialRing(QQ, "x")
x = R.gen()
p = x**4 - 1
faktor = p.factor()

assert p.parent() is R
assert R.base_ring() is QQ
assert faktor.prod() == p
print(faktor)

```

Faktorisasi tepatnya $(x - 1)(x + 1)(x^2 + 1)$. Pemeriksaan `prod()` mengalikan faktor di dalam parent yang sama dan membandingkannya dengan polinom asal.

8.8.4 Penyelesaian dalam SR dan residual eksak

Untuk persamaan simbolik, domain dan parent masih harus dicatat. Lab memakai SR, meminta solusi berbentuk kamus, lalu mensubstitusikan setiap kandidat ke residual.

```

from sage.all import SR, solve

y = SR.var("y")
solusi = solve(y**2 == 2, y, solution_dict=True)
nilai = [baris[y] for baris in solusi]
assert all((akar**2 - 2).simplify_full() == 0 for akar in nilai)

```

Residual nol memeriksa dua kandidat yang dikembalikan. Ia tidak sendirinya membuktikan bahwa tidak ada solusi lain pada domain berbeda.

8.8.5 Konversi pendekatan harus eksplisit

Jangan mengubah objek eksak menjadi desimal hanya agar tampilannya pendek. Pertahankan objek asal dan tuliskan parent tujuan:

```

from sage.all import QQ, RR

eksak = QQ(1) / 3
hampiran = RR(eksak)
assert eksak.parent() is QQ
assert hampiran.parent() is RR
print(eksak, hampiran, RR.precision())

```

`RR(eksak)` adalah keputusan representasi yang dapat ditemukan dalam sumber. Rekaman lab menyimpan nilai eksak, teks konversi, nilai pendekatan, parent, dan presisi 53 bit.

8.8.6 Rekaman Sage yang dapat diaudit

`unit05-sage-results.json` mencatat SageMath 9.5, keharusan eksekusi lokal, parent, peta coercion, gelang polinom, faktor, solusi, residual, konversi ke `RR`, dan `core_sha256`. Semua objek diubah menjadi bentuk JSON yang ditentukan sebelum serialisasi. Dua eksekusi dengan runtime yang sama harus menghasilkan byte sama.

Skrip menolak versi selain 9.5. Pembatasan ini bukan klaim bahwa versi lain salah; ia mengikat bukti kerja pada runtime yang benar-benar diuji. Hasil SageMath tetap bukan pengganti bukti umum. Parent dan residual menjelaskan apa yang diperiksa oleh komputasi; argumen matematika menjelaskan mengapa klaim berlaku pada domainnya.

8.9 Latihan

8.9.1 Latihan 1 - ekspresi dan nilai

Bentuk ekspresi $q(t) = t^3 - 2t$ di SymPy. Sebutkan simbol bebasnya, lalu evaluasi pada $t = 1/2$ secara eksak.

Petunjuk

Gunakan `Symbol`, `Rational`, atribut `free_symbols`, dan `subs`.

Jawaban dan solusi

Kode `t=sp.Symbol("t"); q=t**3-2*t` menghasilkan simbol bebas `{t}`. Substitusi `q.subs(t, sp.Rational(1,2))` memberi $1/8 - 1 = -7/8$. Ekspresi `q` tetap memuat `t`; $-7/8$ adalah nilai pada satu masukan, bukan pengganti ekspresi asal.

8.9.2 Latihan 2 - asumsi yang hilang

Seorang pembaca menyederhanakan $\sqrt{z^2}$ menjadi z untuk semua bilangan real. Berikan contoh penyangkal dan tuliskan hasil umum yang benar.

Petunjuk

Coba satu bilangan real negatif.

Jawaban dan solusi

Ambil $z = -3$. Maka $\sqrt{z^2} = \sqrt{9} = 3$, sedangkan $z = -3$, sehingga penyederhanaan itu gagal. Untuk z real, bentuk umum yang benar ialah $\sqrt{z^2} = |z|$. Bentuk z sah jika ditambahkan asumsi $z \geq 0$, dan khusus simbol SymPy positif dapat dideklarasikan dengan `positive=True`.

8.9.3 Latihan 3 - ekuivalensi dan domain

Bandingkan $(u^2 - 4)/(u - 2)$ dengan $u + 2$. Nyatakan pada himpunan mana nilainya sama dan jelaskan mengapa domain kedua fungsi tidak otomatis sama.

i Petunjuk

Faktorkan pembilang dan periksa penyebut sebelum membatalkan faktor.

💡 Jawaban dan solusi

Karena $u^2 - 4 = (u - 2)(u + 2)$, kedua rumus memberi nilai sama untuk setiap $u \neq 2$. Rumus pecahan tidak terdefinisi pada $u = 2$, sementara $u + 2$ bernilai 4. Jadi keduanya setara pada domain bersama, tetapi bukan fungsi dengan domain yang sama kecuali pembatasan $u \neq 2$ dipertahankan.

8.9.4 Latihan 4 - faktor, akar, dan pemeriksaan

Faktorkan $v^3 - v$, cari semua akar realnya secara eksak, lalu tuliskan dua pemeriksaan yang dapat dilakukan tanpa mempercayai tampilan keluaran.

i Petunjuk

Keluarkan faktor v , kemudian faktorkan selisih kuadrat.

💡 Jawaban dan solusi

$v^3 - v = v(v^2 - 1) = v(v - 1)(v + 1)$, sehingga akar realnya ialah $(-1, 0, 1)$. Pemeriksaan pertama: ekspansi $v \cdot (v - 1) \cdot (v + 1)$ harus kembali ke $v^3 - v$. Pemeriksaan kedua: substitusi setiap akar ke polinom harus memberi residual eksak nol. Kedua pemeriksaan ini mengikat klaim lokal yang jelas.

8.9.5 Latihan 5 - artefak simbolik yang stabil

Mengapa menyimpan hanya tangkapan layar keluaran `pretty print` tidak cukup untuk eksperimen simbolik yang dapat direproduksi? Sebutkan sedikitnya empat unsur rekaman yang lebih baik.

i Petunjuk

Pikirkan pohon ekspresi, asumsi, domain, versi, dan byte keluaran.

💡 Jawaban dan solusi

Tangkapan layar dapat kehilangan struktur, mudah berubah karena tata letak, dan tidak dapat dievaluasi ulang. Rekaman yang lebih baik memuat (1) pohon ekspresi kanonis, (2) asumsi simbol, (3) domain masalah, (4) versi SymPy atau SageMath, (5) masukan dan metode, serta (6) JSON atau format teks dengan aturan urutan dan akhir baris yang tetap. Hash kemudian dapat membandingkan byte, sedangkan residual atau ekspansi memeriksa klaim matematis tertentu.

8.9.6 Latihan Sage 1 - parent, coercion, dan faktor

Di SageMath 9.5 lokal, bentuk $\mathbb{Z}\mathbb{Z}(5)$, $\mathbb{Q}\mathbb{Q}(2)/3$, dan jumlah keduanya. Catat parent setiap objek dan jelaskan coercion yang terjadi. Lalu buat gelang $\mathbb{Q}\mathbb{Q}[z]$, faktorkan $z^4 - 1$, dan periksa bahwa hasil kali faktor kembali ke polinom asal.

i Petunjuk

Gunakan `parent()`, `QQ.has_coerce_map_from(ZZ)`, `PolynomialRing(QQ, "z")`, serta metode `factor()` dan `prod()`.

! Pemeriksaan mandiri

Jalankan blok ini hanya melalui `/usr/bin/sage -python`, bukan kernel Quarto Windows.

```
from sage.all import PolynomialRing, QQ, ZZ

a = ZZ(5)
b = QQ(2) / 3
c = a + b
Rz = PolynomialRing(QQ, "z")
z = Rz.gen()
pz = z**4 - 1
fz = pz.factor()

assert a.parent() is ZZ
assert b.parent() is QQ
assert c.parent() is QQ
assert QQ.has_coerce_map_from(ZZ)
assert Rz.base_ring() is QQ
assert fz.prod() == pz
```

💡 Jawaban dan solusi

```
from sage.all import PolynomialRing, QQ, ZZ

bilangan_bulat = ZZ(5)
bilangan_rasional = QQ(2) / 3
hasil_campuran = bilangan_bulat + bilangan_rasional

assert str(bilangan_bulat.parent()) == "Integer Ring"
assert str(bilangan_rasional.parent()) == "Rational Field"
assert hasil_campuran == QQ(17) / 3
assert hasil_campuran.parent() is QQ

gelang = PolynomialRing(QQ, "z")
z = gelang.gen()
polinom = z**4 - 1
faktorisasi = polinom.factor()
assert faktorisasi.prod() == polinom
assert {str(faktor) for faktor, _ in faktorisasi} == {
    "z - 1",
    "z + 1",
    "z^2 + 1",
}
```

Sage memakai peta kanonis ZZ ke QQ, sehingga hasil penjumlahan hidup di QQ dan tetap eksak. Polinom hidup di $\mathbb{Q}[z]$; mengalikan faktor di parent yang sama memberi sertifikat lokal bagi faktorisasi tersebut.

8.9.7 Latihan Sage 2 - solusi eksak dan pendekatan

Selesaikan $w^2 = 5$ di SR, periksa residual setiap solusi secara eksak, lalu ubah akar positif secara eksplisit ke RR. Simpan nilai eksak, parent asal, nilai pendekatan, parent tujuan, dan presisinya. Jelaskan apa yang dibuktikan residual dan apa yang tidak.

i Petunjuk

Gunakan `solve(..., solution_dict=True)`, `simplify_full()`, dan `RR(akar)`. Jangan mengganti objek SR sebelum residual eksak diperiksa.

! Pemeriksaan mandiri

```
from sage.all import RR, SR, solve

w = SR.var("w")
baris_solusi = solve(w**2 == 5, w, solution_dict=True)
akar = [baris[w] for baris in baris_solusi]

assert {str(nilai) for nilai in akar} == {"-sqrt(5)", "sqrt(5)"}
assert all((nilai**2 - 5).simplify_full() == 0 for nilai in akar)

akar_positif = next(nilai for nilai in akar if nilai > 0)
hampiran = RR(akar_positif)
assert akar_positif.parent() is SR
assert hampiran.parent() is RR
assert RR.precision() == 53
```

💡 Jawaban dan solusi

```
from sage.all import RR, SR, solve

w = SR.var("w")
solusi = solve(w**2 == 5, w, solution_dict=True)
nilai_eksak = sorted((baris[w] for baris in solusi), key=str)
residual_nol = [
    bool((nilai**2 - 5).simplify_full() == 0)
    for nilai in nilai_eksak
]

akar_positif = next(nilai for nilai in nilai_eksak if nilai > 0)
nilai_hampiran = RR(akar_positif)
rekaman = {
    "exact": str(akar_positif),
    "exact_parent": str(akar_positif.parent()),
    "approximate": str(nilai_hampiran),
    "approximate_parent": str(nilai_hampiran.parent()),
    "precision_bits": RR.precision(),
}

assert residual_nol == [True, True]
assert rekaman["exact"] == "sqrt(5)"
assert rekaman["exact_parent"] == "Symbolic Ring"
assert rekaman["approximate_parent"].startswith("Real Field")
assert rekaman["precision_bits"] == 53
```

Residual eksak memeriksa bahwa dua kandidat memenuhi persamaan. Konversi `RR(akar_positif)` menghasilkan pendekatan 53 bit tanpa menghapus rekaman objek eksak. Pemeriksaan ini tidak membuktikan kelengkapan algoritme `solve` untuk semua persamaan atau semua domain.

8.10 Ringkasan

- Ekspresi simbolik menyatakan struktur; substitusi menghasilkan nilai pada masukan tertentu.
- Aritmetika eksak mempertahankan hubungan rasional dan aljabar tanpa pembulatan desimal.
- Asumsi serta domain menentukan penyederhanaan mana yang sah.

- Kesamaan struktur, ekuivalensi pada domain bersama, dan kesamaan fungsi merupakan tiga pertanyaan berbeda.
- Faktorisasi, ekspansi kembali, penyelesaian eksak, dan residual saling melengkapi sebagai pemeriksaan lokal.
- Keluaran sistem aljabar komputer adalah bukti kerja yang harus dibatasi klaimnya, bukan pengganti otomatis bagi bukti umum.
- Representasi kanonis, versi pustaka, dan hash membuat artefak dapat dibandingkan.
- Lab SageMath 9.5 lokal wajib memakai `ZZ`, `QQ`, `RR`, `SR`, `parent` dan `coercion`, gelang polinom, faktorisasi, penyelesaian, residual, serta konversi pendekatan eksplisit; layanan jarak jauh tidak menggantikannya.

9 Titik-Mengambang, Galat, dan Stabilitas

9.1 Tujuan belajar

Setelah menyelesaikan unit ini, pembaca dapat:

- menjelaskan model dasar bilangan titik-mengambang biner;
- membaca `ulp`, epsilon mesin, serta batas luapan dan aliran-bawah;
- mengenali pembatalan yang menghilangkan angka signifikan;
- membedakan galat maju dari galat mundur;
- membedakan pengondisian masalah dari stabilitas algoritme;
- memilih reformulasi dan metode penjumlahan yang lebih stabil;
- membandingkan evaluasi naif dengan `scipy.special` pada domain yang dinyatakan;
- menetapkan toleransi sebagai anggaran galat sebelum hasil diperiksa; dan
- membatasi bukti eksperimen terhadap kasus serta lingkungan yang dijalankan.

Prasyarat lokal: Unit 1, Unit 2, dan Unit 3; pangkat bilangan bulat, akar kuadrat, nilai mutlak, fungsi rasional, dan galat relatif pada tingkat A30. Tidak diperlukan kalkulus.

9.2 Model biner yang berhingga

Pada lingkungan proyek ini, `float` memakai format biner 64-bit. Secara sederhana, bilangan berhingga normal disimpan dalam bentuk

$$(-1)^s (1.b_1b_2 \dots b_{52})_2 2^e.$$

Ada satu bit tanda, eksponen berhingga, dan 53 bit ketelitian signifikan jika angka satu di depan titik ikut dihitung. Karena hanya ada sejumlah pola bit, hanya sejumlah bilangan real yang dapat disimpan. Hasil operasi biasanya dibulatkan ke bilangan representabel terdekat.

Metode `hex()` memperlihatkan nilai biner sebuah `float` tanpa kehilangan informasi:

```
x = 0.1
(x, x.hex(), x.as_integer_ratio())
```

Daftar 9.1

(0.1, '0x1.999999999999ap-4', (3602879701896397, 36028797018963968))

Pecahan dari `as_integer_ratio()` adalah nilai rasional **yang benar-benar tersimpan**. Nilai itu dekat dengan $1/10$, tetapi bukan $1/10$.

9.3 ulp, epsilon, dan jarak yang berubah

Satu *unit in the last place* (ulp) adalah jarak lokal antara bilangan representabel yang berdekatan. Jarak ini berubah mengikuti besar bilangan.

```
import math
import sys

(
    sys.float_info.epsilon,
    math.ulp(1.0),
    math.ulp(float(2**53)),
)
```

Daftar 9.2

(2.220446049250313e-16, 2.220446049250313e-16, 2.0)

Di sekitar 1, `ulp(1.0)` sama dengan `sys.float_info.epsilon`, yaitu selisih antara 1 dan bilangan `float` berikutnya. Beberapa buku memakai istilah “epsilon mesin” untuk angka ini. Buku lain memakai *unit roundoff* $u = \epsilon/2$ dalam model pembulatan-ke-terdekat. Karena istilahnya tidak seragam, nyatakan definisi yang dipakai.

Di sekitar 2^{53} , satu ulp bernilai 2. Akibatnya:

```
besar = float(2**53)
(besar + 1.0 == besar, besar + 2.0 == besar + 2)
```

Ini bukan kesalahan Python dalam menjumlahkan bilangan bulat. Masalahnya adalah hasil sedang dipaksa kembali ke kisi `float64` yang pada skala itu berjarak 2.

Daftar 9.3

(True, True)

9.4 Luapan, subnormal, dan aliran-bawah

Ketelitian dan rentang adalah dua batas berbeda. `sys.float_info.max` adalah bilangan `float` berhingga terbesar. Mengalikannya dengan 2 menghasilkan takhingga positif pada lingkungan yang diuji. Di ujung kecil, `sys.float_info.min` adalah bilangan positif **normal** terkecil, bukan bilangan positif terkecil.

Bilangan subnormal mengisi sebagian celah menuju nol dengan ketelitian yang berkurang. `math.ulp(0.0)` memberi subnormal positif terkecil. Membaginya dua memberi nol karena tidak ada bilangan positif representabel yang lebih kecil.

```
terbesar = sys.float_info.max
subnormal_terkecil = math.ulp(0.0)
(
    terbesar * 2.0,
    sys.float_info.min,
    subnormal_terkecil,
    subnormal_terkecil / 2.0,
)
```

Daftar 9.4

(inf, 2.2250738585072014e-308, 5e-324, 0.0)

Periksa nilai dengan `math.isfinite` sebelum hasil takhingga atau nol akibat aliran-bawah diteruskan ke tahap berikutnya. Namun, pemeriksaan setelah kejadian tidak mengganti analisis skala sebelum komputasi.

9.5 Pembatalan dan reformulasi stabil

Pengurangan dua bilangan hampir sama dapat menghapus sebagian besar angka signifikan. Untuk x besar, pertimbangkan

$$d(x) = \sqrt{x+1} - \sqrt{x}.$$

Pada $x = 10^{16}$, penjumlahan $x + 1.0$ sudah membulat kembali ke x . Pengurangan langsung lalu menghasilkan nol. Sebelum menghitung, rasionalkan secara aljabar:

$$d(x) = \frac{(x+1) - x}{\sqrt{x+1} + \sqrt{x}} = \frac{1}{\sqrt{x+1} + \sqrt{x}}.$$

```
x = 1e16
langsung = math.sqrt(x + 1.0) - math.sqrt(x)
stabil = 1.0 / (math.sqrt(x + 1.0) + math.sqrt(x))
(x + 1.0 == x, langsung, stabil)
```

Daftar 9.5

(True, 0.0, 5e-09)

Bentuk kedua menghindari pengurangan dua akar yang hampir sama. Reformulasi ini berasal dari identitas aljabar pada bilangan real; program Unit 6 membandingkan kedua implementasi dengan referensi `Decimal` berketelitian tinggi. Referensi berketelitian tinggi tetap merupakan alat pemeriksaan, bukan definisi bukti yang berdiri sendiri.

9.6 SciPy: `expm1` stabil dekat nol

SciPy menyediakan implementasi khusus bagi fungsi

$$E(x) = \frac{e^x - 1}{x} \quad (x \neq 0), \quad E(0) = 1.$$

Domain matematisnya adalah seluruh bilangan real setelah nilai di nol didefinisikan melalui perpanjangan kontinu tersebut. Lab rinci pada unit ini memakai domain komputasional yang lebih sempit, $0 < |x| \leq 10^{-4}$, karena galat mundur relatif yang dicatat membagi dengan $|x|$. Oracle `Decimal` tidak menghitung `exp(x) - 1`: ia menjumlahkan deret pangkat bagi E dan E' tanpa pembatalan. Presisinya sekurang-kurangnya 80 digit dan bertambah menurut eksponen x , dengan 50 digit penjaga setelah orde x . Karena itu, koreksi pertama $x/2$ tetap terurai bahkan untuk subnormal binary64 terkecil. Batas atas dan pengecualian $x = 0$ adalah batas laporan galat lab, bukan batas domain matematis `expm1`.

Evaluasi langsung menghitung `exp(x)`, mengurangkan 1, lalu membagi dengan x . Ketika x kecil, `exp(x)` dan 1 hampir sama sehingga pengurangan dapat membuang angka signifikan. Pada nilai yang lebih ekstrem, `exp(x)` dapat dibulatkan tepat menjadi 1 dan pembilang naif menjadi nol.

```

import math
import scipy
from scipy import special

x = 1e-8
naive = (math.exp(x) - 1.0) / x
stable = float(special.exprel(x))

extreme_x = 1e-16
extreme_naive = (math.exp(extreme_x) - 1.0) / extreme_x
extreme_stable = float(special.exprel(extreme_x))

(scipy.__version__, naive, stable, extreme_naive, extreme_stable)

```

Daftar 9.6

```
('1.15.2', 0.999999993922529, 1.000000005, 0.0, 1.0)
```

Perbedaan itu tidak otomatis berarti masalah matematis berkondisi buruk. Ukuran pengondisian relatif lokal ialah

$$\kappa_E(x) = \left| \frac{x E'(x)}{E(x)} \right|.$$

Untuk $x \neq 0$,

$$E'(x) = \frac{(x-1)e^x + 1}{x^2}.$$

Dekat nol, $E(x)$ mendekati 1, $E'(x)$ mendekati $1/2$, dan $\kappa_E(x)$ mendekati $|x|/2$. Formula turunan ini dipakai sebagai alat diagnostik yang disediakan; menurunkan bukan prasyarat A30. Pada $x = 10^{-8}$, angka kondisi sekitar 5×10^{-9} , jadi gangguan relatif kecil pada masukan seharusnya tidak diperbesar. Galat besar evaluasi naif karena itu merupakan ketidakstabilan algoritmik, bukan kepekaan inheren fungsi.

Skrip pendamping membandingkan kedua hasil dengan referensi deret `Decimal` berpresisi adaptif pada domain lab yang dinyatakan. Galat maju yang dicatat ialah

$$e_f = |\widehat{E} - E(x)|.$$

Untuk memberi pembacaan galat mundur, skrip memakai estimasi orde pertama

$$|\Delta x| \approx \frac{e_f}{|E'(x)|}.$$

Ini bertanya berapa besar perubahan masukan yang kira-kira diperlukan agar keluaran terhitung menjadi keluaran tepat bagi masukan yang berubah. Estimasi itu **bukan** solusi invers eksak dan dilabeli demikian di JSON. Untuk $x = 10^{-8}$, estimasi mundur relatif evaluasi naif lebih besar dari 1, sedangkan estimasi SciPy jauh di bawah 10^{-6} . Galat maju SciPy juga berada di bawah 10^{-15} secara relatif pada run yang diuji.

Hasil tersebut merupakan bukti kerja bagi nilai, versi SciPy, dan lingkungan yang direkam. Ia menunjukkan contoh konkret bahwa implementasi khusus lebih akurat daripada formula naif dekat nol. Ia tidak membuktikan bahwa `scipy.special.exprel` stabil untuk setiap bilangan real atau setiap backend. Klaim umum tetap memerlukan analisis algoritme, model pembulatan, dan penanganan rentang.

9.7 Galat maju dan galat mundur

Misalkan masalah meminta $y = f(x)$ dan program menghasilkan $\hat{y}$.

- **Galat maju** mengukur jarak dari jawaban yang diinginkan: $|\hat{y} - f(x)|$.
- **Galat mundur** bertanya seberapa jauh masukan harus diubah agar keluaran menjadi jawaban tepat: cari Δx sehingga $\hat{y} = f(x + \Delta x)$.

Untuk masalah $y = \sqrt{a}$, kuadrat keluaran memberi masukan yang akan membuat keluaran itu eksak. Karena itu, ukuran mundur relatif yang mudah diperiksa adalah

$$\frac{|\hat{y}^2 - a|}{|a|}.$$

```
y_hat = math.sqrt(2.0)
residual = y_hat * y_hat - 2.0
(y_hat, residual)
```

Daftar 9.7

(1.4142135623730951, 4.440892098500626e-16)

Galat mundur kecil berarti hasil merupakan solusi tepat bagi masalah yang sedikit terganggu. Apakah galat majunya juga kecil bergantung pada pengondisian masalah.

9.8 Pengondisian bukan stabilitas

Pengondisian adalah sifat pertanyaan matematika: seberapa besar keluaran berubah ketika masukan sedikit berubah? **Stabilitas algoritmik** adalah sifat cara menghitung: apakah pembulatan internal menambah galat jauh melampaui kepekaan masalah?

Untuk

$$f(x) = \frac{1}{1-x},$$

masukan dekat 1 sangat peka. Pada $x = 0,99999999$, gangguan sekitar 10^{-12} dapat diperbesar kira-kira 10^8 kali dalam galat relatif keluaran. Ini adalah masalah berkondisi buruk di dekat titik singular $x = 1$. Mengganti bahasa pemrograman tidak dapat menghapus kepekaan matematis tersebut.

Sebaliknya, masalah $d(x) = \sqrt{x+1} - \sqrt{x}$ untuk x besar tidak memerlukan jawaban nol. Bentuk pengurangan langsung kehilangan informasi, sedangkan bentuk rasional mempertahankannya. Di sini perbedaan utamanya adalah stabilitas cara evaluasi.

Empat kemungkinan harus dibedakan:

Masalah	Algoritme	Makna praktis
berkondisi baik	stabil	galat kecil biasanya diharapkan
berkondisi baik	tidak stabil	algoritme menyalakan informasi
berkondisi buruk	stabil	hasil tetap dapat peka terhadap data
berkondisi buruk	tidak stabil	kepekaan masalah dan algoritme menumpuk

9.9 Penjumlahan dan urutan operasi

Penjumlahan titik-mengambang tidak asosiatif. Secara real,

$$10^{16} + 1 - 10^{16} = 1.$$

Namun, algoritme naif yang menjumlahkan dari kiri kehilangan angka 1 sebelum pengurangan terakhir.

```

nilai = [1e16, 1.0, -1e16]

def jumlah_naif(data):
    total = 0.0
    for angka in data:
        total += angka
    return total

(
    jumlah_naif(nilai),
    jumlah_naif([1e16, -1e16, 1.0]),
    math.fsum(nilai),
)

```

Daftar 9.8

(0.0, 1.0, 1.0)

`math.fsum` melacak bagian kecil yang hilang dengan lebih cermat dan memberi 1 untuk contoh ini. `sum` bawaan dapat memakai strategi yang lebih baik pada versi Python tertentu; karena itu, skrip juga merekam hasilnya tetapi tidak menjadikannya definisi algoritme naif. `math.fsum` umumnya lebih akurat bagi data bercampur skala, tetapi bukan janji bahwa setiap masalah penjumlahan bebas galat. Rekam metode, versi, urutan data, dan skala nilainya.

9.10 Toleransi sebagai anggaran galat

Tes kedekatan yang digunakan unit ini berbentuk

$$|\hat{y} - y_{ref}| \leq A + R|y_{ref}|,$$

dengan A anggaran absolut dan R anggaran relatif. Bagian absolut penting di dekat nol; bagian relatif mengikuti skala referensi.

Anggaran harus berasal dari masalah sebelum hasil dilihat. Sumbernya dapat mencakup ketelitian pengukuran, pendekatan model, diskretisasi, penghentian iterasi, dan pembulatan. Penjumlahan batas tiap komponen memberi batas konservatif bila tidak ada analisis yang lebih tajam.

`isclose` atau fungsi sejenis hanya memeriksa aturan yang diberikan. Ia tidak memilih toleransi, tidak memperbaiki algoritme, dan tidak membuktikan bahwa referensi benar. Simpan nilai A , R , galat teramati, dan alasan pemilihannya.

9.11 Rekaman deterministik dan batas bukti

Jalankan dari akar proyek:

```
python source/code/unit06_floating.py --output output/unit06-results.json
```

Skrip mencatat versi Python dan SciPy, profil binary64, pembatalan, perbandingan `scipy.special.exprel`, galat maju/mundur, pengondisian, penjumlahan, dan keputusan anggaran galat. Bilangan yang perlu dibedakan bit demi bit disimpan dengan `repr`, `hex`, atau teks `Decimal`. JSON memakai UTF-8, kunci terurut, inden tetap, dan akhir baris LF. Medan `core_sha256` mengikat rekaman sebelum hash ditambahkan.

Eksperimen tersebut membuktikan perilaku kasus yang benar-benar dijalankan pada lingkungan yang dicatat. Ia dapat menunjukkan contoh penyangkal terhadap klaim seperti “urutan penjumlahan tidak pernah berpengaruh”. Ia tidak membuktikan teorema tentang semua operasi binary64.

Pernyataan umum memerlukan model, misalnya: setiap operasi normal dibulatkan ke nilai representabel terdekat dengan galat relatif paling besar sekitar u , selama tidak terjadi luapan atau aliran-bawah. Dari model itu, batas galat dapat diturunkan langkah demi langkah. Tes lalu memeriksa apakah implementasi dan lingkungan mengikuti asumsi yang dicatat.

9.12 Latihan

9.12.1 Latihan 1 - ulp dan kenaikan yang hilang

Di sekitar 2^{53} , ulp bernilai 2. Jelaskan mengapa `float(2**53) + 1.0 == float(2**53)`, tetapi penambahan 2 dapat menghasilkan bilangan berikutnya.

Petunjuk

Bayangkan kisi bilangan representabel yang titik-titiknya berjarak 2.

Jawaban dan solusi

Pada skala itu, bilangan `float` berhingga yang berdekatan adalah 2^{53} , $2^{53} + 2$, $2^{53} + 4$, Nilai real $2^{53} + 1$ berada tepat di antara dua titik kisi dan dibulatkan menurut aturan ikatan ke nilai dengan bit terakhir genap, yaitu 2^{53} . Nilai $2^{53} + 2$ sendiri representabel, sehingga kenaikan 2 tidak hilang.

9.12.2 Latihan 2 - membuang pembatalan

Turunkan bentuk stabil untuk $\sqrt{x+4} - \sqrt{x}$ dan jelaskan mengapa bentuk itu lebih baik untuk x besar.

Petunjuk

Kalikan pembilang dan penyebut dengan jumlah kedua akar.

Jawaban dan solusi

$$\sqrt{x+4} - \sqrt{x} = \frac{(x+4) - x}{\sqrt{x+4} + \sqrt{x}} = \frac{4}{\sqrt{x+4} + \sqrt{x}}.$$

Bentuk langsung mengurangkan dua bilangan besar yang hampir sama sehingga angka signifikan dapat hilang. Bentuk rasional membagi 4 dengan jumlah positif dan menghindari pengurangan tersebut. Domain real yang dibahas ialah $x \geq 0$.

9.12.3 Latihan 3 - galat maju dan mundur

Untuk pendekatan $\hat{y} = 1,414$ terhadap $\sqrt{2}$, hitung pendekatan galat maju absolut dan galat mundur relatif berdasarkan persamaan $y^2 = 2$.

Petunjuk

Gunakan $\sqrt{2} \approx 1,41421356$ dan hitung $1,414^2$.

Jawaban dan solusi

Galat maju absolut kira-kira $|1,414 - 1,41421356| = 0,00021356$. Karena $1,414^2 = 1,999396$, galat mundur relatif terhadap masukan 2 ialah

$$\frac{|1,999396 - 2|}{2} = 0,000302.$$

Angka pertama membandingkan keluaran dengan akar yang diinginkan; angka kedua menanyakan perubahan relatif pada masukan agar keluaran menjadi akar eksak.

9.12.4 Latihan 4 - kondisi atau algoritme?

Klasifikasikan dua gejala berikut: (a) nilai $1/(1-x)$ berubah besar ketika x dekat 1 diganggu sedikit; (b) pengurangan dua akar yang hampir sama memberi nol, tetapi bentuk rasional

memberi nilai bukan nol yang akurat.

Petunjuk

Tanyakan apakah kepekaan sudah ada pada fungsi matematika atau muncul dari urutan operasi.

Jawaban dan solusi

Gejala (a) terutama merupakan pengondisian buruk: fungsi mempunyai titik singular pada 1 dan memang peka di dekatnya. Algoritme stabil pun tidak dapat menghilangkan kepekaan data itu. Gejala (b) terutama merupakan ketidakstabilan evaluasi langsung: masalah yang sama dapat dihitung jauh lebih akurat setelah reformulasi aljabar. Dalam pekerjaan nyata, keduanya dapat terjadi bersamaan.

9.12.5 Latihan 5 - jumlah, toleransi, dan batas klaim

Untuk data `[1e16, 1.0, -1e16]`, target matematisnya 1. Bandingkan hasil `jumlah_naif` di atas dan `math.fsum`. Dengan anggaran absolut 0,5 dan anggaran relatif 10^{-12} , tentukan mana yang lolos. Apakah satu keberhasilan membuktikan `math.fsum` selalu eksak?

Petunjuk

Gunakan batas $A + R|y_{ref}|$ dengan $y_{ref} = 1$.

Jawaban dan solusi

`jumlah_naif` memberi 0 karena angka 1 hilang ketika ditambahkan ke 10^{16} . `math.fsum` memberi 1 pada kasus ini. Batas yang diizinkan adalah $0,5 + 10^{-12}$; galat algoritme naif adalah 1 sehingga gagal, sedangkan galat `math.fsum` adalah 0 sehingga lolos. Keberhasilan ini hanya membuktikan hasil kasus tersebut pada lingkungan yang diuji; ia tidak membuktikan bahwa `math.fsum` selalu eksak untuk setiap daftar.

9.12.6 Latihan SciPy - stabilitas `exprel`

Jalankan `scipy_exprel_report(1e-8)` dari kode pendamping. Catat domain matematis dan domain lab, angka kondisi relatif, galat maju relatif kedua implementasi, serta estimasi galat mundur relatifnya. Tentukan apakah hasil naif yang buruk terutama menunjukkan masalah berkondisi buruk atau algoritme yang tidak stabil. Ulangi perbandingan nilai pada `x=1e-16` dan batasi klaim Anda pada bukti yang benar-benar dijalankan.

Petunjuk

Jika angka kondisi jauh di bawah 1 tetapi galat maju naif besar, masalahnya bukan penguatan gangguan masukan. Periksa juga apakah `math.exp(1e-16) == 1.0`.

Cek yang dapat dijalankan

Dari akar proyek, jalankan:

```
python tests/test_unit06.py
Unit06FloatingTests.test_scipy_special_exprel_is_stable_near_zero -v
```

Cek mengikat domain lab, versi/fungsi SciPy, galat maju, estimasi galat mundur, pengondisian, pembatalan ekstrem, dan batas bukti. Tes regresi terpisah juga menjalankan subnormal binary64 terkecil dan memastikan oracle masih menangkap koreksi pertama $x/2$, bukan membulatkannya diam-diam menjadi nol.

 Jawaban dan solusi

Daftar 9.9

```
import runpy
from pprint import pprint

unit06 = runpy.run_path("source/code/unit06_floating.py")
report = unit06["scipy_exprel_report"](1e-8)
ringkasan = {
    "domain": report["mathematical_domain"],
    "lab": report["laboratory_domain"],
    "condition": report["conditioning"]["relative_condition_number"],
    "forward": report["forward_error"],
    "backward": report["backward_error"],
    "extreme": report["extreme_cancellation"],
    "proof": report["evidence_boundary"],
}
pprint(ringkasan, sort_dicts=True, width=58)

{'backward': {'is_exact_inverse_solution': False,
              'method': 'aproksimasi orde pertama '
              'abs(delta_x) ~= '
              'abs(delta_y)/abs(E'(x))",
              'naive_absolute_estimate': '2.21549418274780203244E-8',
              'naive_relative_estimate': '2.21549418274780198609E+0',
              'scipy_absolute_estimate': '9.41080427077268199700E-17',
              'scipy_relative_estimate': '9.41080427077268180010E-9'},
 'condition': '5.00000000833333343795E-9',
 'domain': 'semua x real',
 'extreme': {'math_exp_x_equals_one': True,
             'naive': '0.0',
             'scipy_special_exprel': '1.0',
             'x': '1e-16'},
 'forward': {'naive_absolute': '1.10774709875888165308E-8',
             'naive_relative': '1.10774709322014616851E-8',
             'scipy_absolute': '4.70540216675568868538E-17',
             'scipy_is_more_accurate': True,
             'scipy_relative': '4.70540214322867789081E-17'},
 'lab': '0 < |x| <= 1e-4 untuk masukan binary64',
 'proof': {'claim': 'Pelaksanaan ini menunjukkan '
                   'pembatalan dan perbaikan pada nilai '
                   'yang dicatat; klaim umum memerlukan '
                   'analisis algoritme dan model '
                   'pembulatan.',
            'executed_case_only': True,
            'proves_stability_for_all_real_inputs': False}}
```

Domain matematisnya semua bilangan real dengan $E(0) = 1$; lab dibatasi pada $0 < |x| \leq 10^{-4}$ bagi masukan binary64. Pada $x = 10^{-8}$, angka kondisi sekitar 5×10^{-9} , tetapi galat maju relatif naif lebih besar dari 10^{-9} dan estimasi galat mundur relatifnya lebih besar dari 1. Sebaliknya, galat maju relatif SciPy di bawah 10^{-15} dan estimasi mundurnya di bawah 10^{-6} . Karena masalahnya berkondisi baik dekat nol, selisih tersebut menunjukkan pembatalan pada algoritme naif.

Pada $x = 10^{-16}$, `math.exp(x)` membulat menjadi 1 sehingga formula naif memberi 0, sedangkan `scipy.special.exprel` memberi 1 pada run ini. Tes tersebut tidak membuktikan perilaku semua masukan atau backend; ia hanya memeriksa kasus dan lingkungan yang dicatat.

9.13 Ringkasan

- Bilangan `float` membentuk kisi biner berhingga; `ulp` berubah mengikuti skala.
- Epsilon, *unit roundoff*, rentang normal, dan rentang subnormal menjawab pertanyaan yang berbeda.
- Pembatalan dapat dihindari melalui reformulasi aljabar sebelum evaluasi.
- `scipy.special.exprel` memberi contoh fungsi khusus yang dapat dijalankan dan menghindari pembatalan naif dekat nol pada domain lab yang dinyatakan.
- Galat maju mengukur jawaban; galat mundur mengukur perubahan masalah yang membuat jawaban itu eksak.
- Pengondisian milik masalah, sedangkan stabilitas milik algoritme.
- Metode penjumlahan dan urutan operasi memengaruhi galat.
- Toleransi merupakan anggaran yang dijelaskan sebelumnya, bukan angka yang disetel agar tes lulus.
- Eksperimen memberi bukti kerja terbatas; klaim umum memerlukan model pembulatan dan argumen matematis.

10 Rancangan Eksperimen yang Dapat Diperiksa

10.1 Tujuan belajar

Setelah menyelesaikan unit ini, pembaca dapat:

- mengubah pertanyaan umum menjadi hipotesis dan ukuran hasil yang dapat diperiksa;
- membedakan variabel yang diubah, hasil yang diukur, dan kondisi yang dikendalikan;
- menetapkan kisi parameter, banyak replikasi, dan aturan berhenti sebelum melihat hasil;
- menjelaskan apa yang dapat dan tidak dapat diulang oleh benih acak semu;
- memisahkan data pengembangan dari data uji yang ditahan (*holdout*);
- melaporkan ukuran efek dan hasil negatif tanpa mengubah sasaran sesudahnya; dan
- membatasi hasil eksperimen agar tidak diperlakukan sebagai bukti umum.

Prasyarat lokal: Unit 1-4, persentase, rata-rata, perbandingan pecahan, dan fungsi bertahap sederhana pada tingkat A30. Tidak diperlukan kalkulus atau statistika inferensial.

10.2 Pertanyaan dahulu, hasil kemudian

Eksperimen Unit 7 mempelajari aturan keputusan sederhana. Setiap kasus mempunyai skor bulat dari 0 sampai 9. Untuk ambang k , program memprediksi hasil positif jika

$$\text{skor} \geq k.$$

Kita memilih satu ambang dari kisi $\{3, 4, 5, 6, 7\}$. Pertanyaan yang dibekukan sebelum menjalankan eksperimen ialah:

Apakah ambang yang dipilih pada data pengembangan meningkatkan akurasi pada data *holdout* sedikitnya 0,10 dibandingkan ambang referensi 6?

Angka 0,10 berarti sepuluh poin persentase, bukan “sepuluh persen dari akurasi lama”. Hipotesis awal menjawab “ya”. Eksperimen kemudian boleh mendukung atau tidak mendukung jawaban itu; hipotesis bukan janji tentang hasil.

Pertanyaan ini menyatakan tiga hal yang sering hilang:

1. aturan pembandingan, yaitu ambang 6;
2. ukuran hasil, yaitu proporsi prediksi benar; dan
3. batas efek yang dianggap bermakna, yaitu 0,10.

Tanpa ketiganya, mudah memilih cerita setelah melihat angka.

10.3 Variabel, kontrol, dan perancu

Dalam eksperimen ini:

Peran	Isi
variabel yang diubah	ambang keputusan k
hasil yang diukur	akurasi, yaitu banyak prediksi benar dibagi banyak kasus
kondisi yang dikendalikan	pembangkit kasus, ukuran sampel, replikasi, dan jadwal benih
faktor acak	skor dan hasil buatan pada setiap kasus

Sebuah **perancu** adalah faktor lain yang berubah bersama variabel utama sehingga penyebab perbedaan menjadi kabur. Jika ambang 3 diuji pada kasus yang mudah sedangkan ambang 7 diuji pada kasus yang sulit, selisih akurasi dapat berasal dari kasus, bukan ambang.

Skrip mencegah perancu itu dengan rancangan berpasangan: dalam satu replikasi, semua ambang dinilai pada kasus yang persis sama. Hanya ambangnya yang berubah. Rancangan ini tidak menghapus semua kemungkinan kesalahan; aturan pembangkit kasus sendiri masih dapat buruk. Namun, ia mengisolasi perbandingan yang sedang dilakukan.

10.4 Kisi parameter dan aturan pemilihan

Kisi parameter ditentukan sebagai

$$K = (3, 4, 5, 6, 7).$$

Program tidak menambahkan ambang baru setelah melihat hasil. Setiap kandidat menyelesaikan 20 replikasi pengembangan, masing-masing berisi 200 kasus. Ambang dengan jumlah prediksi benar terbesar dipilih. Jika dua ambang seri, ambang yang lebih kecil menang. Aturan seri itu tampak sepele, tetapi tanpa aturan tetap dua pelaksana dapat memilih hasil berbeda.

Pencarian kisi hanya membandingkan lima kandidat tersebut. Jika ambang 5 menang, kesimpulan yang sah adalah “5 terbaik pada kisi dan data pengembangan ini menurut ukuran yang ditetapkan”, bukan “5 adalah ambang terbaik yang mungkin untuk setiap populasi”.

Kisi yang sangat rapat juga bukan hadiah gratis. Makin banyak pilihan yang dicoba pada data sama, makin besar peluang menemukan pilihan yang cocok dengan kebetulan khusus data pengembangan. Itulah salah satu alasan memakai *holdout*.

10.5 Benih dan nondeterminisme

`random.Random(seed)` menghasilkan urutan acak **semu**. Benih yang sama pada lingkungan yang sama menghasilkan urutan yang sama, sehingga kegagalan dapat dijalankan ulang. Unit ini memakai benih dasar 1729 untuk pengembangan dan 271828 untuk *holdout*. Replikasi ke-*i* memakai `benih_dasar + i`.

Benih tidak mengubah eksperimen menjadi bukti, dan tidak dapat menangkap semua nondeterminisme nyata. Contohnya:

- derau sensor fisik;
- perubahan suhu atau tegangan;
- urutan proses paralel;
- paket data yang tiba dari jaringan; dan
- tindakan manusia selama pengukuran.

Untuk sumber semacam itu, catat waktu, perangkat, kalibrasi, urutan perlakuan, lingkungan, dan data mentah sejauh relevan. Replikasi independen tetap diperlukan. Menulis angka `seed=42` tidak mengulang sebuah laboratorium.

10.6 Replikasi dan aturan berhenti

Satu pengulangan dapat kebetulan menguntungkan satu ambang. Replikasi menunjukkan apakah hasil bertahan pada beberapa sampel. Rencana unit menetapkan:

- pengembangan: tepat $20 \times 200 = 4.000$ kasus; dan
- *holdout*: tepat $12 \times 250 = 3.000$ kasus.

Kata “tepat” adalah bagian dari metode. Program tidak berhenti ketika hasil pertama kali tampak baik. Jika kita terus melihat hasil dan berhenti pada replikasi yang menyenangkan, aturan pengumpulan data bergantung pada hasil; ringkasan akhir lalu lebih optimistis daripada rencana awal.

Aturan berhenti dapat berupa jumlah kasus tetap, batas waktu tetap, atau kriteria lain. Yang penting, aturan itu ditulis sebelum hasil diperiksa dan dilaporkan bersama penyimpangan apa pun.

10.7 Pengembangan dan data holdout

Data pengembangan dipakai untuk membandingkan kisi dan memilih ambang. Data *holdout* dibangkitkan dengan jadwal benih terpisah, tidak ikut menentukan pilihan, dan diperiksa sekali setelah ambang dibekukan.

Analogi sederhananya adalah latihan dan ujian. Jika jawaban ujian dipakai untuk mengubah cara menjawab lalu nilai ujian yang sama dilaporkan sebagai penilaian akhir, ujian itu telah menjadi latihan. Demikian juga, jika hasil *holdout* dilihat berulang kali untuk mengubah kisi, fitur, atau aturan, ia bukan lagi penilaian yang tidak tersentuh. Kita memerlukan *holdout* baru atau harus jujur menyebut tahap tersebut pengembangan tambahan.

Pemisahan ini tidak membuat hasil berlaku bagi semua dunia nyata. Data *holdout* masih berasal dari pembangkit kasus tertentu. Ia hanya mengurangi satu jenis penyesuaian berlebihan terhadap data pengembangan.

10.8 Ukuran efek, bukan kesan gambar

Pada rekaman acuan, ambang 5 dipilih. Di *holdout*, ambang 5 benar pada 2.355 dari 3.000 kasus, sedangkan ambang referensi 6 benar pada 2.204 kasus. Ukuran efeknya ialah

$$\frac{2355}{3000} - \frac{2204}{3000} = \frac{151}{3000} \approx 0,0503.$$

Jadi peningkatannya sekitar 5,03 poin persentase. Sebuah grafik dengan sumbu terpotong dapat membuat selisih itu tampak besar, tetapi ukuran efek numeriknya tidak berubah. Karena batas yang ditetapkan adalah 10 poin persentase, hipotesis awal **tidak didukung**.

“Tidak didukung” tidak sama dengan “kedua aturan pasti identik”. Hasil menunjukkan efek teramati positif tetapi lebih kecil daripada sasaran yang dibekukan. Unit ini tidak menghitung interval kepercayaan atau nilai-p; jangan menyisipkan makna statistik yang tidak dihitung.

10.9 Hasil negatif tetap merupakan hasil

Setelah melihat 0,0503, kita tidak boleh menurunkan ambang efek dari 0,10 ke 0,05 lalu mengaku rencana awal berhasil. Kita juga tidak boleh menghapus ambang yang kalah atau hanya menampilkan replikasi yang menguntungkan.

Hasil negatif berguna karena:

- ia membatasi klaim yang masuk akal;
- ia mencegah orang lain mengulangi jalan buntu tanpa mengetahui hasilnya;
- ia dapat mengungkapkan bahwa sasaran praktis terlalu besar; dan
- ia membantu merancang eksperimen berikutnya tanpa mengubah sejarah eksperimen ini.

Eksperimen baru boleh mempunyai hipotesis baru. Yang tidak jujur adalah menulis ulang rencana lama seolah-olah sasaran baru telah ditetapkan sejak awal.

10.10 Rekaman kanonis Unit 7

Jalankan dari akar proyek:

```
python source/code/unit07_experiment_design.py --output  
output/unit07-results.json
```

JSON menyimpan pertanyaan, hipotesis, kisi, variabel, kontrol, aturan seleksi, aturan berhenti, jadwal benih, setiap ringkasan replikasi, hasil pengembangan, hasil *holdout*, ukuran efek eksak, hasil negatif, dan batas bukti. Pecahan disimpan sebagai pembilang dan penyebut serta desimal empat tempat. Kunci diurutkan, inden dan akhir baris ditetapkan, lalu `core_sha256` mengikat rekaman sebelum medan hash ditambahkan.

Pada lingkungan Python yang sama, perintah yang sama menghasilkan byte sama. Versi Python ikut dicatat karena rincian pembangkit acak semu merupakan bagian dari lingkungan yang perlu dibekukan.

10.11 Eksperimen dan bukti

Eksperimen ini membuktikan hal yang sangat terbatas tentang eksekusi: untuk benih dan kode yang direkam, jumlah keberhasilan dapat dihitung ulang. Ia memberi bukti empiris tentang lima ambang pada sampel buatan berhingga.

Ia tidak membuktikan bahwa ambang 5 terbaik:

- di luar kisi 3 sampai 7;
- untuk setiap urutan data;
- untuk pembangkit kasus lain; atau
- untuk fenomena nyata yang tidak dimodelkan oleh generator.

Bukti umum memerlukan klaim matematis yang tepat. Misalnya, jika distribusi skor dan peluang hasil didefinisikan secara eksak, seseorang dapat menghitung akurasi harapan setiap ambang dengan menjumlahkan peluang pada sepuluh skor. Perhitungan itu merupakan argumen tentang model yang didefinisikan, bukan jaminan bahwa model mewakili dunia nyata.

10.12 Latihan

10.12.1 Latihan 1 - variabel dan perancu

Sebuah eksperimen menguji ambang 4 pada 100 kasus pertama dan ambang 6 pada 100 kasus berikutnya. Sebutkan variabel utama, hasil yang diukur, dan perancu yang mungkin. Berikan satu perbaikan.

Petunjuk

Tanyakan apakah kedua ambang menghadapi kasus yang sama sulitnya.

Jawaban dan solusi

Variabel utamanya adalah ambang, dan hasilnya dapat berupa proporsi prediksi benar. Kelompok kasus menjadi perancu karena urutan pertama dan kedua mungkin mempunyai tingkat kesulitan berbeda. Perbaikan paling langsung ialah menilai kedua ambang pada 200 kasus yang sama. Jika hal itu tidak mungkin, acak urutan penugasan dan lakukan replikasi dengan aturan yang dibekukan.

10.12.2 Latihan 2 - benih bukan mesin waktu

Jelaskan mengapa menyimpan benih cukup untuk mengulang simulasi Unit 7 tetapi tidak cukup untuk mengulang pengukuran suhu di laboratorium.

Petunjuk

Bedakan urutan acak semu dari gangguan fisik dan keadaan alat.

Jawaban dan solusi

Simulasi menghitung seluruh urutan dari algoritme dan benih, sehingga pasangan yang sama dapat membangkitkan kasus yang sama pada lingkungan yang dibekukan. Suhu laboratorium dipengaruhi sensor, kalibrasi, waktu, lokasi, dan gangguan fisik yang tidak berasal dari `random.Random`. Benih harus dilengkapi data mentah, keadaan alat, waktu, prosedur, dan replikasi independen.

10.12.3 Latihan 3 - pemilihan dan holdout

Pada data pengembangan, ambang 4, 5, dan 6 memperoleh 760, 810, dan 790 jawaban benar dari 1.000 kasus. Ambang mana dipilih? Mengapa Anda tidak boleh mengganti pilihan setelah melihat bahwa ambang 6 sedikit lebih baik pada *holdout*?

Petunjuk

Gunakan aturan seleksi pada pengembangan, lalu pertahankan fungsi *holdout* sebagai penilaian yang belum disentuh.

Jawaban dan solusi

Ambang 5 dipilih karena 810 adalah jumlah benar terbesar. Jika pilihan diubah menjadi 6 setelah membaca *holdout*, data itu ikut melakukan pemilihan dan tidak lagi menjadi penilaian terpisah. Ambang 6 dapat menjadi hipotesis untuk eksperimen baru dengan *holdout* baru, tetapi hasil eksperimen lama harus tetap dilaporkan menurut rencana semula.

10.12.4 Latihan 4 - ukuran efek dan aturan berhenti

Hitung peningkatan ambang 5 atas ambang 6 dari 2.355 dan 2.204 jawaban benar dalam 3.000 kasus. Apakah sasaran 0,10 tercapai? Bolehkah eksperimen berhenti lebih awal pada replikasi yang menunjukkan 0,11?

Petunjuk

Kurangi jumlah benar lalu bagi dengan jumlah kasus yang sama.

💡 Jawaban dan solusi

Peningkatannya $(2355 - 2204)/3000 = 151/3000 \approx 0,0503$, yaitu sekitar 5,03 poin persentase. Sasaran 10 poin tidak tercapai. Eksperimen tidak boleh berhenti pada satu replikasi 0,11 karena aturan awal mewajibkan seluruh replikasi; berhenti berdasarkan hasil akan mengubah rancangan dan membuat ringkasan terlalu menguntungkan.

10.12.5 Latihan 5 - hasil negatif dan batas bukti

Tuliskan kesimpulan jujur untuk eksperimen Unit 7 dan satu klaim yang tidak boleh dibuat. Lalu sebutkan apa yang diperlukan untuk membuktikan klaim umum tentang ambang terbaik pada model yang didefinisikan.

i Petunjuk

Pisahkan efek pada *holdout*, sasaran 0,10, dan seluruh ruang kemungkinan.

💡 Jawaban dan solusi

Kesimpulan jujur: ambang 5 terpilih pada kisi, lalu mengungguli ambang 6 sekitar 5,03 poin persentase pada *holdout*, tetapi tidak mencapai efek minimum 10 poin yang ditetapkan. Klaim terlarang: ambang 5 telah terbukti terbaik untuk semua data atau dunia nyata. Untuk klaim umum pada model eksak, tentukan distribusi skor dan peluang hasil, hitung akurasi harapan setiap kandidat (atau seluruh domain ambang), lalu bandingkan dengan argumen yang mencakup semuanya.

10.13 Ringkasan

- Pertanyaan, hipotesis, pembandingan, ukuran hasil, dan efek minimum ditetapkan sebelum melihat hasil.
- Variabel lain dikendalikan agar perbedaan tidak tercampur dengan kesulitan sampel atau perubahan prosedur.
- Kisi, aturan seri, replikasi, jadwal benih, dan aturan berhenti merupakan bagian dari rencana.
- Benih mengulang acak semu, bukan nondeterminisme fisik yang tidak direkam.
- Data pengembangan memilih parameter; *holdout* menilai pilihan sekali.
- Ukuran efek numerik lebih penting daripada kesan visual, dan hasil negatif harus tetap dilaporkan.
- Eksperimen berhingga memberi bukti empiris terbatas; bukti umum memerlukan argumen yang mencakup domain klaim.

11 Pengujian dan Validasi Komputasi

11.1 Tujuan belajar

Setelah menyelesaikan unit ini, pembaca dapat:

- memakai *assertion* untuk menyatakan hasil atau invarian yang dapat diperiksa;
- membedakan pengujian berbasis contoh dari pengujian berbasis sifat;
- membedakan tes unit, integrasi, dan regresi;
- memilih oracle tanpa menyalin algoritme yang sedang diuji;
- memakai kesamaan eksak dan toleransi numerik pada tempat yang tepat;
- menguji nilai batas, prasyarat, dan perilaku galat;
- merumuskan hubungan metamorfik dan invarian;
- memvalidasi implementasi terhadap hasil yang diturunkan secara independen; dan
- membatasi bukti pengujian agar tidak disalahartikan sebagai bukti matematis umum.

Prasyarat lokal: Unit 1-5, fungsi, jumlah berhingga, persamaan kuadrat, dan pengertian domain. Unit ini hanya memerlukan pustaka standar Python.

11.2 *Assertion* dan kontrak

Assertion menyatakan kondisi yang diharapkan benar pada satu titik pelaksanaan. Dalam tes `unittest`, bentuk seperti `self.assertEqual(actual, expected)` mencatat kegagalan sebagai hasil tes. Pernyataan `assert` bawaan Python lebih cocok untuk invarian internal yang jika gagal menandakan kesalahan pemrograman.

Jangan memakai `assert n >= 0` sebagai satu-satunya validasi masukan publik. Python dapat menghapus pernyataan `assert` ketika dijalankan dengan optimasi. Prasyarat pengguna harus diperiksa secara eksplisit dan menghasilkan jenis galat yang terdokumentasi.

11.3 Contoh dan sifat

Tes contoh memasang masukan tertentu dengan keluaran yang diketahui:

Daftar 11.1

```
def triangular_iterative(n):
    if isinstance(n, bool) or not isinstance(n, int):
        raise TypeError("n harus berupa bilangan bulat")
    if n < 0:
        raise ValueError("n harus taknegatif")

    total = 0
    for k in range(1, n + 1):
        previous = total
        total += k
        assert total >= previous
    return total
```

n	$1 + 2 + \dots + n$
0	0
1	1
2	3
10	55
100	5.050

Contoh mudah dibaca dan sangat baik bagi kasus penting atau kesalahan yang pernah ditemukan. Namun, beberapa contoh tidak menjelaskan struktur seluruh domain.

Tes sifat memeriksa hubungan yang harus tetap benar pada banyak masukan. Jika $T(n) = 1 + 2 + \dots + n$, maka

$$T(n+1) - T(n) = n + 1$$

dan

$$T(2n) = 2T(n) + n^2.$$

Hubungan kedua tidak menentukan satu keluaran dari nol; ia menghubungkan beberapa pemanggilan fungsi. Hubungan semacam ini disebut **metamorfik** dan berguna ketika keluaran benar bagi setiap masukan sulit dihitung langsung.

Tes sifat pada seribu nilai tetap masih merupakan pemeriksaan berhingga. Ia lebih luas daripada lima contoh, tetapi bukan bukti universal.

11.4 Tes unit, integrasi, dan regresi

Ketiga label ini menjawab pertanyaan berbeda.

Tingkat	Fokus	Contoh Unit 8
unit	satu fungsi atau kontrak kecil	<code>triangular_iterative(10)</code> <code>== 55</code>
integrasi	beberapa bagian bekerja bersama	oracle, sifat, dan penulis JSON menghasilkan laporan sah
regresi	kesalahan tertentu tidak kembali	nilai batas <code>n=0</code> tetap menghasilkan 0

Tes regresi biasanya ditambahkan setelah sebuah kesalahan ditemukan dan diperbaiki. Ia harus menyimpan masukan minimum yang memicu kesalahan serta hasil yang benar. Menamai setiap tes besar sebagai “regresi” mengaburkan riwayat kegagalan yang sebenarnya hendak dijaga.

11.5 Oracle dan nilai referensi

Oracle adalah sumber keputusan tentang keluaran yang benar. Pilihannya dapat berupa:

- nilai kecil yang dihitung dan ditinjau sebelumnya;
- definisi atau identitas matematis;
- implementasi referensi yang lebih sederhana;
- data acuan dengan asal-usul yang jelas; atau
- invarian dan hubungan metamorfik.

Oracle yang mengulang baris kode produksi dengan nama berbeda tidak mandiri. Kesalahan yang sama dapat hidup di kedua tempat dan semua tes tetap lulus.

Implementasi yang diuji pada unit ini menjumlahkan 1 sampai n dengan perulangan. Oracle utamanya diturunkan melalui pasangan. Tulis jumlah maju dan mundur:

$$T(n) = 1 + 2 + \cdots + n,$$

$$T(n) = n + (n - 1) + \cdots + 1.$$

Menjumlahkan kedua baris memberi n pasangan, masing-masing bernilai $n + 1$. Jadi

$$2T(n) = n(n+1), \quad T(n) = \frac{n(n+1)}{2}.$$

Fungsi referensi memakai rumus ini, bukan perulangan kedua. Dengan demikian, kesepakatan keduanya lebih informatif daripada membandingkan dua salinan loop.

11.6 Eksak dan bertoleransi

Jika hasilnya bilangan bulat atau rasional eksak, gunakan kesamaan eksak. `triangular_iterative(10)` harus sama dengan 55, bukan sekadar dekat. Toleransi pada tes semacam itu dapat menyembunyikan kesalahan satu satuan.

Untuk pendekatan titik-mengambang, tentukan besaran yang diperiksa dan alasan toleransinya. Iterasi Newton untuk $x^2 = 2$ menghasilkan pendekatan akar dua. Tes Unit 8 memeriksa residual

$$|x^2 - 2| \leq 10^{-12}$$

setelah lima iterasi dari $x_0 = 1$.

```
from math import isclose

x = 1.0
for _ in range(5):
    x = 0.5 * (x + 2.0 / x)

isclose(x*x, 2.0, rel_tol=0.0, abs_tol=1e-12)
```

Daftar 11.2

True

Toleransi diterapkan pada residual yang mempunyai makna jelas. Jangan memilih toleransi setelah melihat kegagalan hanya agar tes berubah menjadi hijau.

11.7 Nilai batas dan domain

Domain fungsi jumlah pada unit ini adalah bilangan bulat taknegatif. Tes harus memeriksa sekurangnya:

- batas `n=0`;
- nilai pertama `n=1`;
- nilai biasa seperti `n=10`;
- nilai negatif yang harus ditolak;
- bilangan bukan bulat yang harus ditolak; dan
- `True`, karena `bool` merupakan subkelas `int` di Python tetapi bukan masukan yang dimaksud kontrak matematika ini.

Tes galat perlu memeriksa jenis kegagalan, bukan hanya bahwa “sesuatu gagal”. `ValueError` bagi bilangan bulat negatif dan `TypeError` bagi 2.5 menyatakan dua pelanggaran kontrak yang berbeda.

11.8 Invarian dan pengujian metamorfik

Invarian loop sederhana adalah jumlah tidak berkurang setelah bilangan positif berikutnya ditambahkan. Hubungan metamorfik membandingkan pemanggilan berbeda:

Daftar 11.3

```
for n in range(201):
    value = triangular_iterative(n)
    assert triangular_iterative(n + 1) - value == n + 1
    assert triangular_iterative(2*n) == 2*value + n*n
```

Hubungan metamorfik tidak memerlukan tabel 201 jawaban, tetapi tetap memerlukan derivasi. Untuk hubungan penggandaan,

$$T(2n) - 2T(n) = n(2n + 1) - n(n + 1) = n^2.$$

Jika hubungan salah dirumuskan, tes dapat menolak implementasi benar. Kode tes harus ditinjau seperti kode produksi.

11.9 Validasi independen

Validasi bertanya apakah program menjawab persoalan yang dimaksud, bukan hanya apakah ia konsisten dengan dirinya sendiri. Pada unit ini:

1. spesifikasi mendefinisikan $T(n)$ sebagai jumlah bilangan 1 sampai n ;
2. implementasi memakai penjumlahan berulang;
3. oracle memakai rumus pasangan yang diturunkan terpisah;
4. tabel nilai kecil memberi jangkar yang mudah diperiksa; dan
5. sifat inkremen serta penggandaan memberi pemeriksaan silang tambahan.

Lapisan yang berbeda mengurangi risiko satu kesalahan tunggal lolos tanpa terlihat. Mereka tidak membuat validasi sempurna: spesifikasi dapat salah, oracle dapat salah diturunkan, atau domain uji dapat melewati cacat.

11.10 Laporan deterministik

Jalankan dari akar proyek:

```
python source/code/unit08_validation.py
```

Laporan memuat nilai referensi, perbandingan dengan rumus pasangan, pelanggaran sifat, pemeriksaan batas dan domain, residual numerik, tingkat tes, serta batas kesimpulan. JSON memakai UTF-8, kunci terurut, inden tetap, dan akhir baris LF. `core_sha256` mengikat isi inti sebelum medan hash ditambahkan. Tidak ada waktu saat ini, jaringan, atau bilangan acak dalam jalur ini.

Dua keluaran byte-identik menunjukkan bahwa dua pelaksanaan merekam hasil yang sama pada kondisi yang sama. Hal itu tidak menunjukkan bahwa oracle atau spesifikasi pasti benar.

11.11 Cakupan, bukti kerja, dan bukti

Cakupan mempunyai beberapa arti. Cakupan baris mengatakan bagian kode mana yang sempat dijalankan. Cakupan cabang mengatakan pilihan kendali mana yang diambil. Cakupan domain mengatakan masukan matematika mana yang diperiksa. Cakupan 100% pada baris tidak berarti semua nilai atau semua sifat telah diuji.

Jika program cocok dengan rumus pasangan untuk $0 \leq n \leq 1000$, kesimpulan yang sah ialah tidak ditemukan ketidakcocokan pada domain berhingga itu dalam lingkungan yang dijalankan. Bukti rumus pasangan berlaku untuk seluruh bilangan bulat taknegatif karena memilih n

secara umum dan memasangkan suku. Bukti bahwa loop mengimplementasikan jumlah dapat memakai invarian loop. Tes mendukung bahwa implementasi konkret sesuai dengan argumen pada kasus yang diperiksa; tes sendiri tidak menggantikan kedua bukti tersebut.

11.12 Latihan

11.12.1 Latihan 1 - memilih assertion

Untuk setiap klaim berikut, pilih kesamaan eksak atau perbandingan bertoleransi dan jelaskan: (a) jumlah $1 + \dots + 100$, (b) pendekatan numerik $\sqrt{2}$, dan (c) panjang daftar keluaran.

Petunjuk

Tanyakan apakah representasi dan hasil yang ditetapkan bersifat eksak.

Jawaban dan solusi

Gunakan kesamaan eksak untuk (a), yaitu 5050, karena operasi bilangan bulat Python pada rentang ini eksak. Gunakan perbandingan bertoleransi untuk (b), dengan toleransi yang diturunkan dari residual atau kebutuhan masalah. Gunakan kesamaan eksak untuk (c), karena panjang daftar adalah bilangan bulat. Memakai toleransi untuk panjang dapat menerima daftar dengan jumlah unsur salah.

11.12.2 Latihan 2 - membangun oracle

Anda menguji fungsi yang menghitung $1 + 3 + 5 + \dots + (2n - 1)$ dengan loop. Turunkan oracle yang tidak memakai loop yang sama dan sebutkan domainnya.

Petunjuk

Gambar persegi yang bertambah satu bingkai berbentuk L pada setiap langkah.

Jawaban dan solusi

Jumlah n bilangan ganjil pertama adalah n^2 . Persegi berukuran $(n - 1) \times (n - 1)$ memperoleh $2n - 1$ titik baru untuk menjadi persegi $n \times n$. Jadi oracle dapat menghitung $n*n$ untuk bilangan bulat taknegatif n . Implementasi loop kemudian dibandingkan dengan rumus tersebut, bukan dengan loop salinan.

11.12.3 Latihan 3 - hubungan metamorfik

Untuk fungsi $S(n) = n^2$, tuliskan dua hubungan metamorfik yang dapat diuji tanpa tabel nilai lengkap. Jelaskan mengapa hubungan itu sah.

Petunjuk

Bandingkan $S(n + 1)$ dengan $S(n)$, lalu coba masukan $-n$.

Jawaban dan solusi

Dua hubungan ialah $S(n + 1) - S(n) = 2n + 1$ dan $S(-n) = S(n)$. Yang pertama mengikuti $(n + 1)^2 - n^2 = 2n + 1$; yang kedua mengikuti $(-n)^2 = n^2$. Keduanya dapat menemukan kelas kesalahan berbeda, tetapi tetap perlu contoh batas dan pemeriksaan langsung terhadap beberapa nilai acuan.

11.12.4 Latihan 4 - tes regresi batas

Sebuah fungsi rata-rata pernah membagi dengan nol ketika menerima daftar kosong, padahal kontraknya harus menolak masukan itu dengan `ValueError`. Tuliskan inti tes regresinya dan jelaskan mengapa tes tersebut harus tetap ada.

Petunjuk

Gunakan `with self.assertRaises(ValueError):`.

Jawaban dan solusi

Intinya ialah `with self.assertRaises(ValueError): mean([])`. Tes harus tetap ada karena menyimpan masukan minimum yang pernah menembus kontrak dan perilaku yang sekarang diwajibkan. Jika refaktor kemudian mengembalikan `NaN` atau membocorkan `ZeroDivisionError`, tes menunjukkan bahwa perilaku publik berubah.

11.12.5 Latihan 5 - batas bukti pengujian

Semua tes lulus, semua baris kode tercakup, dan sejuta masukan acak tidak menemukan kegagalan. Nyatakan kesimpulan terkuat yang sah dan dua alasan mengapa hal itu belum menjadi bukti umum.

Petunjuk

Bedakan cakupan pelaksanaan dari cakupan domain dan periksa oracle.

Jawaban dan solusi

Kesimpulan terkuat ialah implementasi melewati rangkaian tes, cabang, dan sampel yang dinyatakan pada lingkungan tersebut. Itu belum menjadi bukti umum karena domain tak hingga tidak habis oleh sejuta sampel dan oracle atau spesifikasi dapat memuat kesalahan bersama. Bukti umum membutuhkan argumen yang mencakup seluruh domain; validasi tambahan membutuhkan oracle yang mandiri, kasus batas, sifat, serta rekaman lingkungan.

11.13 Ringkasan

- *Assertion* yang tepat menyatakan kontrak lokal yang dapat diperiksa.
- Contoh mengikat kasus penting; sifat dan hubungan metamorfik memperluas pemeriksaan tanpa menyediakan bukti universal secara otomatis.
- Tes unit, integrasi, dan regresi melindungi lapisan risiko yang berbeda.
- Oracle harus berasal dari nilai, derivasi, data, atau implementasi yang cukup mandiri dari kode yang diuji.
- Kesamaan eksak sesuai bagi hasil eksak; toleransi harus mengikuti makna numerik dan skala.
- Nilai batas, domain salah, dan jenis galat merupakan bagian kontrak.
- Validasi silang dan laporan kanonis memperkuat bukti kerja.
- Cakupan serta tes yang lulus bukan pengganti bukti matematis umum.

12 Data, Konfigurasi, dan Provenans

12.1 Tujuan belajar

Setelah menyelesaikan unit ini, pembaca dapat:

- membedakan data mentah, data turunan, dan laporan run;
- menyatakan schema, tipe, makna kolom, dan satuan;
- memisahkan konfigurasi yang berubah dari kode transformasi;
- mencatat identitas lingkungan dan batas berkas kunci;
- memakai jalur relatif yang tidak dapat keluar dari akar proyek;
- merekam asal-usul serta status hak setiap komponen;
- mengikat artefak pada ukuran byte dan SHA-256 tanpa melebih-lebihkan fungsi hash;
- mengecualikan data privat dari artefak publik; dan
- membedakan pengulangan semantik dari pengulangan byte demi byte.

Prasyarat lokal: Unit 1-4 dan 7-8, tabel, rasio, satuan, berkas teks, dan fungsi sederhana. Jalur dasar hanya memakai pustaka standar Python.

12.2 Tiga lapisan data

Data mentah adalah rekaman yang diterima sebelum transformasi analitis unit. “Mentah” tidak berarti tanpa sejarah: sensor, formulir, atau proses ekspor mungkin sudah mengubahnya. Istilah itu hanya menyatakan titik masuk yang dibekukan bagi run ini.

Unit 9 memakai lima rekaman sintetis:

sample_id	length_cm	mass_g
S01	10,0	20,0
S02	12,0	24,6
S03	8,0	15,8
S04	14,0	28,7
S05	11,0	21,8

Data turunan dihasilkan dengan aturan

$$r = \frac{\text{massa dalam gram}}{\text{panjang dalam sentimeter}},$$

lalu dibulatkan ke tiga tempat desimal dengan aturan *round half even*. Nama kolom turunannya ialah `mass_per_length_g_per_cm`.

Manifest run bukan data pengukuran baru. Ia menjawab berkas apa yang dipakai, transformasi apa yang dijalankan, konfigurasi dan lingkungan apa yang berlaku, serta byte keluaran mana yang dihasilkan.

Jangan menimpa data mentah dengan hasil turunan. Jika sebuah transformasi perlu diperbaiki, data mentah yang terikat memungkinkan keluaran dibuat ulang dan dibandingkan.

12.3 Schema dan satuan

Schema adalah kontrak struktur. Schema mentah unit ini menetapkan:

Medan	Tipe	Satuan	Makna
<code>sample_id</code>	teks berbentuk S + dua digit	tidak ada	pengenal sintetis
<code>length_cm</code>	teks desimal positif	cm	panjang
<code>mass_g</code>	teks desimal positif	g	massa

Menyimpan desimal sebagai teks menghindari perubahan awal ke titik-mengambang. Kode menolak nilai numerik yang bukan teks, lalu mengubah teks tervalidasi itu menjadi `Decimal` ketika menghitung. Pengenal tidak mempunyai satuan; `None` lebih jujur daripada satuan kosong yang dapat berarti “lupa diisi”.

Nama `length` tanpa satuan tidak cukup. Nilai 10 dapat berarti 10 mm, 10 cm, atau 10 m, dan ketiganya menghasilkan rasio berbeda. Satuan harus mengikuti data sampai keluaran; schema turunan menetapkan `g/cm`.

Validasi memeriksa nama medan, bentuk pengenal, keunikan, bilangan berhingga, dan kepositifan. Schema yang lulus belum menjamin pengukuran benar, tetapi ia mencegah beberapa salah tafsir struktural.

12.4 Konfigurasi terpisah dari kode

Kode transformasi menyatakan **bagaimana** rasio dihitung. Konfigurasi menyatakan pilihan run yang sah berubah, misalnya:

```
{
  "derived_path": "data/derived/unit09_mass-per-length.csv",
  "ratio_decimal_places": 3,
  "raw_path": "data/raw/unit09_measurements.csv",
  "rounding": "ROUND_HALF_EVEN",
  "schema": "o002.unit09.config.v1"
}
```

Jika jumlah tempat desimal ditanam di banyak baris kode, perubahan kecil mudah menjadi tidak konsisten. Satu konfigurasi tervalidasi membuat pilihan itu dapat ditinjau dan diikat dengan hash.

Memisahkan konfigurasi tidak berarti menerima sembarang nilai. Unit ini hanya menerima 0 sampai 9 tempat desimal dan satu aturan pembulatan yang telah diuji. Konfigurasi adalah masukan yang mempunyai schema, bukan jalan belakang untuk menjalankan teks sebagai kode.

12.5 Identitas dan kunci lingkungan

Keluaran dapat berubah walaupun data dan kode tampak sama karena versi bahasa, pustaka, sistem operasi, atau perangkat keras berbeda. Manifest Unit 9 mencatat implementasi dan versi Python serta menyatakan bahwa jalur dasar hanya memakai pustaka standar.

Karena tidak ada dependensi eksternal, “kunci lingkungan” unit ini kecil: identitas Python dan konteks aritmetika `Decimal` diserialisasikan secara kanonis lalu diikat dengan SHA-256. Konteks tersebut membekukan presisi 28 digit, pembulatan *round half even*, batas eksponen, serta perangkat operasi; ia tidak mewarisi presisi atau perangkat proses pemanggil. Proyek yang memakai NumPy, SymPy, SageMath, atau pustaka lain memerlukan berkas kunci yang menyebut versi tepat beserta sumber paket. Daftar nama tanpa versi bukan kunci.

Berkas kunci juga tidak membekukan seluruh dunia. Perbedaan sistem operasi, lokal, CPU, atau pustaka sistem masih dapat relevan. Catat lapisan yang mempengaruhi pertanyaan, dan jangan menyebut lingkungan “identik” hanya karena satu nomor versi cocok.

12.6 Jalur relatif yang aman

Manifest menyimpan jalur logis seperti `data/raw/unit09_measurements.csv`, bukan `direktori-profil>Nama/Desktop/data.csv`. Jalur relatif dapat dipindahkan bersama proyek dan tidak membocorkan nama akun lokal.

Validator unit menerima jalur POSIX relatif yang kanonis. Ia menolak:

- `/etc/secret` atau `C:/secret`, karena absolut;
- `../secret` dan `data/../secret`, karena dapat keluar atau menyamarkan tujuan;
- garis miring terbalik, agar satu bentuk dipakai pada semua sistem; dan
- bentuk ganda seperti `data//raw/file.csv`.

Setiap peran artefak juga harus mempunyai jalur unik setelah normalisasi Unicode dan pelipatan kapitalisasi. Karena itu, `data/raw.csv` dan `DATA/RAW.CSV` dianggap bertabrakan agar satu manifest tetap aman pada sistem berkas yang tidak membedakan huruf besar dan kecil.

Validasi jalur adalah batas keamanan dan reproduksibilitas. Namun, ia bukan izin membaca setiap berkas yang kebetulan berada di dalam proyek. Daftar masukan yang diizinkan tetap harus dibatasi oleh rencana run.

12.7 Provenans dan hak per komponen

Provenans menjawab “dari mana komponen ini berasal dan apa yang terjadi padanya?”. Hak menjawab “atas dasar apa komponen boleh dipakai atau didistribusikan?”. Keduanya berkaitan tetapi tidak sama.

Manifest mencatat secara terpisah:

- data mentah sintetis yang ditetapkan di kode;
- data turunan beserta rumus transformasinya;
- konfigurasi;
- identitas lingkungan; dan
- kode Unit 9, yang menunjuk `LICENSE-CODE.md` dan lisensi MIT.

Data contoh tidak memuat bahan pihak ketiga. Rekaman itu tidak dipakai untuk membuat klaim lisensi menyeluruh atas setiap berkas proyek. Jika sebuah gambar CC BY-SA, kode MIT, dan data berizin khusus digabungkan, masing-masing tetap memerlukan pencipta/sumber, lisensi, perubahan, dan pembatasannya sendiri. Provenans tidak boleh dihapus hanya karena hasil turunannya berupa angka.

12.8 SHA-256 dan ukuran byte

Untuk setiap komponen, manifest menyimpan jalur logis, jenis media, peran, ukuran byte, dan SHA-256. Jika satu byte berubah, hash hampir pasti berubah. Ukuran membantu mendeteksi berkas kosong atau pemotongan yang tidak sengaja dan memberi pemeriksaan sederhana sebelum hashing.

Hash menjawab pertanyaan identitas byte. Ia tidak membuktikan bahwa:

- rumus transformasi benar;
- satuan sesuai;
- data dikumpulkan secara etis;
- lisensi yang dicatat sah; atau
- manifest memuat semua komponen yang semestinya.

Semua pertanyaan itu memerlukan pemeriksaan lain. Hash yang cocok atas berkas salah hanya memastikan bahwa dua orang memiliki berkas salah yang sama.

12.9 Privasi dan pengecualian

Reproduksibilitas bukan alasan untuk menerbitkan nama, surel, nomor telepon, lokasi rinci, atau teks bebas yang dapat mengidentifikasi seseorang. Dataset unit memakai S01 sampai S05 sebagai pengenalan sintetis dan secara eksplisit menolak medan privat.

Pada penelitian nyata, strategi dapat meliputi:

- memisahkan data terbatas dari artefak publik;
- menerbitkan schema, kode, data sintetis, dan ringkasan aman;
- mencatat cara peneliti berwenang dapat memohon akses;
- menghapus medan yang tidak diperlukan sebelum analisis; dan
- memeriksa apakah gabungan beberapa medan masih dapat mengidentifikasi orang.

Menulis hash data rahasia ke manifest publik juga dapat membocorkan informasi dalam keadaan tertentu, khususnya bila ruang kemungkinan kecil. Putuskan apa yang boleh dipublikasikan sebelum membuat manifest publik.

12.10 Manifest run yang kanonis

Jalankan dari akar proyek:

```
python source/code/unit09_provenance.py --output output/unit09-manifest.json
```

Skip tidak mengakses jaringan atau waktu saat ini. Manifest memuat rekaman mentah dan turunan, schema, konfigurasi, identitas lingkungan, ikatan artefak, rantai input-kode-konfigurasi-output, kebijakan privasi, status hak per komponen, dan batas reproduksibilitas. JSON memakai UTF-8, kunci terurut, inden tetap, dan satu akhir baris LF. `core_sha256` mengikat isi sebelum medan hash tersebut ditambahkan.

CSV mentah dan turunan pada contoh ini adalah artefak logis yang byte kanonisnya dibangun di memori lalu diikat di manifest. Dengan demikian, perintah hanya menulis JSON yang diminta, tetapi pembaca dapat membangun ulang CSV persis dari rekaman dan aturan kolom yang disimpan.

12.11 Semantik dan byte

Dua keluaran **reproduktif secara byte** jika setiap byte sama. Keduanya pasti mempunyai SHA-256 sama. Syarat ini berguna untuk memeriksa alur kerja deterministik, arsip, dan distribusi.

Dua keluaran dapat **reproduktif secara semantik** walaupun byte berbeda. Contohnya, JSON berikut menyimpan pemetaan sama:

```
{"a":1,"b":2}  
{  
  "b": 2,  
  "a": 1  
}
```

Spasi dan urutan kunci berbeda, sehingga hash berbeda, tetapi pengurai JSON menghasilkan nilai yang sama. Dalam komputasi numerik, dua algoritme juga dapat menghasilkan nilai yang setara dalam toleransi dan satuan yang ditetapkan tanpa menghasilkan byte identik.

Karena itu, laporan harus menyatakan tingkat yang diminta. Pengulangan deterministik Unit 9 menargetkan byte identik pada lingkungan terkunci. Validasi ilmiah lebih luas juga harus membandingkan schema, satuan, rumus, dan makna hasil.

12.12 Latihan

12.12.1 Latihan 1 - mentah, turunan, dan garis keturunan

Jika `mass_g` untuk S03 berubah dari 15.8 menjadi 15.9, komponen mana yang harus berubah hash dan komponen mana yang seharusnya tetap?

Petunjuk

Ikuti rantai data mentah → transformasi → data turunan; kode dan konfigurasi tidak diedit.

Jawaban dan solusi

Hash dan ukuran data mentah mungkin berubah, dan hash data turunan harus berubah karena rasio S03 dihitung dari massa baru. `core_sha256` manifest juga berubah karena ikatannya berubah. Hash kode, konfigurasi, dan identitas lingkungan tetap jika byte ketiganya tidak berubah. Provenans harus mencatat bahwa keluaran baru berasal dari revisi data mentah tersebut.

12.12.2 Latihan 2 - schema dan satuan

Sebuah tabel mengganti `length_cm` dengan kolom `length` tanpa memperbarui schema. Mengapa nilai 10.0 tidak cukup untuk menghitung rasio yang dapat ditafsirkan? Berikan perbaikan.

Petunjuk

Angka tidak membawa satuan secara otomatis.

Jawaban dan solusi

10.0 dapat berarti milimeter, sentimeter, meter, atau satuan lain. Rasio dan satuannya berubah mengikuti pilihan itu. Perbaiki dengan medan yang tidak ambigu seperti `length_cm`, nyatakan `unit: cm` dalam schema, validasi data terhadap schema, dan ubah rumus keluaran jika konversi satuan diperlukan.

12.12.3 Latihan 3 - konfigurasi dan jalur aman

Konfigurasi berisi `raw_path: "../../private.csv"` dan jumlah tempat desimal ditulis langsung pada tiga fungsi berbeda. Temukan dua masalah dan berikan perbaikannya.

Petunjuk

Periksa pelintasan direktori dan satu sumber keputusan run.

Jawaban dan solusi

Jalur `../../private.csv` dapat keluar dari akar proyek dan harus ditolak; gunakan jalur relatif kanonis dalam daftar masukan yang diizinkan. Menanam jumlah desimal di tiga tempat membuat perubahan mudah tidak seragam. Simpan satu nilai `ratio_decimal_places` dalam konfigurasi berversi, validasi rentangnya, dan buat semua pemanggilan membaca nilai yang sama.

12.12.4 Latihan 4 - hak dan privasi

Anda menerima tabel pihak ketiga berisi pengukuran, nama, dan surel. Sebutkan apa yang harus dipisahkan atau dicatat sebelum membuat artefak publik.

Petunjuk

Provenans, izin distribusi, dan kebutuhan analitis adalah tiga pertanyaan berbeda.

Jawaban dan solusi

Catat sumber, pencipta, versi, lisensi atau dasar izin, serta perubahan pada komponen tabel. Pisahkan nama dan surel dari data publik kecuali ada dasar, kebutuhan, dan persetujuan yang sah; gunakan pengenal aman atau data sintetis bila sesuai. Periksa apakah pengukuran gabungan masih dapat mengidentifikasi orang. Jangan menganggap lisensi kode proyek otomatis mencakup data pihak ketiga.

12.12.5 Latihan 5 - semantik, byte, dan hash

Dua berkas JSON mempunyai urutan kunci dan spasi berbeda, tetapi setelah diurai menghasilkan objek yang sama. Apa yang dapat dikatakan tentang reproduksibilitas semantik, byte, dan SHA-256 mereka?

Petunjuk

Bandingkan nilai hasil penguraian dengan urutan karakter asli.

💡 Jawaban dan solusi

Keduanya reproduktif secara semantik untuk pemetaan JSON tersebut karena nilai hasil penguraiannya sama. Keduanya tidak reproduktif secara byte karena urutan karakternya berbeda, sehingga SHA-256 hampir pasti berbeda. Hash hanya mengikat byte masing-masing; untuk menuntut hash sama, keduanya harus diserialisasikan dengan aturan kanonis yang sama.

12.13 Ringkasan

- Data mentah, data turunan, dan manifest run mempunyai peran berbeda dan tidak boleh saling menimpa.
- Schema memuat tipe, makna, dan satuan; validasi struktur bukan bukti mutu pengukuran.
- Konfigurasi dipisahkan dari kode, tetapi tetap mempunyai schema dan batas.
- Identitas Python dan dependensi berversi membentuk kunci lingkungan minimum.
- Jalur relatif kanonis meningkatkan portabilitas dan mencegah pelintasan direktori.
- Setiap komponen mempertahankan provenans dan status haknya sendiri.
- Ukuran serta SHA-256 mengikat byte, bukan kebenaran, etika, atau lisensi.
- Privasi dapat mewajibkan pengecualian data dari artefak publik.
- Reproduksi semantik tidak selalu memerlukan byte identik; manifest harus menyatakan tingkat yang ditargetkan.

13 Otomasi dan Alur Kerja yang Dapat Direproduksi

13.1 Tujuan belajar

Setelah menyelesaikan unit ini, pembaca dapat:

- menggambarkan eksperimen sebagai graf tugas dan dependensi;
- membedakan sumber, produk antara, dan artefak akhir;
- membangun ulang hanya ketika masukan, parameter, kode, atau lingkungan yang terikat berubah;
- mendeteksi dependensi hilang dan siklus sebelum menjalankan tugas;
- menulis artefak secara atomik agar kegagalan tidak meninggalkan hasil seolah lengkap;
- menjelaskan mengapa cap waktu saja bukan bukti kesamaan isi; dan
- mengaudit rekaman alur kerja tanpa menganggap keberhasilan proses sebagai bukti matematika.

Prasyarat lokal: Unit 1-2 dan Unit 8-9. Pembaca harus memahami fungsi, berkas, hash, konfigurasi, dan tes sebelum mengotomatisasikannya.

13.2 Dari daftar perintah ke graf

Eksperimen kecil sering ditulis sebagai daftar perintah:

```
generate-data  
analyze-data  
draw-figure  
write-report
```

Urutan itu menyembunyikan alasan setiap langkah. Graf dependensi menyatakan bahwa analisis memerlukan data, gambar memerlukan analisis, dan laporan memerlukan analisis serta gambar. Dengan informasi itu, pengelola alur kerja dapat menentukan urutan yang sah dan pekerjaan yang perlu diulang.

Graf harus asiklik. Jika **analysis** memerlukan **report**, sementara **report** memerlukan **analysis**, tidak ada tugas pertama yang dapat diselesaikan.

13.3 Sumber dan artefak

- **Sumber** ditulis atau diperoleh dari luar alur kerja: kode, konfigurasi, data mentah, dan lisensi.
- **Produk antara** dapat dibuat ulang tetapi membantu langkah berikutnya.
- **Artefak akhir** diserahkan kepada pembaca atau peninjau.

Jangan menjadikan artefak akhir sebagai satu-satunya salinan data sumber. Jika produk antara mahal dibuat, ia boleh disimpan dalam tembok, tetapi hash dan aturan pembuatannya tetap harus diketahui.

13.4 Kapan tugas dianggap mutakhir

Cap waktu berguna untuk optimisasi lokal, tetapi dapat berubah ketika berkas disalin atau jam mesin tidak cocok. Rekaman yang kuat mengikat:

1. hash byte setiap masukan;
2. hash atau versi kode tugas;
3. parameter terkanonisasi;
4. identitas lingkungan yang relevan; dan
5. hash setiap keluaran.

Scrip Unit 10 menyimpan preimage fingerprint secara lengkap dan kanonis: nama serta SHA-256 setiap masukan, SHA-256 kode tugas, parameter, dan identitas runtime yang benar-benar menjalankan contoh. Identitas runtime memuat implementasi dan versi Python, tag cache, keluarga sistem operasi, arsitektur mesin, dan urutan byte. Ia sengaja tidak memuat nama host atau jalur profil. **task_fingerprint** adalah SHA-256 byte JSON kanonis preimage tersebut, sehingga hash dapat dihitung ulang langsung dari rekaman.

Jika salah satu masukan berubah, tugas dan seluruh keturunannya menjadi tidak mutakhir. Jika semuanya cocok dan keluaran masih mempunyai hash yang dicatat, tugas dapat dilewati secara aman. Pemeriksa juga menerima daftar nama keluaran yang diwajibkan. Pemetaan keluaran kosong, keluaran wajib yang hilang, atau keluaran tambahan yang tidak dideklarasikan dinilai tidak mutakhir. Unit ini tidak mempunyai kontrak tugas tanpa keluaran.

13.5 Gagal secara atomik

Menulis langsung ke `results.json` dapat meninggalkan berkas terpotong ketika proses berhenti. Pola yang lebih aman ialah:

1. tulis ke berkas sementara baru dalam direktori yang sama;
2. tutup dan sinkronkan bila ketahanan terhadap crash diperlukan;
3. verifikasi bentuk dan hash;
4. ganti nama sementara menjadi nama tujuan secara atomik; dan
5. jangan menghapus hasil lama sampai pengganti siap.

Nama sementara harus unik dan bukan tautan yang sudah ada. Pembersihan berkas sementara adalah penanganan kegagalan lokal, bukan pengganti manifest.

13.6 Target bernama dan pemeriksaan

Alur kerja O002 sekurang-kurangnya mempunyai target:

<code>test</code>	menjalankan tes tanpa membangun pembaca
<code>results</code>	membuat artefak eksperimen
<code>html</code>	membangun pembaca web setelah test dan results
<code>pdf</code>	membangun pembaca cetak setelah test dan results
<code>qa</code>	memeriksa kedua pembaca serta manifest
<code>all</code>	menjalankan seluruh batas penerimaan

Target `all` tidak boleh menerbitkan. Penerbitan merupakan transaksi terpisah yang hanya memakai artefak yang telah melewati `qa`.

13.7 Contoh alur kerja

Skrip unit ini memvalidasi graf kecil, menghitung urutan topologis, dan membuat rekaman kanonis yang memuat preimage fingerprint, runtime aktual, serta kontrak keluaran:

```
python source/code/unit10_pipeline.py --output output/unit10-results.json
```

Contoh tidak menjalankan perintah shell. Ia memodelkan keputusan alur kerja sehingga logika urutan, siklus, dan kesegaran dapat diuji tanpa efek eksternal.

13.8 Otomasi tidak memutuskan kebenaran

Alur kerja yang hijau membuktikan bahwa pemeriksaan yang diprogram lulus pada lingkungan tersebut. Ia tidak membuktikan bahwa pemeriksaan cukup, model tepat, atau teorema benar. Otomasi mengurangi kelalaian berulang; tanggung jawab matematis tetap pada perancang dan peninjau.

13.9 Latihan

13.9.1 Latihan 1 - urutan tugas

Tugas **data** tidak mempunyai dependensi; **fit** memerlukan **data**; **plot** memerlukan **data** dan **fit**; **report** memerlukan **plot**. Berikan satu urutan yang sah.

Petunjuk

Setiap tugas harus muncul setelah seluruh dependensinya.

Jawaban dan solusi

Urutan **data**, **fit**, **plot**, **report** sah. **plot** tidak boleh mendahului **fit**, dan **report** tidak boleh mendahului **plot**. Jika ada beberapa tugas independen, lebih dari satu urutan dapat sama-sama sah.

13.9.2 Latihan 2 - siklus

Jelaskan mengapa dependensi $a \rightarrow b$, $b \rightarrow c$, dan $c \rightarrow a$ harus ditolak sebelum satu tugas pun dijalankan.

Petunjuk

Cari tugas yang seluruh dependensinya sudah selesai.

Jawaban dan solusi

Tidak ada tugas awal: **a** menunggu **b**, **b** menunggu **c**, dan **c** menunggu **a**. Menjalankan sebagian alur kerja sebelum mendeteksi siklus dapat meninggalkan artefak yang tampak sah, sehingga validasi graf harus mendahului efek.

13.9.3 Latihan 3 - cap waktu

Sebuah data disalin dengan isi sama tetapi cap waktu baru. Apakah analisis harus selalu diulang? Bandingkan kebijakan cap waktu dan hash.

Petunjuk

Cap waktu mengukur peristiwa sistem berkas, bukan isi.

Jawaban dan solusi

Kebijakan cap waktu dapat membangun ulang walaupun isi sama. Kebijakan hash dapat melewati analisis jika byte data, kode, parameter, lingkungan, dan keluaran yang dicatat semuanya tetap cocok. Pilihan hash lebih mahal dibaca, tetapi lebih langsung mengikat isi.

13.9.4 Latihan 4 - kegagalan penulisan

Mengapa menulis keluaran baru langsung di atas artefak lama berbahaya? Susun urutan penulisan yang aman.

Petunjuk

Pertimbangkan proses yang berhenti setelah setengah byte ditulis.

Jawaban dan solusi

Kegagalan dapat menghancurkan artefak lama dan meninggalkan keluaran terpotong di nama resmi. Tulis ke sementara unik pada direktori sama, tutup dan verifikasi, lalu ganti tujuan secara atomik. Jika langkah gagal, artefak lama tetap tersedia dan berkas sementara dapat didisposisikan.

13.9.5 Latihan 5 - alur kerja hijau

Semua target alur kerja lulus. Sebutkan dua alasan mengapa kesimpulan matematika masih dapat salah.

Petunjuk

Tes dapat tidak lengkap dan model dapat tidak sesuai pertanyaan.

Jawaban dan solusi

Tes mungkin tidak mencakup kasus yang membantah implementasi, dan model atau asumsi matematika dapat keliru walaupun kode melaksanakannya dengan tepat. Alur kerja hijau menguatkan reproduksibilitas pemeriksaan, bukan kelengkapan logika.

13.10 Ringkasan

- Graf dependensi menjelaskan urutan dan dampak perubahan.
- Preimage fingerprint yang lengkap membuat hash masukan, kode, parameter, dan runtime dapat dihitung ulang.
- Hash isi dan kontrak keluaran mengikat kesegaran lebih langsung daripada cap waktu; pemetaan keluaran kosong tidak dianggap mutakhir.
- Siklus dan dependensi hilang harus ditolak sebelum efek.
- Penulisan atomik mencegah keluaran setengah jadi memakai nama resmi.
- Otomasi membuat pemeriksaan konsisten, tetapi tidak menggantikan penilaian matematika.

14 Eksperimen Numerik Berprasyarat

14.1 Tujuan belajar

Setelah menyelesaikan unit ini, pembaca dapat:

- memilih metode numerik yang sesuai dengan prasyarat matematikanya;
- mencatat invarian, toleransi, batas iterasi, dan kriteria berhenti;
- membedakan galat masalah, galat diskretisasi, galat pembulatan, dan galat implementasi;
- memeriksa hasil melalui substitusi, residual, batas, atau metode pembandingan;
- menjalankan eksperimen konvergensi tanpa mengubahnya menjadi bukti umum; dan
- menolak penggunaan pustaka yang menyembunyikan asumsi domain.

Unit ini mempunyai jalur lokal. Jalur akar memerlukan A30 dan konsep fungsi kontinu. Jalur kuadratur memerlukan B30, jalur sistem linear memerlukan B40, dan jalur persamaan diferensial memerlukan B70. Pembaca boleh menunda jalur yang prasyaratnya belum dipenuhi.

! Batas inti B80

Hanya metode dan teorema, jalur A30 bagi-dua, perbandingan SciPy, sumber galat, serta Latihan 1, 2, dan latihan SciPy s01 yang menjadi inti B80 unit ini. Jalur B30, B40, dan B70 tetap tersedia sebagai ekstensi, tetapi tidak boleh dipakai sebagai bukti penyelesaian B80 sebelum prasyarat masing-masing dipenuhi.

14.2 Metode datang bersama teorema

Perintah perangkat lunak tidak menjamin bahwa masalah memenuhi asumsi metode. Metode bagi-dua, misalnya, mempertahankan selang yang kedua ujungnya mempunyai tanda berbeda. Jaminan adanya akar di dalam selang memerlukan kekontinuan. Program dapat memeriksa tanda ujung; program tidak dapat menyimpulkan kekontinuan dari beberapa sampel. Implementasi unit juga menolak titik ujung, titik tengah, atau nilai fungsi yang tidak berhingga.

Setiap eksperimen numerik menuliskan:

1. masalah dan domain;
2. asumsi matematis;

3. metode dan representasi;
4. parameter serta kriteria berhenti;
5. invarian yang diperiksa;
6. ukuran galat atau residual; dan
7. batas kesimpulan.

14.3 Jalur A30: bagi-dua

Untuk mencari akar $f(x) = 0$, mulai dengan $[a, b]$ sehingga $f(a)f(b) < 0$. Setiap langkah memilih titik tengah dan mempertahankan separuh selang yang masih mempunyai perubahan tanda.

Jika lebar awal $w_0 = b - a$, setelah k langkah lebarnya $w_k = w_0/2^k$. Hubungan ini memberi batas galat berdasarkan selang, bukan hanya berdasarkan nilai kecil $f(x)$.

```
from source.code.unit11_numerical import bisection

result = bisection(lambda x: x*x - 2.0, 1.0, 2.0, width_tolerance=1e-12)
(result.midpoint, result.width, result.iterations)
```

Daftar 14.1

(1.414213562372879, 9.094947017729282e-13, 40)

Nilai itu adalah pendekatan. Pernyataan eksak $x = \sqrt{2}$ berasal dari definisi akar positif persamaan $x^2 = 2$, bukan dari digit yang dicetak.

14.4 Jalur A30: pembandingan SciPy yang dapat dieksekusi

Implementasi asli membuat setiap langkah bagi-dua dapat dibaca. Implementasi kedua dari `scipy.optimize.root_scalar` memberi pembandingan yang dikembangkan secara terpisah. Keduanya menerima fungsi, selang awal, metode, dan batas iterasi yang dinyatakan, tetapi istilah toleransinya tidak boleh disamakan. `original_width_tolerance` membatasi **lebar selang akhir** implementasi asli; `scipy_xtol` dan `scipy_rtol` adalah kriteria kedekatan absolut dan relatif untuk akar yang dikembalikan SciPy. Nilainya boleh sama secara numerik pada contoh ini, tetapi maknanya tetap berbeda dan direkam terpisah.

```

from source.code.unit11_numerical import (
    SCIPY_VERSION,
    compare_bisection_with_scipy,
    square_minus_two,
)

perbandingan_u11 = compare_bisection_with_scipy(
    square_minus_two,
    1.0,
    2.0,
    original_width_tolerance=1e-12,
    scipy_xtol=1e-12,
)

assert perbandingan_u11["scipy_version"] == SCIPY_VERSION
assert perbandingan_u11["verification"]["strict_sign_change"]
assert perbandingan_u11["scipy"]["converged"]
assert perbandingan_u11["verification"]["roots_inside_initial_bracket"]
assert perbandingan_u11["verification"]["roots_agree"]

(
    perbandingan_u11["scipy_version"],
    perbandingan_u11["original"]["midpoint"],
    perbandingan_u11["scipy"]["root"],
    perbandingan_u11["verification"]["absolute_root_difference"],
)

```

Daftar 14.2

('1.15.2', 1.414213562372879, 1.4142135623724243, 4.547473508864641e-13)

Rekaman menyimpan versi SciPy yang benar-benar diimpor, bukan sekadar nama pustaka. `strict_sign_change` memeriksa $f(a)f(b) < 0$ sebelum pemanggilan. `converged` dan `flag` mencatat laporan SciPy. Kedua akar harus tetap berada dalam selang awal dan selisihnya tidak boleh melebihi toleransi kesepakatan yang direkam.

Kesepakatan dua implementasi meningkatkan peluang mendeteksi kesalahan kode lokal. Ia tidak membuktikan bahwa fungsi kontinu, tidak membuktikan bahwa selang memuat tepat satu akar, dan tidak mengubah digit pendekatan menjadi nilai eksak. Kekontinuan dan jaminan keberadaan berasal dari asumsi serta Teorema Nilai Antara; pemeriksaan versi dan konvergensi adalah bukti kerja komputasional.

14.5 Jalur B30: kuadratur dan pemurnian

 Gate B30 - ekstensi yang ditunda

Bagian kuadratur dan Latihan 3 memerlukan B30. Keduanya boleh dijalankan untuk pengayaan setelah prasyarat tersedia, tetapi tidak dihitung sebagai inti atau bukti penyelesaian B80.

Aturan trapesium majemuk mengganti kurva pada setiap subselang dengan ruas garis. Eksperimen pemurnian memakai $n, 2n, 4n, \dots$ subselang dan mencatat perubahan hasil. Nilai yang tampak stabil mendukung dugaan konvergensi, tetapi tidak menggantikan analisis galat.

Untuk fungsi linear, aturan trapesium eksak dalam aritmetika eksak. Kasus itu menjadi pemeriksaan implementasi yang lebih kuat daripada angka referensi acak.

14.6 Jalur B40: sistem linear dan residual

 Gate B40 - ekstensi yang ditunda

Bagian sistem linear dan Latihan 4 memerlukan B40. Keduanya tidak dihitung sebagai inti atau bukti penyelesaian B80.

Jika program mengusulkan $\hat{x}$ bagi $Ax = b$, hitung residual $r = b - A\hat{x}$. Residual kecil berarti persamaan hampir dipenuhi pada skala yang dipakai. Residual kecil belum selalu berarti galat solusi kecil; hubungannya dipengaruhi kondisi matriks.

Untuk sistem berukuran dua yang koefisiennya rasional, solusi eksak dengan `Fraction` dapat menjadi pembandingan independen bagi jalur titik-mengambang.

14.7 Jalur B70: langkah Euler

 Gate B70 - ekstensi yang ditunda

Bagian persamaan diferensial dan Latihan 5 memerlukan B70. Keduanya tidak dihitung sebagai inti atau bukti penyelesaian B80.

Untuk masalah $y' = g(t, y)$, langkah Euler memakai

$$y_{k+1} = y_k + hg(t_k, y_k).$$

Mengurangi h dan membandingkan dengan solusi yang diketahui dapat mengungkap pola galat pada contoh. Itu menguji implementasi dan perilaku kasus, bukan membuktikan orde konvergensi bagi semua fungsi g .

14.8 Empat sumber ketidakcocokan

Ketika hasil tidak cocok dengan harapan, pisahkan:

- **galat masalah:** model atau data tidak menggambarkan pertanyaan;
- **galat diskretisasi:** metode mengganti masalah kontinu dengan masalah berhingga;
- **galat pembulatan:** representasi titik-mengambang mengubah operasi; dan
- **galat implementasi:** kode tidak melaksanakan metode yang dimaksud.

Mengubah toleransi hanya menanggapi sebagian sumber tersebut. Karena itu, laporan tidak boleh menyebut satu angka “galat numerik” tanpa definisi.

14.9 Artefak eksperimen

Jalankan seluruh contoh lokal yang prasyaratnya telah dipenuhi:

```
python source/code/unit11_numerical.py --output output/unit11-results.json
```

Keluaran mencatat selang akhir bagi-dua, tabel pemurnian kuadratur, solusi dan residual sistem, serta galat langkah Euler. Identitas fungsi, domain atau data awal, toleransi, kisi pemurnian, banyak langkah, dan nilai referensi yang dipakai pemeriksaan disimpan bersama hasil.

Rekaman juga memuat pembanding SciPy, versi SciPy, pemeriksaan selang, konvergensi, kesepakatan hasil, dan batas bukti. Tabel kuadratur, sistem linear, serta Euler dipertahankan sebagai regresi ekstensi; keberadaannya dalam JSON tidak menjadikannya bagian inti B80. Medan `curriculum_routes` menyatakan pemisahan itu secara eksplisit.

14.10 Latihan

14.10.1 Latihan 1 - invarian bagi-dua

Sebutkan dua invarian yang harus diperiksa pada setiap langkah bagi-dua dan jelaskan mengapa nilai tengah saja tidak cukup.

Petunjuk

Perhatikan perubahan tanda dan penyusutan selang.

Jawaban dan solusi

Ujung selang harus tetap mengurung perubahan tanda, dan lebar selang harus berkurang setengah pada setiap langkah. Nilai tengah saja tidak membuktikan bahwa akar masih terkurung atau bahwa implementasi memilih separuh yang benar.

14.10.2 Latihan 2 - batas iterasi

Berapa langkah minimum yang menjamin lebar selang paling besar 10^{-6} jika lebar awal 1?

Petunjuk

Cari k sehingga $2^{-k} \leq 10^{-6}$.

Jawaban dan solusi

Karena $2^{19} = 524288 < 10^6$ dan $2^{20} = 1048576 > 10^6$, diperlukan 20 langkah. Setelah 20 langkah, lebarnya $2^{-20} < 10^{-6}$.

14.10.3 Latihan SciPy - dua implementasi pada akar kubik

Gunakan `compare_bisection_with_scipy` untuk mencari akar $f(x) = x^3 - x - 2$ pada $[1, 2]$ dengan toleransi lebar 10^{-10} . Verifikasi perubahan tanda, versi SciPy, laporan konvergensi, letak kedua hasil dalam selang, dan kesepakatan akar. Jelaskan satu hal yang tetap memerlukan argumen matematis.

Petunjuk

Definisikan fungsi Python terlebih dahulu. Hasil perbandingan mempunyai medan `endpoint_values`, `scipy_version`, `scipy`, `original`, dan `verification`.

! Pemeriksaan mandiri

```
from source.code.unit11_numerical import SCIPY_VERSION,
compare_bisection_with_scipy

def fungsi_kubik_u11(x):
    return x**3 - x - 2

hasil_kubik_u11 = compare_bisection_with_scipy(
    fungsi_kubik_u11,
    1.0,
    2.0,
    original_width_tolerance=1e-10,
    scipy_xtol=1e-10,
)

assert hasil_kubik_u11["endpoint_values"][0] < 0
assert hasil_kubik_u11["endpoint_values"][1] > 0
assert hasil_kubik_u11["scipy_version"] == SCIPY_VERSION
assert hasil_kubik_u11["scipy"]["converged"]
assert hasil_kubik_u11["verification"]["roots_inside_initial_bracket"]
assert hasil_kubik_u11["verification"]["roots_agree"]
assert (
    hasil_kubik_u11["verification"]["absolute_root_difference"]
    <= hasil_kubik_u11["verification"]["agreement_tolerance"]
)
```

💡 Jawaban dan solusi

Pemecahan 14.1.

```

from source.code.unit11_numerical import SCIPY_VERSION,
compare_bisection_with_scipy

def fungsi_kubik_solusi_u11(x):
    return x**3 - x - 2

solusi_kubik_u11 = compare_bisection_with_scipy(
    fungsi_kubik_solusi_u11,
    1.0,
    2.0,
    original_width_tolerance=1e-10,
    scipy_xtol=1e-10,
)

verifikasi_kubik_u11 = solusi_kubik_u11["verification"]
assert solusi_kubik_u11["method"] == "bisect"
assert solusi_kubik_u11["scipy_version"] == SCIPY_VERSION
assert solusi_kubik_u11["scipy"]["flag"] == "converged"
assert verifikasi_kubik_u11["strict_sign_change"]
assert verifikasi_kubik_u11["roots_inside_initial_bracket"]
assert verifikasi_kubik_u11["roots_agree"]

```

Karena $f(1) = -2$ dan $f(2) = 4$, ujungnya mempunyai tanda berbeda. Dengan asumsi f kontinu, Teorema Nilai Antara menjamin sedikitnya satu akar dalam selang. Kedua implementasi sepakat pada toleransi yang direkam dan SciPy melaporkan konvergensi. Namun, kekontinuan serta keunikan akar tidak dibuktikan oleh dua program; keduanya memerlukan argumen matematika terpisah.

14.10.4 Latihan 3 - pemeriksaan kuadratur

Status kurikulum: ekstensi B30 yang ditunda; bukan inti B80.

Gunakan aturan trapesium untuk $f(x) = 3x + 2$ pada $[0, 4]$ dengan satu subselang. Bandingkan dengan integral eksak.

i Petunjuk

Luas trapesium adalah lebar dikali rata-rata tinggi ujung.

Jawaban dan solusi

Nilai ujungnya 2 dan 14, sehingga luas trapesium $4(2 + 14)/2 = 32$. Integral eksak $[3x^2/2 + 2x]_0^4 = 24 + 8 = 32$. Kecocokan eksak ini diharapkan karena grafik fungsi linear sendiri merupakan ruas garis.

14.10.5 Latihan 4 - residual bukan galat solusi

Status kurikulum: ekstensi B40 yang ditunda; bukan inti B80.

Jelaskan mengapa residual kecil tidak otomatis menjamin bahwa solusi hampiran dekat dengan solusi eksak.

Petunjuk

Pikirkan sistem yang sangat peka terhadap perubahan kecil pada ruas kanan.

Jawaban dan solusi

Residual mengukur seberapa baik $\hat{x}$ memenuhi persamaan yang diberikan. Pada matriks berkondisi buruk, perubahan kecil pada ruas kanan dapat berkaitan dengan perubahan besar pada solusi. Karena itu residual harus dibaca bersama skala dan informasi kondisi, bukan sebagai galat solusi langsung.

14.10.6 Latihan 5 - eksperimen Euler

Status kurikulum: ekstensi B70 yang ditunda; bukan inti B80.

Untuk $y' = y$, $y(0) = 1$, bandingkan galat pada $t = 1$ untuk $h = 1/10$ dan $h = 1/20$. Kesimpulan apa yang boleh dibuat?

Petunjuk

Gunakan solusi referensi e^t dan jangan menaikkan dua kasus menjadi teorema.

Jawaban dan solusi

Langkah Euler memberi $(1 + h)^{1/h}$. Untuk $h = 0,1$ nilainya sekitar 2,5937; untuk $h = 0,05$ sekitar 2,6533; sedangkan $e \approx 2,7183$. Galat berkurang pada dua kasus ini. Hasil tersebut mendukung perilaku yang diharapkan dan memeriksa implementasi, tetapi dua ukuran

langkah tidak membuktikan konvergensi atau orde galat secara umum.

14.11 Ringkasan

- Metode numerik datang bersama asumsi dan teorema.
- Implementasi asli dan `scipy.optimize.root_scalar(method="bisect")` memberi pemeriksaan silang A30 yang harus merekam versi, selang, toleransi, konvergensi, dan kesepakatan hasil.
- Kriteria berhenti harus dinyatakan sebelum hasil dilihat.
- Residual, batas selang, dan kasus eksak memberi pemeriksaan yang berbeda.
- Galat masalah, diskretisasi, pembulatan, dan implementasi tidak boleh dicampur.
- Eksperimen pemurnian memberi bukti kerja; bukti konvergensi memerlukan argumen matematika.
- Kuadratur B30, sistem linear B40, dan Euler B70 tetap merupakan ekstensi dan tidak boleh dihitung sebagai inti B80.

15 Proyek Akhir yang Dapat Diperiksa

15.1 Tujuan belajar

Setelah menyelesaikan unit ini, pembaca dapat:

- mengubah pertanyaan matematika menjadi eksperimen dengan batas klaim yang dinyatakan;
- memisahkan klaim matematika, klaim implementasi, dan pengamatan empiris;
- menyerahkan sumber, data, lingkungan, perintah, tes, dan artefak yang saling terikat oleh hash;
- meminta orang lain mereproduksi hasil tanpa petunjuk lisan tambahan;
- menanggapi tinjauan substantif dengan perubahan atau alasan yang dapat diperiksa; dan
- menjelaskan bagian proyek yang merupakan bukti dan bagian yang hanya merupakan bukti kerja komputasional.

Prasyarat lokal: seluruh unit inti O002 yang relevan bagi proyek dan seluruh prasyarat matematika dari topik yang dipilih. Proyek tidak boleh memakai topik matematika lanjut hanya karena pustaka dapat menghitungnya.

15.2 Pertanyaan sebelum perangkat

Proyek dimulai dari pertanyaan, bukan dari keinginan memakai suatu pustaka. Pertanyaan yang baik menentukan objek, domain, besaran yang diamati, dan bentuk jawaban yang dianggap bermakna. Contoh:

Bagaimana galat evaluasi suatu bentuk aljabar berubah ketika masukan mendekati titik pembatalan, dan reformulasi mana yang mempertahankan lebih banyak angka bermakna pada rentang yang ditentukan?

Pertanyaan itu belum menjanjikan teorema universal. Ia memberi wilayah eksperimen, perbandingan metode, dan ukuran galat yang dapat diaudit.

15.3 Tiga jenis klaim

Setiap kesimpulan proyek diberi jenis berikut.

Jenis	Contoh	Bukti yang diperlukan
matematika	dua rumus setara pada domain tertentu	argumen simbolik dengan domain dan pengecualian
implementasi	fungsi mengembalikan residu pada batas yang ditetapkan	inspeksi, tes, dan kasus tepi
empiris	metode A lebih akurat pada kisi uji tertentu	data, parameter, lingkungan, ukuran galat, dan analisis

Kesalahan paling serius ialah menaikkan pengamatan empiris menjadi klaim matematika tanpa argumen. Kesalahan lain ialah memberi bukti matematika yang benar tetapi tidak memeriksa apakah program benar-benar mengimplementasikannya.

15.4 Paket reproduksi minimum

Paket proyek minimum mempunyai struktur berikut.

```
project/  
  README.md  
  LICENSES/  
  environment/  
  source/  
  tests/  
  inputs/  
  outputs/  
  RUN_MANIFEST.json
```

`RUN_MANIFEST.json` mencatat versi skema; `command` sebagai larik argumen; `environment` dengan runtime, versi runtime, dan dependensi; objek `parameters`; daftar `inputs` dan `outputs`; rekaman ukuran dan hash dalam `artifacts`; serta larik `claims`. Setiap jalur masukan dan keluaran harus menunjuk satu artefak terdaftar. Setiap objek klaim memuat `id`, `kind`, `statement`, `evidence`, dan `limitations`, dan nilai `evidence` harus menunjuk keluaran terdaftar. Dengan demikian, hash artefak juga mengikat bukti klaim. Manifest harus memuat sekurangnya satu artefak, satu keluaran, dan satu klaim, dan jalur tidak boleh berulang. Jalur harus berbentuk POSIX kanonis dan relatif terhadap akar paket; jalur profil mesin tidak boleh menjadi bagian publik.

Skrip pemeriksa unit ini menerima sebuah manifest dan akar paket:

```
python source/code/unit12_capstone.py verify
```

Perintah ringkas itu memakai tata letak baku di atas: paket `project`, manifest `project/RUN_MANIFEST.json`, dan laporan `output/unit12-results.json`. Opsi `--root`, `--manifest`, dan `--output` tersedia jika tata letaknya berbeda.

Pemeriksa tidak menentukan apakah argumen matematika benar. Ia memastikan bahwa berkas yang disebut ada, ukurannya dan hash-nya cocok, jalurnya tidak keluar dari paket, kontrak eksekusi mempunyai bentuk minimum, klaim mempunyai medan minimum, dan setiap bukti klaim menunjuk keluaran yang benar-benar telah diperiksa. Pemeriksa tidak menjalankan `command`, menilai isi README dan lisensi, atau membuktikan bahwa struktur folder minimum sudah memadai; hal-hal itu tetap diperiksa melalui reproduksi buta dan rubrik.

15.5 Reproduksi buta terbatas

Berikan paket kepada peninjau yang tidak ikut membuatnya. Peninjau mengikuti README dan merekam:

1. lingkungan yang benar-benar dipasang;
2. perintah yang dijalankan;
3. hash masukan dan keluaran;
4. tes yang lulus atau gagal;
5. perbedaan hasil; dan
6. instruksi yang ambigu atau tidak lengkap.

“Berhasil pada komputer pembuat” bukan standar reproduksi. Sebaliknya, perbedaan antarmesin tidak selalu berarti penelitian salah. Perbedaan harus ditelusuri ke versi, platform, nondeterminisme, toleransi, atau cacat paket.

15.6 Tinjauan dan revisi

Tinjauan substantif menantang klaim, metode, domain, tes, visualisasi, atau reproduksibilitas. Tanggapan proyek memuat:

- ringkasan masalah yang diajukan;
- keputusan menerima, menerima sebagian, atau menolak;
- perubahan byte atau argumen yang dibuat;
- bukti baru yang ditambahkan; dan
- keterbatasan yang masih tersisa.

Menolak komentar diperbolehkan, tetapi “hasil komputer saya sama” bukan alasan yang cukup ketika komentar menyangkut validitas klaim.

15.7 Rubrik kelulusan

Proyek lulus hanya jika seluruh syarat berikut terpenuhi:

1. pertanyaan dan domain eksplisit;
2. sumber, data, dan hak komponen jelas;
3. lingkungan dan perintah dapat dipasang serta dijalankan ulang;
4. tes mencakup invarian dan kasus tepi yang relevan;
5. artefak dan manifest saling cocok;
6. visualisasi mempunyai label, unit, skala, dan uraian;
7. klaim matematika, implementasi, dan empiris tidak dicampur;
8. kesimpulan menyatakan keterbatasan;
9. reproduksi buta terbatas didokumentasikan; dan
10. satu tanggapan terhadap tinjauan substantif disertakan.

Kegagalan satu syarat tidak selalu membatalkan seluruh penelitian, tetapi proyek belum memenuhi standar O002 sampai kekurangan itu diperbaiki atau didisposisikan secara terbuka.

15.8 Latihan

15.8.1 Latihan 1 - klasifikasi klaim

Klasifikasikan pernyataan berikut: “Pada Python 3.13 di Windows, algoritme A memerlukan waktu lebih singkat daripada B untuk semua 100 ukuran yang diuji.” Apakah pernyataan tersebut membuktikan A selalu lebih cepat?

Petunjuk

Perhatikan platform, versi, dan himpunan ukuran.

Jawaban dan solusi

Itu pengamatan empiris yang dibatasi lingkungan dan 100 ukuran uji. Pernyataan tersebut tidak membuktikan A selalu lebih cepat pada semua ukuran, data, platform, versi, atau implementasi. Kesimpulan harus mempertahankan batas itu.

15.8.2 Latihan 2 - manifest yang belum lengkap

Sebuah manifest mencatat nama skrip dan gambar keluaran, tetapi tidak mencatat parameter, versi pustaka, atau hash data. Sebutkan tiga kegagalan reproduksi yang dapat timbul.

Petunjuk

Tanyakan apakah peninjau mengetahui masukan, lingkungan, dan byte yang sama.

Jawaban dan solusi

Peninjau dapat memakai parameter berbeda, memperoleh perilaku berbeda karena versi pustaka, atau memakai data bernama sama tetapi berisi byte lain. Ketiga keadaan dapat menghasilkan gambar berbeda tanpa cara pasti untuk menentukan sumber perbedaannya.

15.8.3 Latihan 3 - jalur berbahaya

Mengapa manifest publik tidak boleh menyebut `direktori-profil>Nama/Desktop/data.csv`? Berikan bentuk pengganti.

Petunjuk

Pertimbangkan privasi dan portabilitas.

Jawaban dan solusi

Jalur itu membocorkan struktur profil lokal dan tidak ada pada mesin peninjau. Simpan data di dalam paket, misalnya `inputs/data.csv`, lalu gunakan jalur relatif terhadap akar proyek dan ikat byte-nya dengan ukuran serta SHA-256.

15.8.4 Latihan 4 - reproduksi tidak identik

Dua mesin menghasilkan nilai yang berbeda pada digit terakhir tetapi keduanya berada dalam toleransi yang ditetapkan sebelum eksperimen. Apakah reproduksi gagal?

Petunjuk

Bedakan reproduksi semantik dari identitas byte.

💡 Jawaban dan solusi

Belum tentu. Jika kontrak memperbolehkan variasi titik-mengambang dan kedua nilai memenuhi toleransi yang dibenarkan, hasil semantik dapat cocok walaupun byte numeriknya berbeda. Namun manifest tetap harus merekam platform dan versi, dan artefak yang diklaim byte-identik harus diperiksa dengan hash.

15.8.5 Latihan 5 - tanggapan tinjauan

Peninjau menunjukkan bahwa grafik memotong sumbu vertikal sehingga perbedaan kecil tampak sangat besar. Tulis tanggapan yang memadai.

i Petunjuk

Jelaskan keputusan, perubahan, dan dampaknya pada kesimpulan.

💡 Jawaban dan solusi

Akui bahwa pilihan sumbu dapat melebihkan kesan visual. Tambahkan grafik dengan rentang penuh atau penanda pemotongan yang jelas, pertahankan grafik terpotong hanya jika membantu membaca variasi lokal, dan nyatakan ukuran efek secara numerik. Perbarui kesimpulan jika klaim sebelumnya bergantung pada kesan visual yang menyesatkan, lalu rekam hash artefak baru.

15.9 Ringkasan

- Proyek dimulai dari pertanyaan dan domain, bukan perangkat.
- Klaim matematika, implementasi, dan empiris memerlukan bukti berbeda.
- Manifest dan hash mengikat artefak, tetapi tidak membuktikan teorema.
- Reproduksi buta menguji kecukupan paket dan instruksi.
- Tinjauan substantif harus dijawab dengan perubahan atau alasan yang dapat diperiksa.
- Kelulusan proyek memerlukan pekerjaan matematis, komputasional, dokumenter, dan sosial yang semuanya dapat ditinjau.